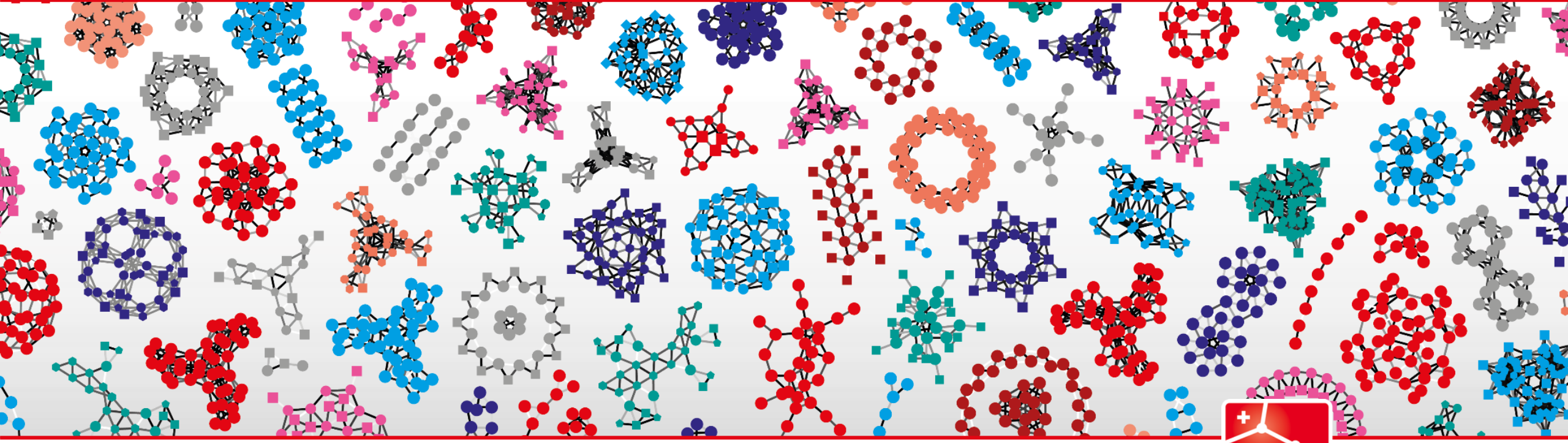


Version control with Git - first steps

Swiss Institute of Bioinformatics

www.sib.swiss

Robin Engler
Vassilios Ioannidis

Lausanne, 5-6 Nov 2026

Introduction to Git: course outline

- **Introduction** to Version Control Systems and Git.
- **Git basics**: creating a Git repository, making commits, and displaying a repository's history.
- **Git concepts**: commits, the HEAD pointer and the Git index.
- **Git branches**: introduction to branched workflows and collaborative workflow examples.
- **Branch management:** merge, rebase and cherry-pick.
- **Retrieving data from the Git database:** git checkout.
- **Working with remotes**: collaborating with Git and GitHub/GitLab.
- **GitHub/GitLab:** an overview.

Course resources

Course home page: Slides, exercises, exercise solutions, command summary (cheat sheet), setting-up your environment guide.

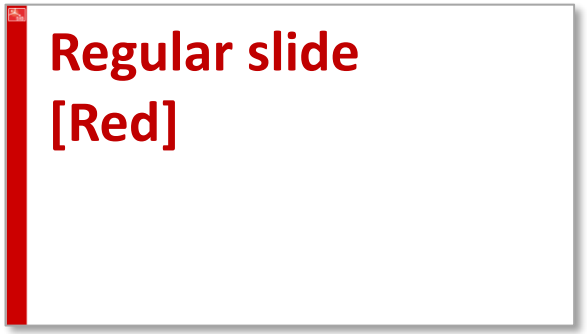
https://github.com/sib-swiss/git-training

Shared online doc: Register for the collaborative exercises (and optionally for exam), ask questions. Link sent via email before the course.

Questions: feel free to interrupt at anytime to ask questions, or use the shared online document.

Course slides

- 3 categories of slides:



Slide covered in detail during the course.





Some slides are specific to **GitHub** or **GitLab**.



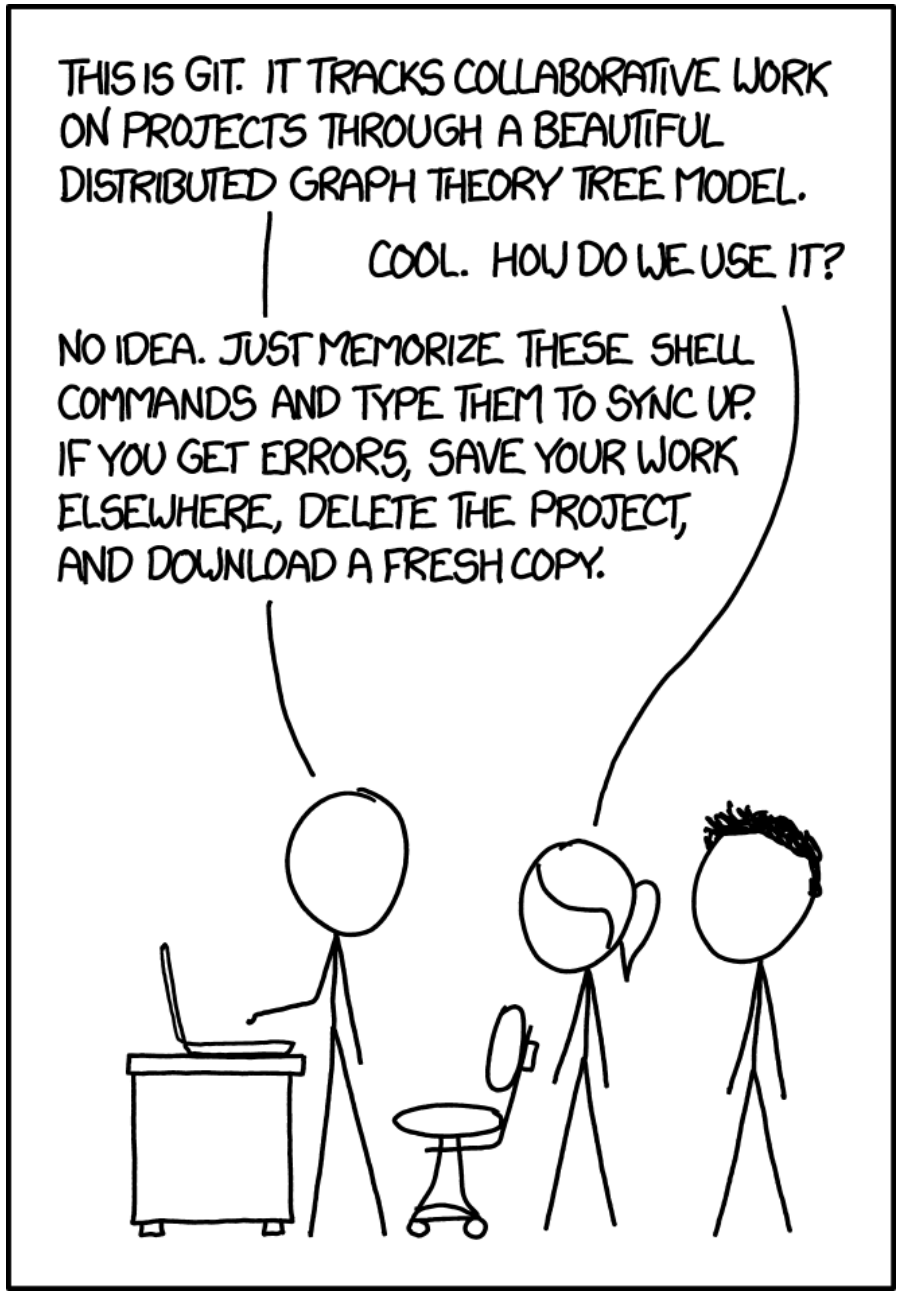
Material available for your interest, to read on your own.
Not formally covered in the course.
We are of course happy to discuss it with you if you have questions.



Material we assume you know.
Covered quickly during the course.

Learning objectives

- Learn the concepts behind Git.
- Understand when and why to use each command.
- Collaborative workflows using GitHub/GitLab.
- Learn to re-write history (day 2).



source: https://xkcd.com/1597

Command line vs. graphical interface (GUI)

- This course focuses exclusively on **Git concepts** and **command line** usage.
- Many GUI (graphical user interface) software are available for Git, often integrated with code or text editors (e.g. Rstudio, Visual Studio Code, PyCharm, …).

  It will be easy for you to start using them (if you wish to) once you know the command line usage and the concepts of Git.

version control

a (very) brief introduction

Why use version control ?

Version control systems (VCS), sometimes also referred to as *source control/code managers* (SCM), are software designed to:

- **Keep a record of changes** made to (mostly) text-based content by recording specific states of a repository's content.
- **Associate metadata to changes**, such as author, date, description, tags (e.g. version).
- **Share files** among several people and **enable collaborative work**.
- **Backup strategy**:
  - Repositories under VCS can typically be mirrored to more than one location.
  - The database allows to retrieve older versions of a document: if you delete something and end-up regretting it, the VCS can restore past content for you.
- In the case of Git, entire ecosystems such as GitHub or GitLab have emerged to offer **additional functionality**:
  - **Distribute** software and documentation.
  - **Run automated pipelines** for code testing and deployment (CI/CD).
  - **Project management features** (e.g. issue tracking, continuous integration).

A brief history of Git

- **First release in 2005**.
- Initially written by **Linus Torvald** (who also wrote the first Linux kernel in his spare time…).
- Created to support the development of the Linux kernel code (> 20 million lines of code).

The first commit of Git's own repository by Linus Torvalds in 2005.

```
commit e83c5163316f89bfbde7d9ab23ca2e25604af29
Author: Linus Torvalds <torvalds@ppc970.osdl.org>
Date: Thu Apr 7 15:13:13 2005 -0700

Initial revision of "git", the information manager
from hell
```

(some of) The principles that guided the development of Git

Linus wasn't satisfied with existing version control software, so he wrote his own…
He had the following objectives (among others) in mind:

- **Distributed development:** allow **parallel, asynchronous work** in independent repositories that do **not require constant synchronization** with a central database. **Each local Git repo is a full copy of the project** so users can work independently and offline.

- **Maintain integrity and trust:** since Git is a distributed VCS, maintaining integrity and trust between the different copies of a repositories is essential. **Git uses a blockchain-like approach to uniquely identify each change to a repository**, making it impossible (or at least very hard) to modify the history of a Git repo without other people noticing it.

- **Enforce documentation:** in Git, **each change to a repo must have an associated message**. This forces users to document their changes.

- **Easy branching/merging:** Git makes it **easy to create new branches** (i.e. lines of development) in a project. This encourages good working practices.

- **Free and open source:** users have the freedom to run, copy, distribute, study, change and improve the software.

Part I

Git basics

Working principle, definitions, and making your first commit

Git working **principles** and **definitions**

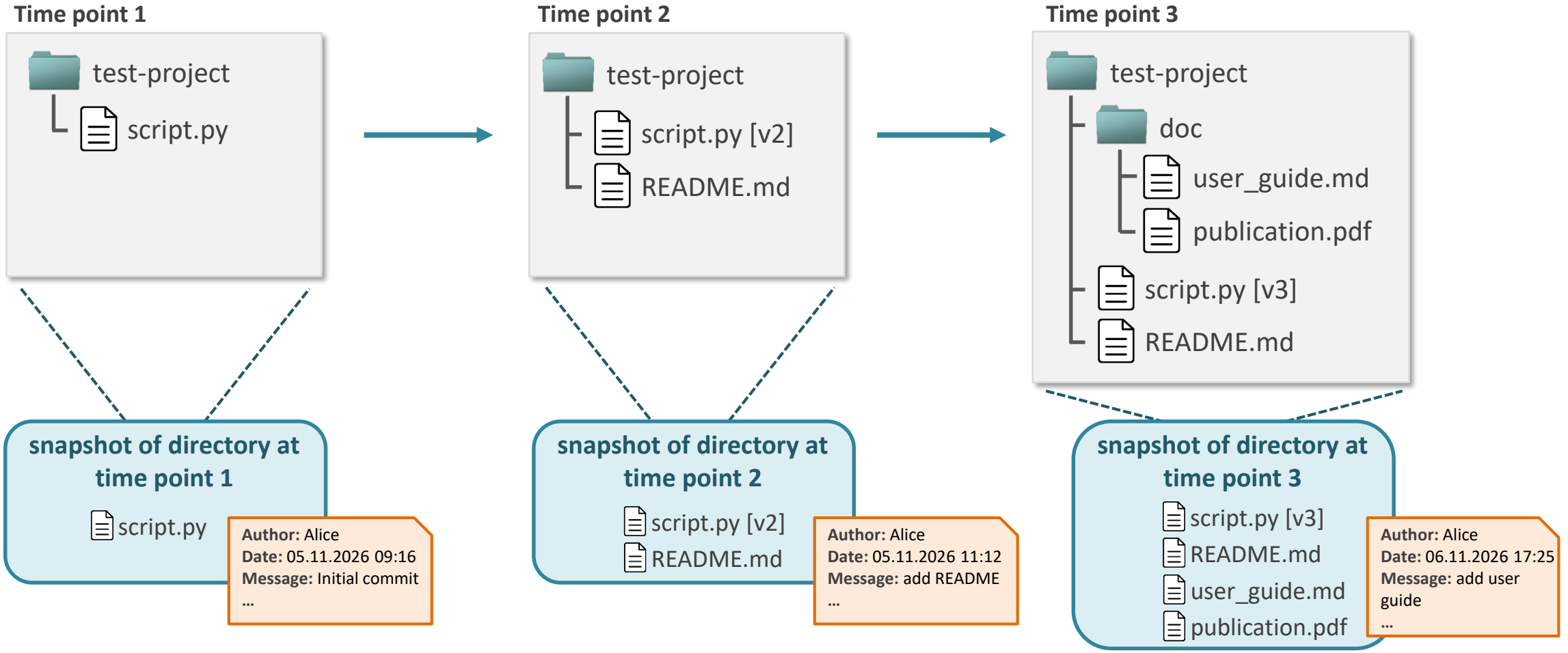
Basic principle of Git

Git can track any types of files (text or binary), but is optimized to work with not-too-large text files.

Our objective: **record the changes made to the content of a directory** on our **local machine**.

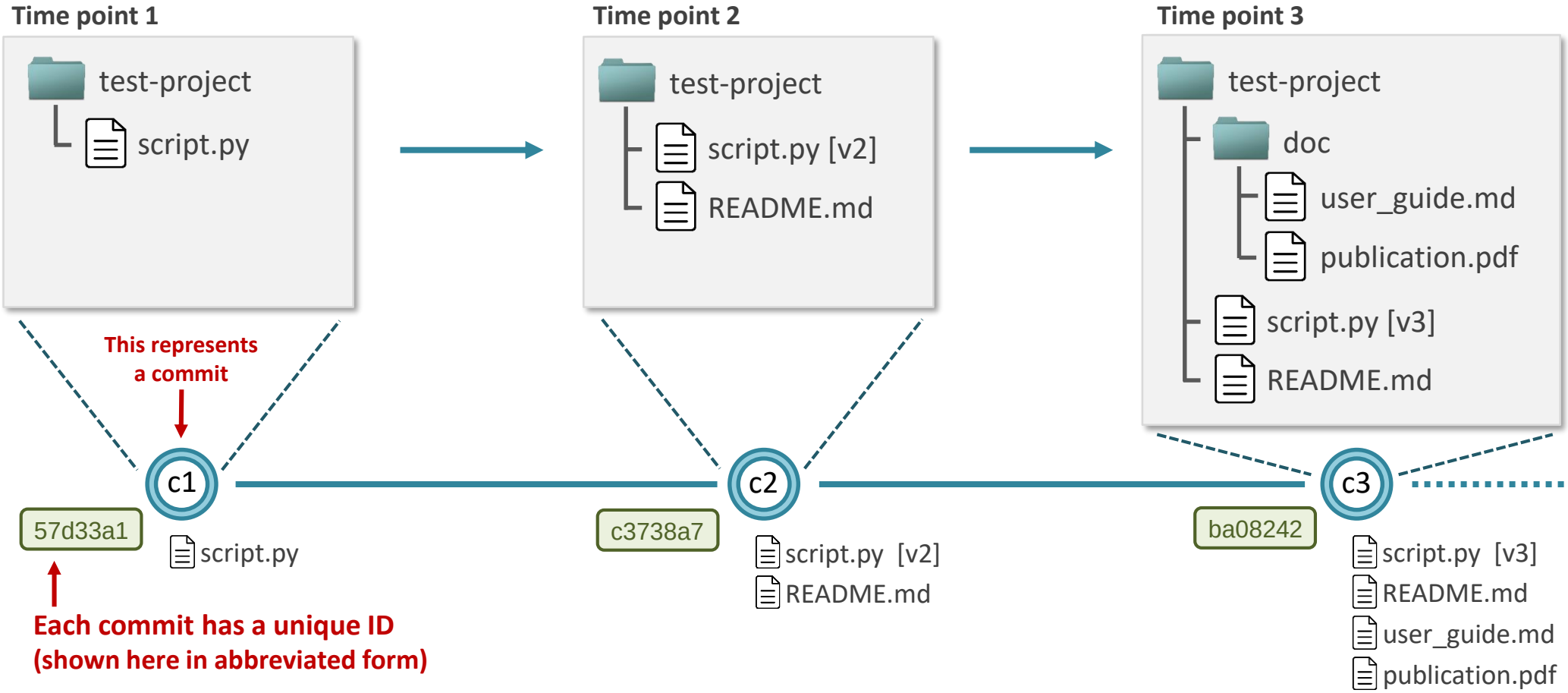
How we proceed:

- Take **snapshots** (current content of files) **at *user defined* time points – they are *not* taken automatically**.
- Keep track of **the order of snapshots** (the relation between them) so their history can be recreated.
- Associate **metadata** with each snapshot: who made it, when, description, …



Definition: snapshots are called "commits"

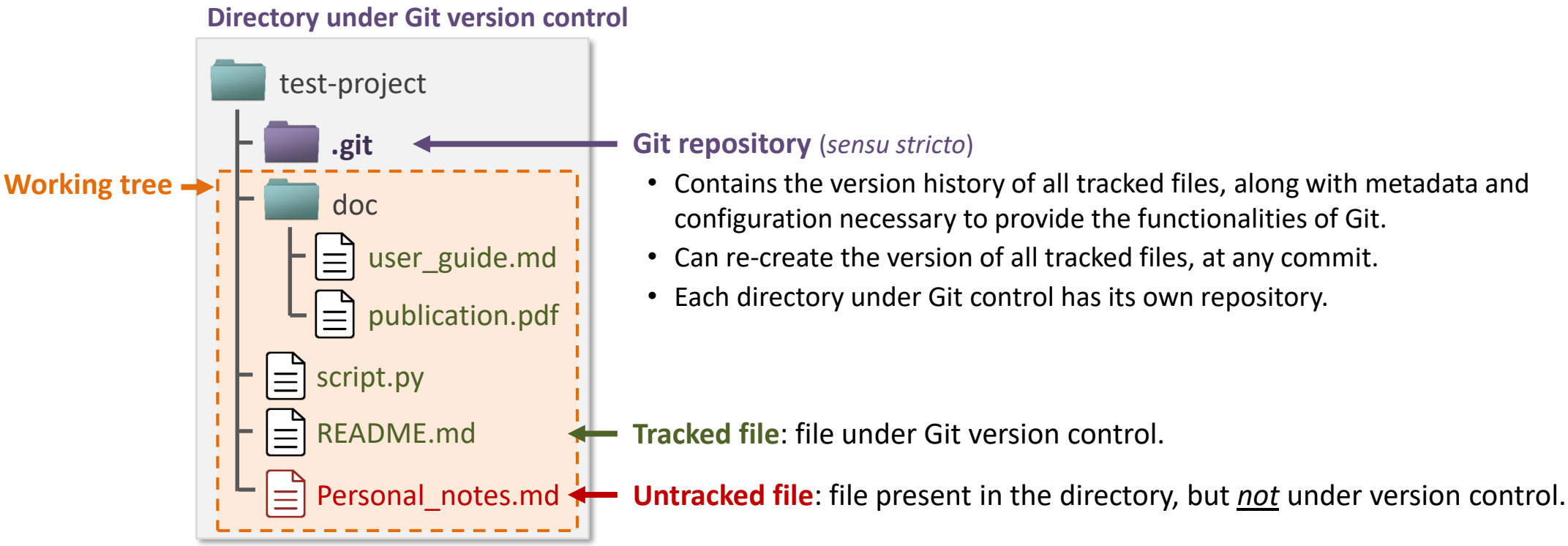
- **Commit = snapshot + metadata** (author, time, commit message, parent commit ID, etc. …).
- Create a new commit = record a new state of the directory's content.
  Note: we will soon see that this statement is *not* 100% correct, but it is a good-enough approximation for now.
- Each commit has a unique **ID number** / **hash** (40 hexadecimal characters): commit ID `3c1bb0cd5d67dddc02fae50bf56d3a3a4cbc7204`



Definition: commits are stored in a repository (or "repo")

- **Git repository/repo**: version history of a project, including snapshots of tracked files, commit history, metadata, references (branches and tags), and repository configuration.
  - Technically, a Git repository is only the hidden "`.git`" directory (see figure below), but often the term is also used to refer to the entire directory under Git control ("test-project" in the example below).
  - Not all files in a directory under Git control have to be tracked: there can be a mix of **tracked** and **untracked** files.

- **Working Tree**: files in the directory under Git control, as they currently exist on disk (on your computer).
  - More exhaustive definition: state of the project files corresponding to the branch/commit that is currently checked out, augmented with uncommitted changes made to files, as well as untracked files.



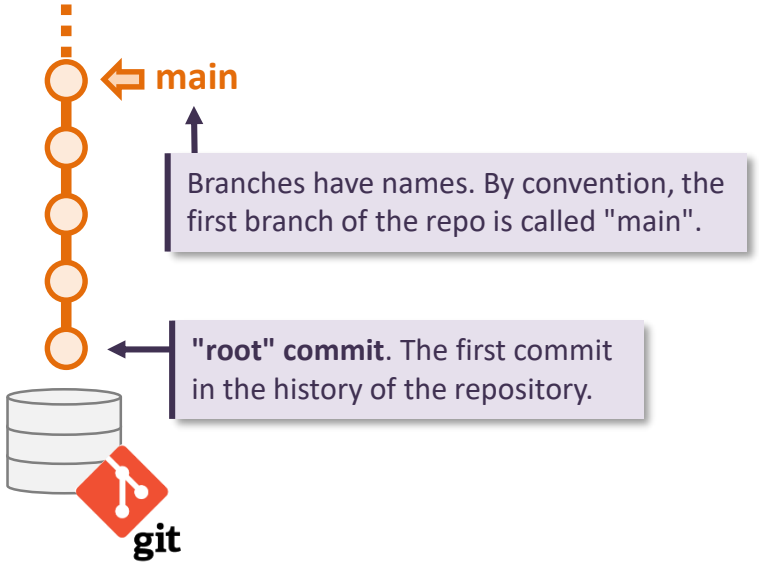
Definition: branches

- **Branch**: refers to a "**line of development**" within the commit history, a **lineage of commits** starting from the latest (newest) commit of the branch and tracing back through the history of the repo.

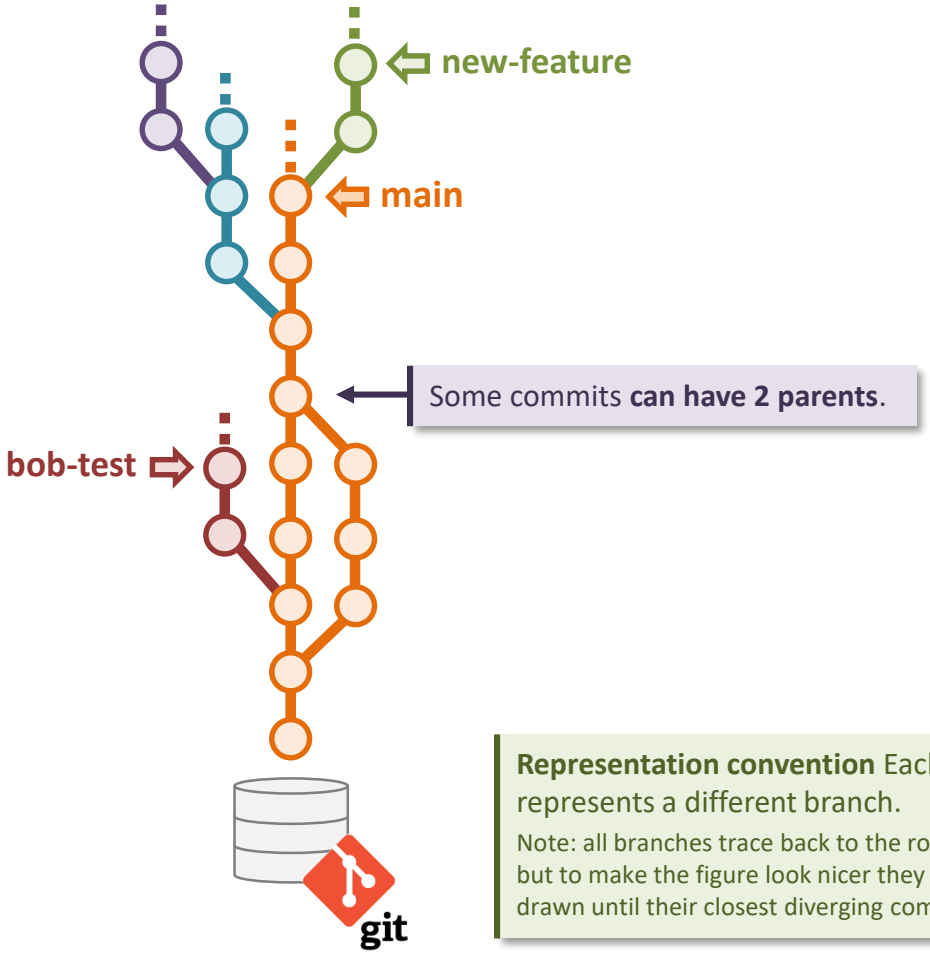
  Technically, a branch is simply a pointer (reference) to a commit.

- **Repository history**: chronology of all of commits reachable from a branch (or tag).

  Technically, a directed acyclic graph (DAG) of commits that describe how the repository content evolved over time.



Representation convention
Each circle represents a commit to the Git repo.



Representation convention Each color represents a different branch.

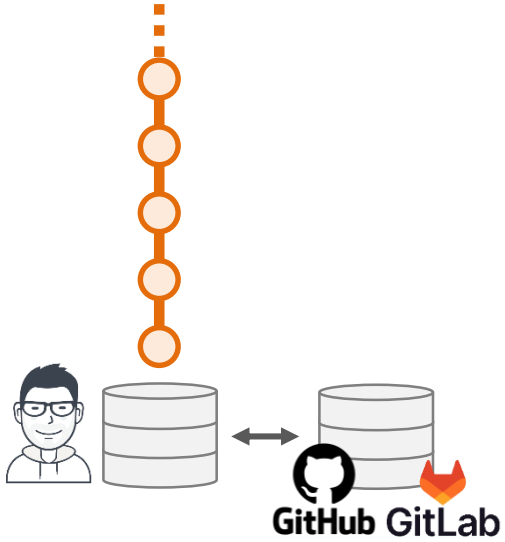
Note: all branches trace back to the root commit, but to make the figure look nicer they are only drawn until their closest diverging commit.

Note: A repository's history is not stored as a separate structure by Git. Instead, it is derived by traversing the commit graph from references such as HEAD, branches and tags.

Git use cases

Exercise 1

Single repo, single branch

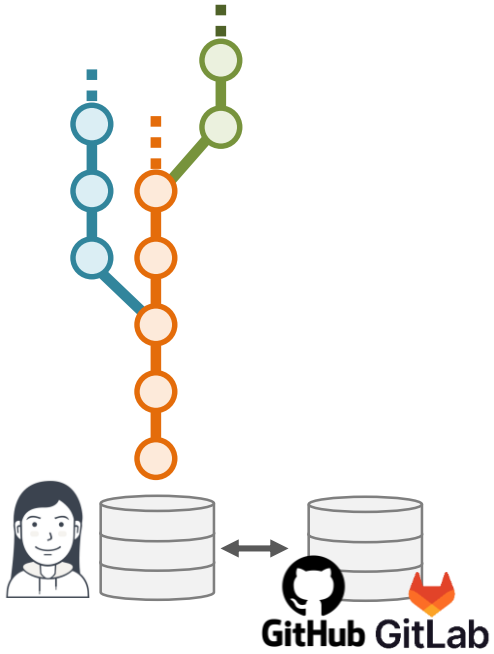


Use case

- Keep a documented log of your work.
- Go back and compare to earlier versions.
- See what changed since last commit – also very useful when working with AI agents.
- Backup (if a paired with a remote).
- Distribute your code (if paired with remote)

Exercises 2, 3, 4

Single repo, branched workflow (multiple development lines)



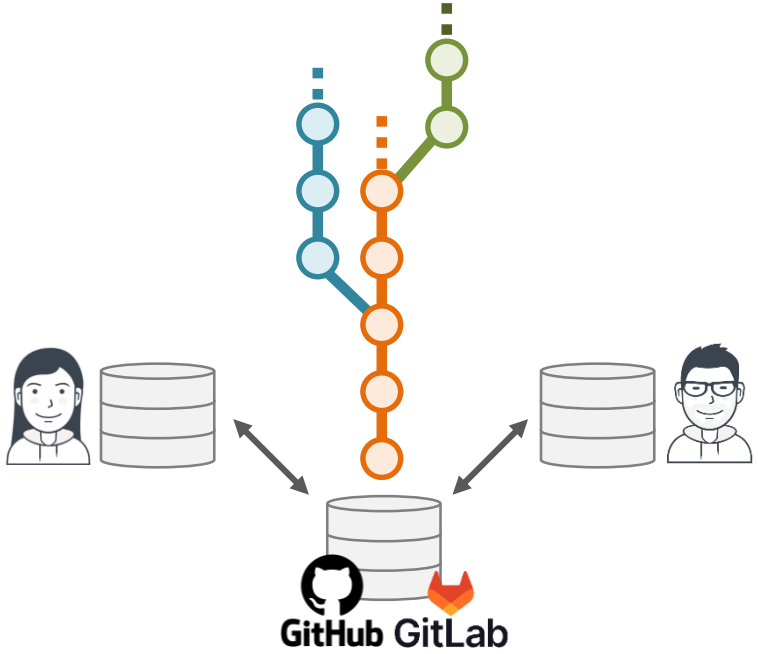
Use case

- Service in production with continued development in parallel (e.g. adding new feature).
- + all benefits of the previous use case.

The local repo must be associated to a remote repository to provide backup functionality (and new commits must be regularly pushed).

Exercise 5

Collaboration with distributed and central repos.



Use case

- Collaborate with others (distributed development).
- + all benefits of the previous use case.

Each user has a full copy of the data *

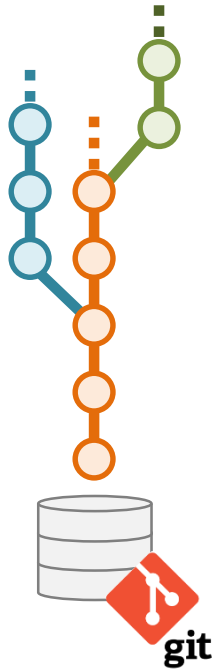
*** Provided they regularly sync their local repo.**

Local vs. Remote repository

- When creating a new Git repository on your computer, **everything is only local**.

- To get a copy of your repository online, you must take the active steps of:
  - **Creating** a new repository on a hosting service (e.g. GitHub, GitLab, Bitbucket, …).
  - **Associate** the online repository with your local repo.
  - **Push** your local content to the remote.

- By design, Git **does not automatically synchronize** a local and remote repo. Download/upload of data must be triggered by the user.



Using Git with large files: the problem

Git can store any type of file ("plain text" or binary).

It would be nice if we could store data (large files) together with code …

Tracking large files together with code is an attractive proposition, e.g. in scientific applications:

- Data analysis/processing pipeline.
- Machine learning applications (training data and code in the same place).

… but Git does not work well with large files

- Git was designed for tracking code – i.e. relatively small text files.
- Adding large files to a Git repo is technically possible, however:
  - Since Git is a distributed VCS (version control system), **each local copy of a repository will contain a full copy of all versions of all tracked files**. Therefore, **adding large files will quickly inflate the size of everyone's repository**, resulting in higher disk space usage (on local hosts).
  - **Git's internal data compression** (i.e. packfiles) is **not optimized to work with binary data** (e.g. image or video files). Each change to a binary file will (more or less) add the full size of the file to the repo, taking disk space and slowing down operations such as repo cloning or update fetching.
  - Commercial **hosting platforms impose limits on the size of files** that can be pushed to hosted Git repos (GitHub: 100 MB, GitLab: no file limit but 10 GB repo limit).

Supplementary material…

Using Git with large files: possible solutions

Git LFS (Large File Storage)

Git LFS (Large File Storage) is an extension for Git, specifically **designed to handle large files**.

- Open source project: https://git-lfs.github.com

Basic principle: large files are not stored in the Git database (the `.git` directory), instead:

- Only a **reference/pointer to large files** is stored in the Git database.
- The actual **files are stored in a separate repository** or "object store".



Not all hosting services support Git LFS, and when they do, storage space is limited (additional space may be purchased).

DVC (Data Version Control)

DVC (Data Version Control) is a software that integrates with Git (a sort of layer used on top of Git) to allow versioning and storage of large files.

- Open source project: https://dvc.org

Supplementary material...

Git **configuration**

```
git config
```

Configuring Git

The minimum configuration is setting a **user name** and **email**. These will be used as default author for each commit.

- Setting user name and email:

```
git config --global user.name <user name>
git config --global user.email <email>
```

> The `--global` option/flag tells Git to store the setting at the "global" (user wide) scope. Global settings apply to all Git repos on your machine.
>
> Without the `--global` option, the setting will only apply to the current Git repo.
>
> Global settings are stored in the following config file:
>
> - Linux: `/home/$USER/.gitconfig`
> - Mac OS: `/Users/<user name>/.gitconfig`
> - Windows: `C:/Users/<user name>/.gitconfig`

- Config values can be retrieved by using the `--get` option.

```
# Set user name and email at the global (user-wide) scope.
❯ git config --global user.name "Alice"
❯ git config --global user.email alice@redqueen.org

# Retrieve setting values.
❯ git config --get user.name
Alice
❯ git config --get user.email
alice@redqueen.org
```

Configuring Git: changing the default text editor

On most systems, the default editor that Git uses is **vim**.

However, this can be configured with the following `git config` command:

```
git config --global core.editor <editor cmd>
```

Example: changing the default editor to “nano” (another command line editor).

```
# Change the default editor to “nano”.
› git config --global core.editor nano

# Change the default editor to “VS Code”.
› git config --global core.editor "code --wait"

# Display the current default editor.
› git config --get core.editor
```

Supplementary material...

Configuring Git: scopes and their config file locations

Depending on their scope, Git configurations apply to all Git repositories of a user, or only to a specific repository.

The main 3 scopes are:

- **Global (user wide)** – settings apply to all Git repositories controlled by the user.
  - To save a setting at the global scope, add the `--global` option: `git config --global ...`
  - Stored in `/home/<user name>/.gitconfig` (Linux), `C:\Users\<user name>\.gitconfig` (Windows) or `/Users/<user name>/.gitconfig` (Mac OS).
- **Local (repo specific)** – settings apply only to a specific Git repo.
  - Stored in the `.git/config` file of the repository.
- **System (system wide)** – settings apply to all users and all repos on a given machine. This can only be modified by a system administrator.

To show the list of all Git configurations, along with their scope and the location of the file they are stored-in:

```
git config --list --show-origin --show-scope
```

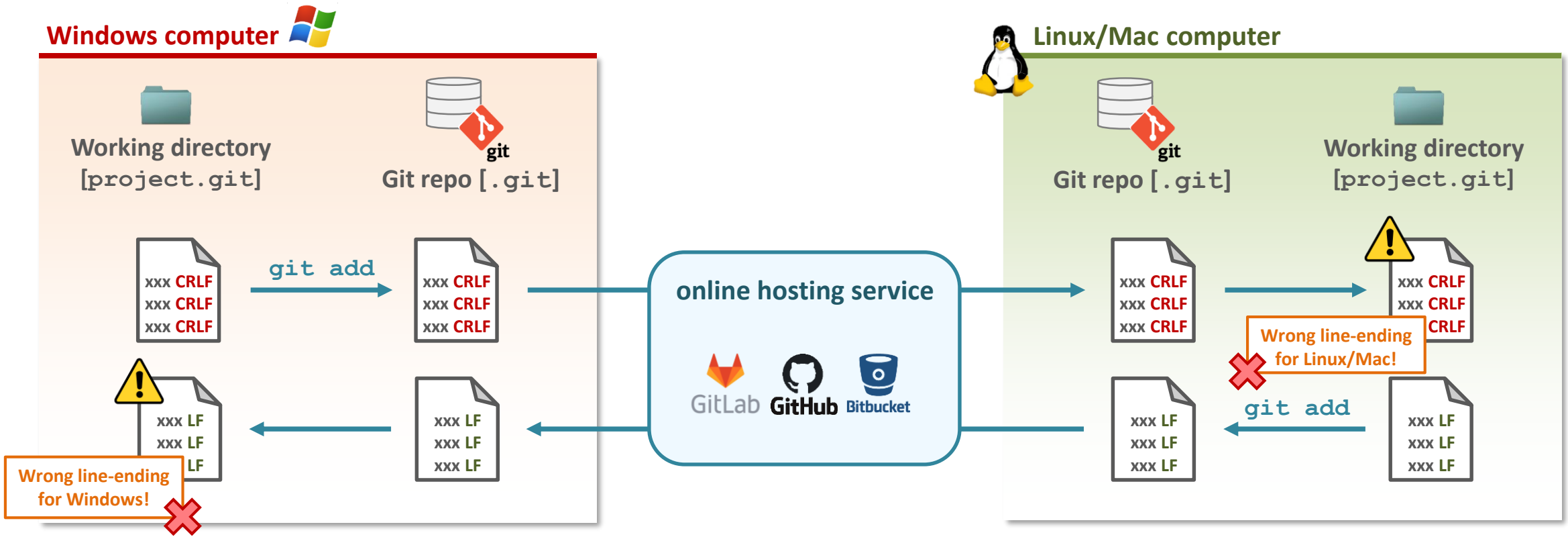
Supplementary material...

Cross-platform collaboration: the line-end problem

Linux/Mac and Windows do not use the same “line-end” characters: this can cause problems when collaborating with people who use a different operating system.

- **Linux/Mac:** uses **LF** (linefeed; `\n`) as line-ending character.
- **Windows:** uses **CRLF** (carriage-return + linefeed; `\r\n`) as line-ending character.

→ **Problem: text files created on Windows will not work well on Linux/Mac and vice versa.**



Supplementary material...

Cross-platform collaboration: solution

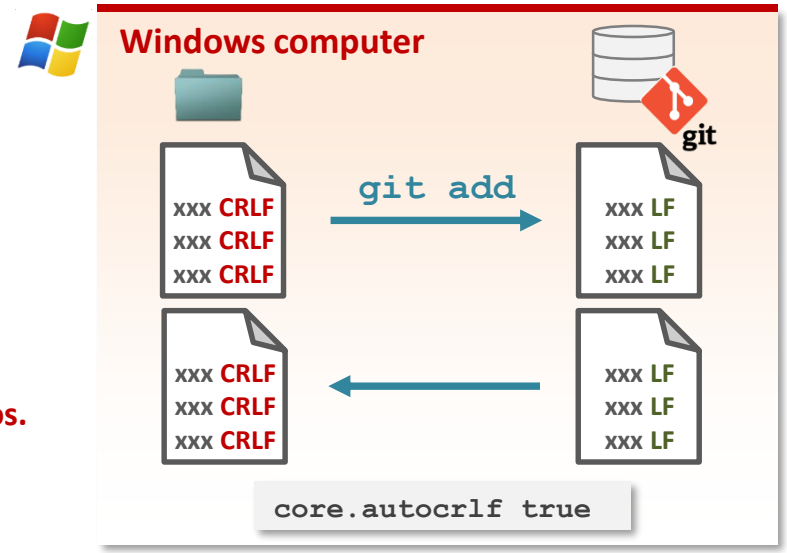
The solution is to ask Git to **automatically convert between LF and CRLF** during add/checkout operations using the configuration option:

```
git config core.autocrlf
```

- On **Windows** computers: `core.autocrlf true` should be set so that LF are automatically changed to CRLF each time a file is checked-in or checked-out.

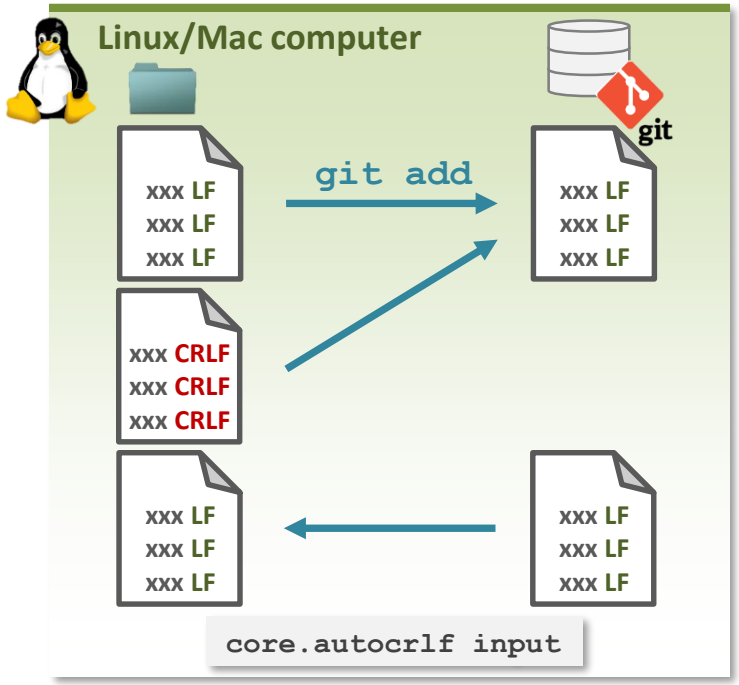
```
git config core.autocrlf true
git config --global core.autocrlf true
```

← Change setting for current repo.
← `--global` = change setting for all repos.



- On **Linux/Mac** computers: `core.autocrlf input` should be set so that LF line-endings (LF) are left untouched, and that CRLF are converted to LF when a file is added (this will only be useful in the rare cases when a file with CRLF ending is somehow present on the machine, e.g. because it was sent via email by a Windows user).

```
git config core.autocrlf input
```



- `core.autocrlf false` to disable LF/CRLF auto-modifications (this is the default):

```
git config core.autocrlf false
```

Supplementary material...

`core.autocrlf` warnings

When `core.autocrlf` is set to `True` (this is in principle only for windows users), a warning is displayed when files are added/checked-out to/from the git repo:

```
› git add test_file.py

warning: LF will be replaced by CRLF in test_file.py
The file will have its original line endings in your working directory
```

Somehow the message is the same during adding and check-out of files… so when adding files to the index (`git add`), the message is actually the wrong way round: it should be something like “CRLF will be changed to LF in checked-in file”.



Creating a **new repo**

```
git init
```

```
git clone
```

Obtaining a new Git repo: two main ways…

Turn a local directory into a Git repo
(start from scratch)

Enter the directory to version-control, then run:

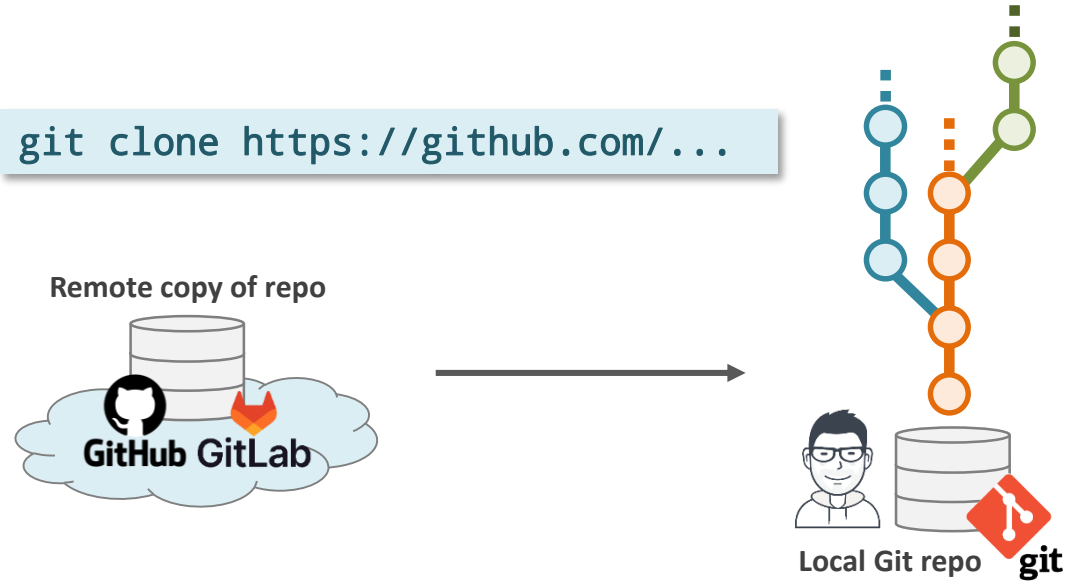
```
git init
```



- A new, empty, Git repository is created in the current directory.
- Files present in the directory can now be version-controlled. However, version-control of files is *not automatic* – more on that later.
- At this point there is *no* online remote associated with the new repo. Everything is only local.

Clone a repo from an online source
(start from an existing repo)

```
git clone https://github.com/...
```



- The entire content of the online Git repository is “cloned” (downloaded) to the local machine.
- The online repo is automatically linked (setup as a “remote”) for the local repo: we can push commits with no additional setup.
- Starting a new project on GitHub/GitLab and cloning it can also be a way to create a new empty local repository and immediately link it to a remote.

Cloning and working with remotes will be presented in more details later in these slides.

Creating a new Git repository (from scratch)

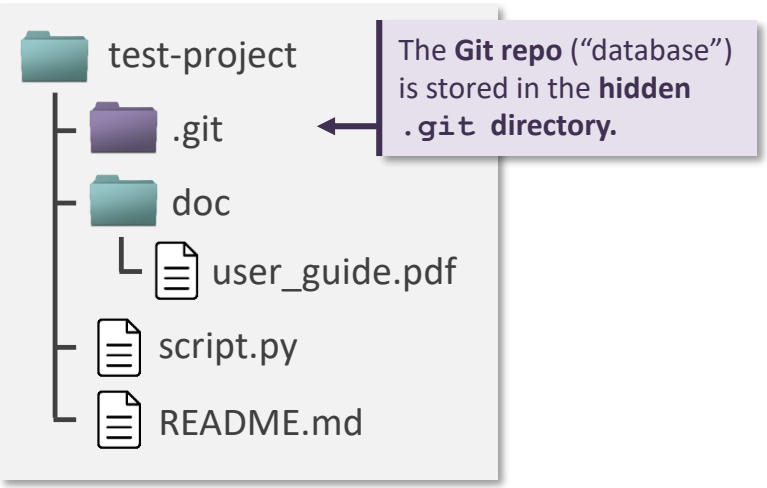
`git init` **Initializes a Git repository** in the current working directory, turning it into a version controlled directory.

Example

```
# Enter directory to version control.
❯ cd /home/alice/test-project

# Initialize a new Git repo.
❯ git init
Initialized empty Git repository in /home/alice/test-project/.git/

# List the content of our directory: we now see a new .git subdirectory.
❯ ls -a
./ ../ .git/ doc/ src/ README.md
```



- You must be located **at the root of the directory to version control** before typing `git init`
- `git init` creates a hidden `.git` directory at the root of the directory.
- **Everything** is stored in this single `.git` directory:
  - Complete version history of all tracked files.
  - All other data associated with the Git repository (e.g. branches, tags).
  - The content of `.git` can re-create the exact state of all your files at any versioned time, e.g. if you delete a file accidentally or want to go back to an earlier version.

Never delete the `.git` directory
Unless you intend to start again your repo from scratch, and accept to lose all its history.

State of the working directory (here just after `git init`)

Useful commands to assess the current status of a Git repo:

- **Show status of files** in project directory (working tree).

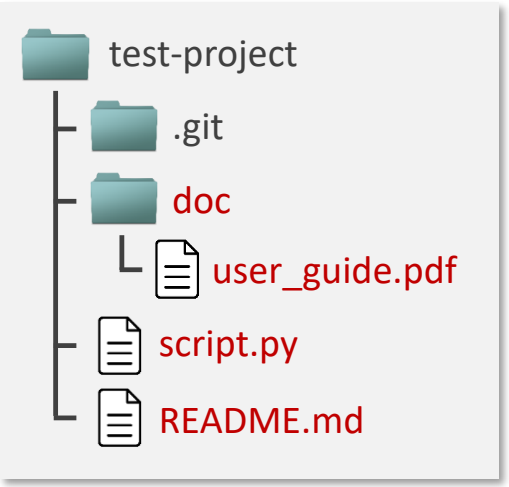
`git status`

```
> git status
On branch main
No commits yet
Untracked files:
      doc/
      README.md
      script.py
```

"**main**" is the default branch name.

Red = untracked files

How it looks in the file system



- **Commit history:** show log of commits, i.e. the history of the repo.

`git log`

```
> git status
fatal: your current branch 'main'
does not have any commits yet
```

Since we just created a new repo there are no commits yet, which is why we get this error.

- **List files currently tracked** by Git (i.e. part of the Git index).

`git ls-files`

```
> git ls-files
<empty output>
```

By default, files are untracked. This is why initially there is no tracked file.

A few more things about Git repos

- When creating a new Git repo, it does not matter whether the directory is empty or already contains files/sub-directories.
- **Files** in the project directory (working tree) **are not automatically tracked** by Git: files are **untracked by default**.
- You can have both tracked and untracked files in a project directory.
- Only files located in the project directory – or one of its sub-directories – can be tracked.
- Project directories are self-contained – you can rename them or move them around in your file system.
- You can (should) have multiple Git repositories on your system – typically one per project or per code/script you develop * - don't use a single Git repo to track the entire content of your computer!
- Nesting Git repositories (i.e. having one repo inside another) is technically possible, but should be avoided unless there is a clear use-case for it.

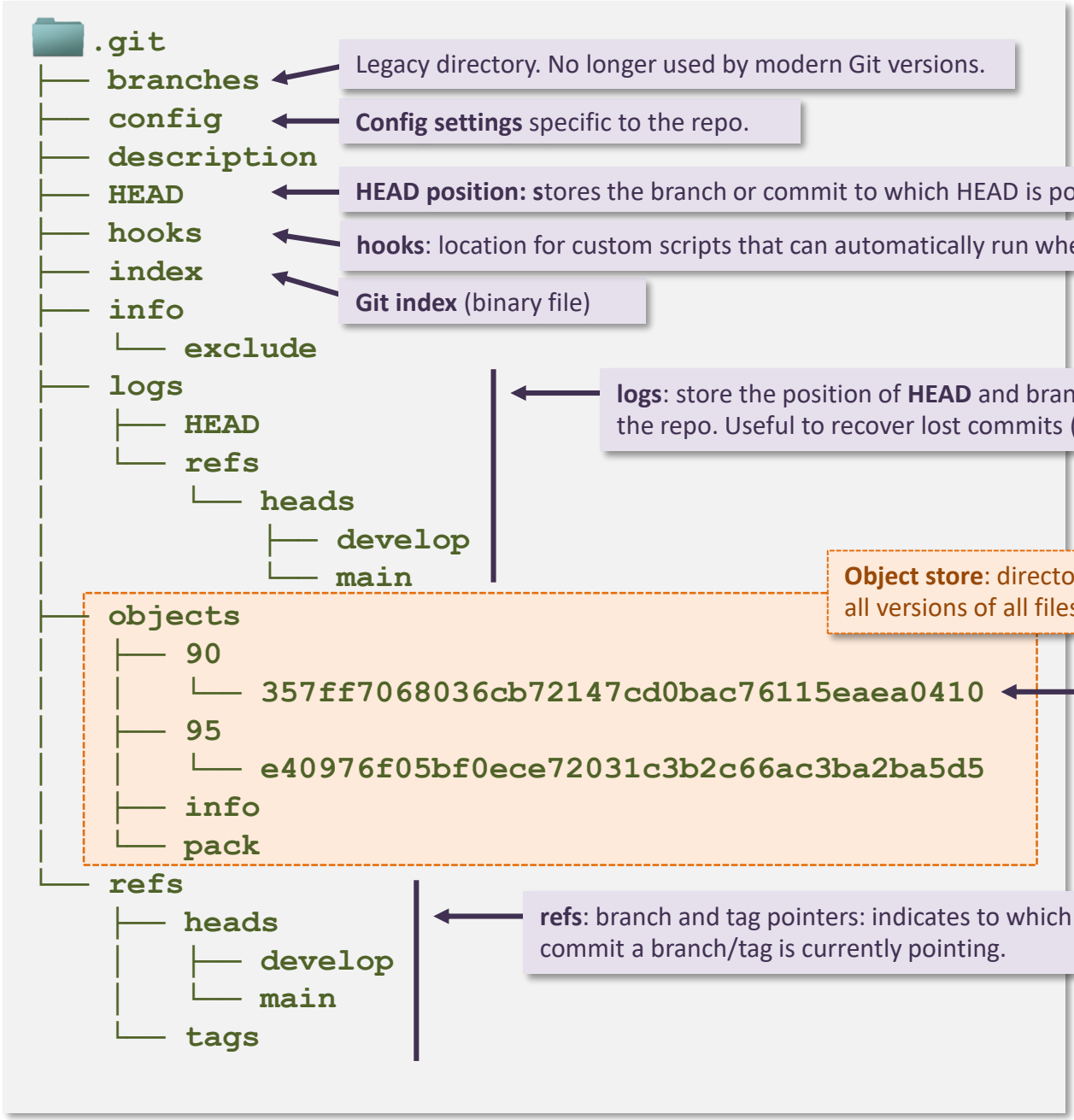
Reminder



Never delete the `.git` directory, you would lose the entire versioning history of your repository (along with all files not currently present in the working tree).

* An exception is the case of multiple projects that are tightly linked to another: in such cases it can be useful to have them all in a single repo – this is known as a **monorepo**.

Behind the scenes: the content of the `.git` directory



Legacy directory. No longer used by modern Git versions.

Config settings specific to the repo.

HEAD position: stores the branch or commit to which HEAD is pointing.

hooks: location for custom scripts that can automatically run when performing git commands (e.g. run tests before a commit is made).

Git index (binary file)

logs: store the position of **HEAD** and branches over the history of the repo. Useful to recover lost commits (see `git reflog`).

Object store: directory where the copies of all versions of all files are stored.

Individual files are stored under their a hash based on their content. File content is stored in zlib-compression format.

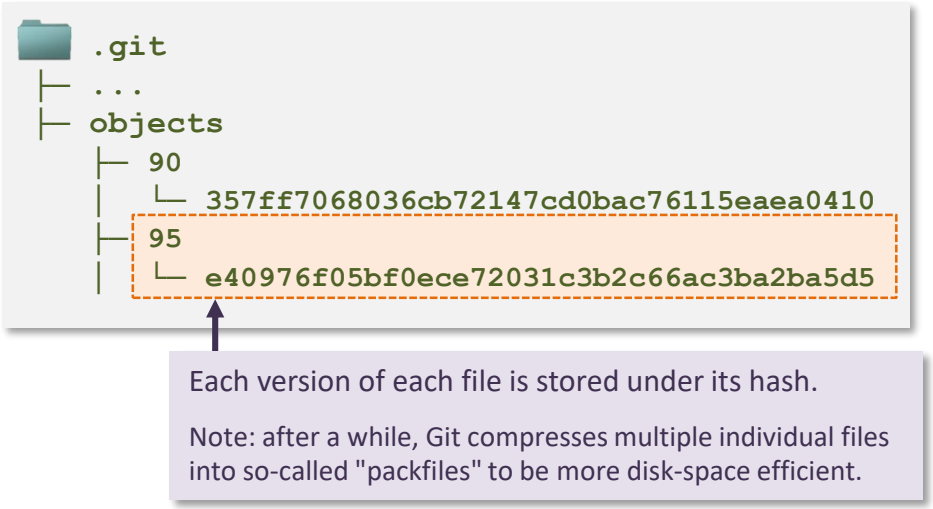
refs: branch and tag pointers: indicates to which commit a branch/tag is currently pointing.

Supplementary material...

Behind the scenes: how Git stores files in `.git/objects`

Each time a new file (or a new version of a file) is added to a Git repo, Git stores it in its object store (`.git/objects`) by:

- Computing a hash values for the file (see details below).
- Compressing the content of the file in zlib-format, and storing the output under the file's hash value. This is called a "blob".



Hashing details

When computing the hash value of a file, Git does not directly hash the raw file content. It prepends a little metadata (object type and size) in the form:

```
blob <size>\0<file content>
```

and runs this content through a hashing function. Historically Git has used SHA-1 as hashing function, but starting with Git 2.51 it can also use SHA-256 (but SHA-1 remains the default).

To compute the hash of a file, as Git would:

`git hash-object`

```
# Compute the hash of the file "README.md".
❯ git hash-object README.md
90357ff7068036cb72147cd0bac76115eaea0410
```

To print the content of a "blob":

`git cat-file`

```
# Print the content of a "blob".
❯ git cat-file -p 90357ff7068036cb72147cd0bac76115eaea0410
...
```

Supplementary material...

“Bare” Git repositories

A **bare repo** is a repo that has **no working tree**: it does not contain any instance of the files that are under Git version control, but only the content of the `.git` directory/database.

This type of repo is found on remote servers used to share and sync changes across multiple Git repositories. They can be initialized with the command:

```
git init --bare
```

Supplementary material...

Making a **commit**

```
git add
```

```
git commit
```

Making a commit

In Git, creating a commit is a 2-step process:

Step 1 – **Staging files**

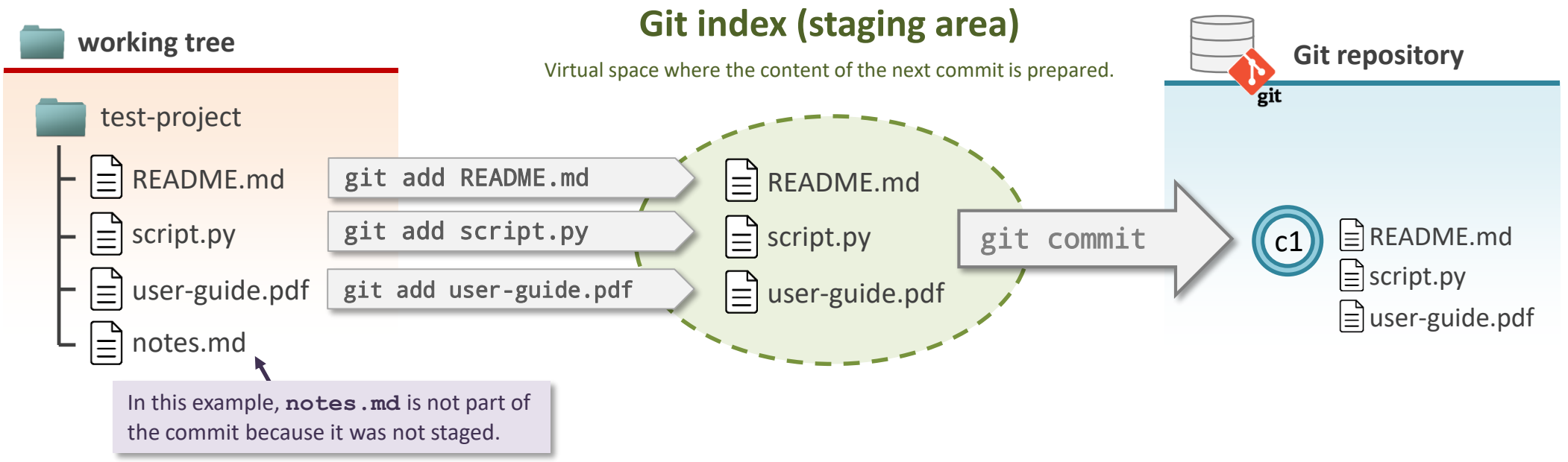
Selection of files to commit. To make a new or modified file part of the next commit, it must be added to the **Git index** (also known as the **staging area**).

```
git add <file or directory>
```

Step 2 – **Commit**

Create a commit with the current content of the Git index. A new commit (containing the current content of the Git index) is added to the repository.

```
git commit -m "commit message..."
```



Definition: the Git index (staging area)

Definition

Git index (staging area): "virtual space" where changes are collected before committing them to the repository, providing fine-grained control over which changes to include in the next commit.

Technical notes:
- In practice, the Git index is a file in Git's database.
- It is also possible to stage only certain changes made to a file, instead of the entire file.

⇨ We can now update our **definition of a commit**.

Definition

Commit = snapshot of the Git index at a given time.
Git index = content of your next commit.

Why do we need the Git index and this 2-step process ? Why not always commit the entire content of a repo ?

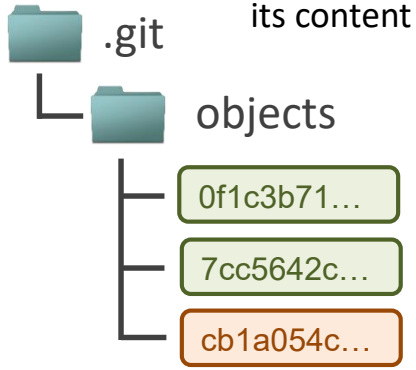
⇨ The objective of this 2-step procedure is to let users make **"well thought" commits**.

- **Commits are** meant to be **meaningful units of change** in your code base (or the content you track).
- Not all changes made to the working tree need to be part of the next commit.
- Well defined commits make it easier to track bugs or revert changes if needed.



The Git index: technicalities

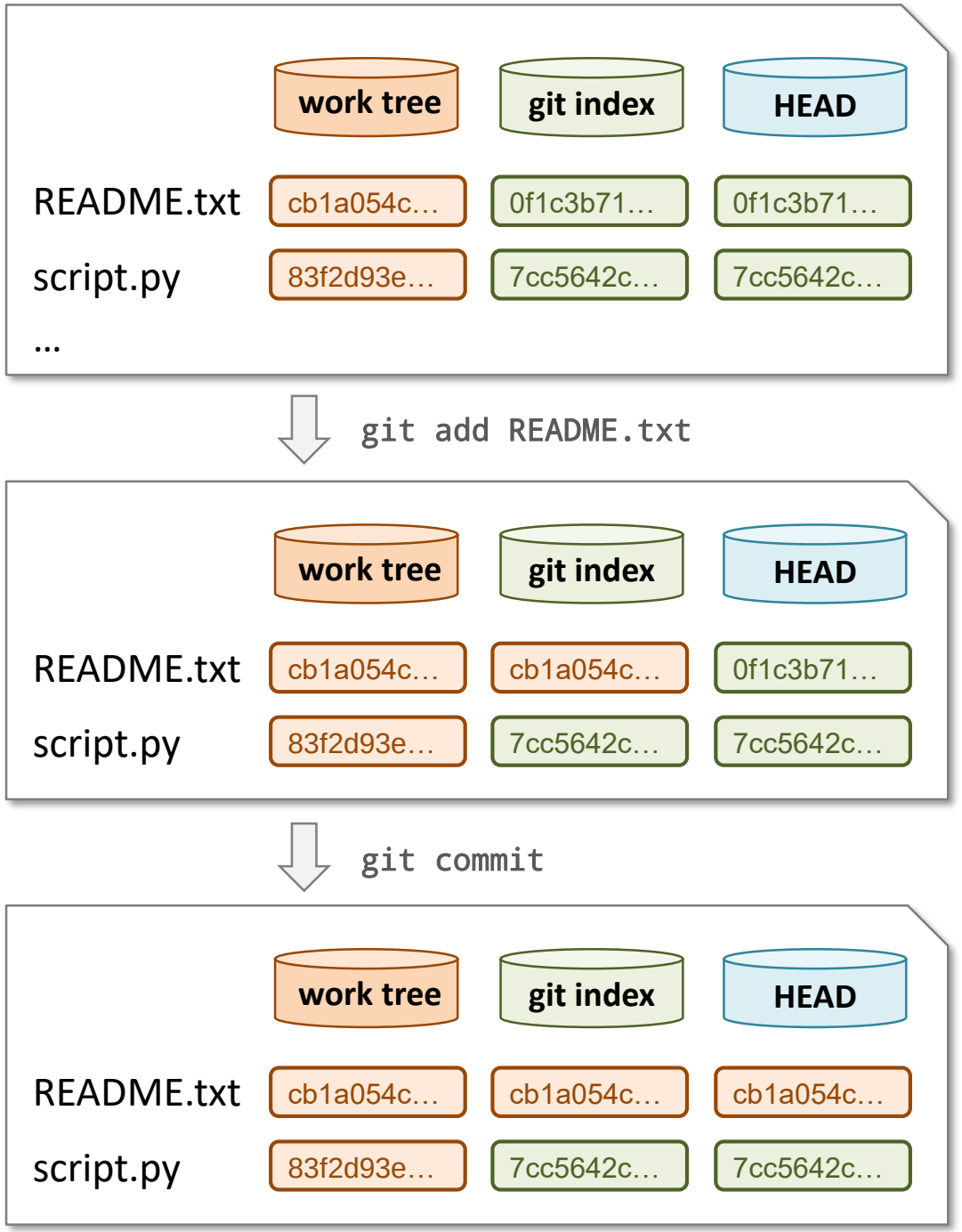
- The Git index is a binary file located in [`.git/index`].
- The index has no copies of the data, it's only a table linking the names of all tacked files with their object ID (blob hash).
- **The index represents the complete proposed content of the next commit**, including files that haven't changed since the previous commit. It can differ from both the latest commit and the working tree.



git add - when a file is added/updated to the index, its content gets stored as blob in the object store.

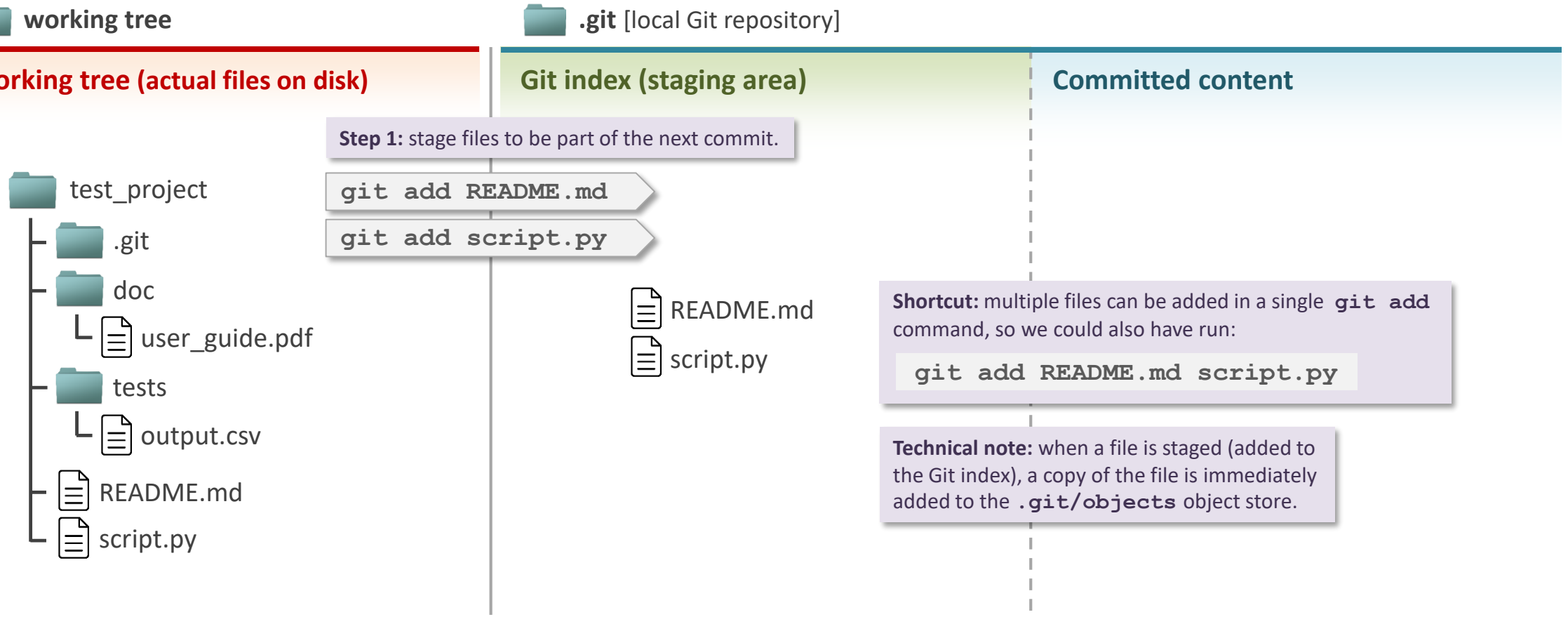
git commit - when a new commit is created, Git writes the tree objects representing the complete directory structure described by the index.

The root tree's object ID (hash) is included in the commit object, along with metadata (parent commit, author, committer, timestamp and commit message). Git then hashes the commit object to generate the commit ID.



Supplementary material...

Staging and making a commit: a step-by-step example



File status (before staging)

```
$ git status
On branch main
No commits yet
Untracked files:
    doc/
    tests/
    README.md
    script.py
```

Initially, all files are untracked.

File status (after staging)

```
$ git status
On branch main
No commits yet
Changes to be committed:
  new file: README.md
  new file: script.py
Untracked files:
```

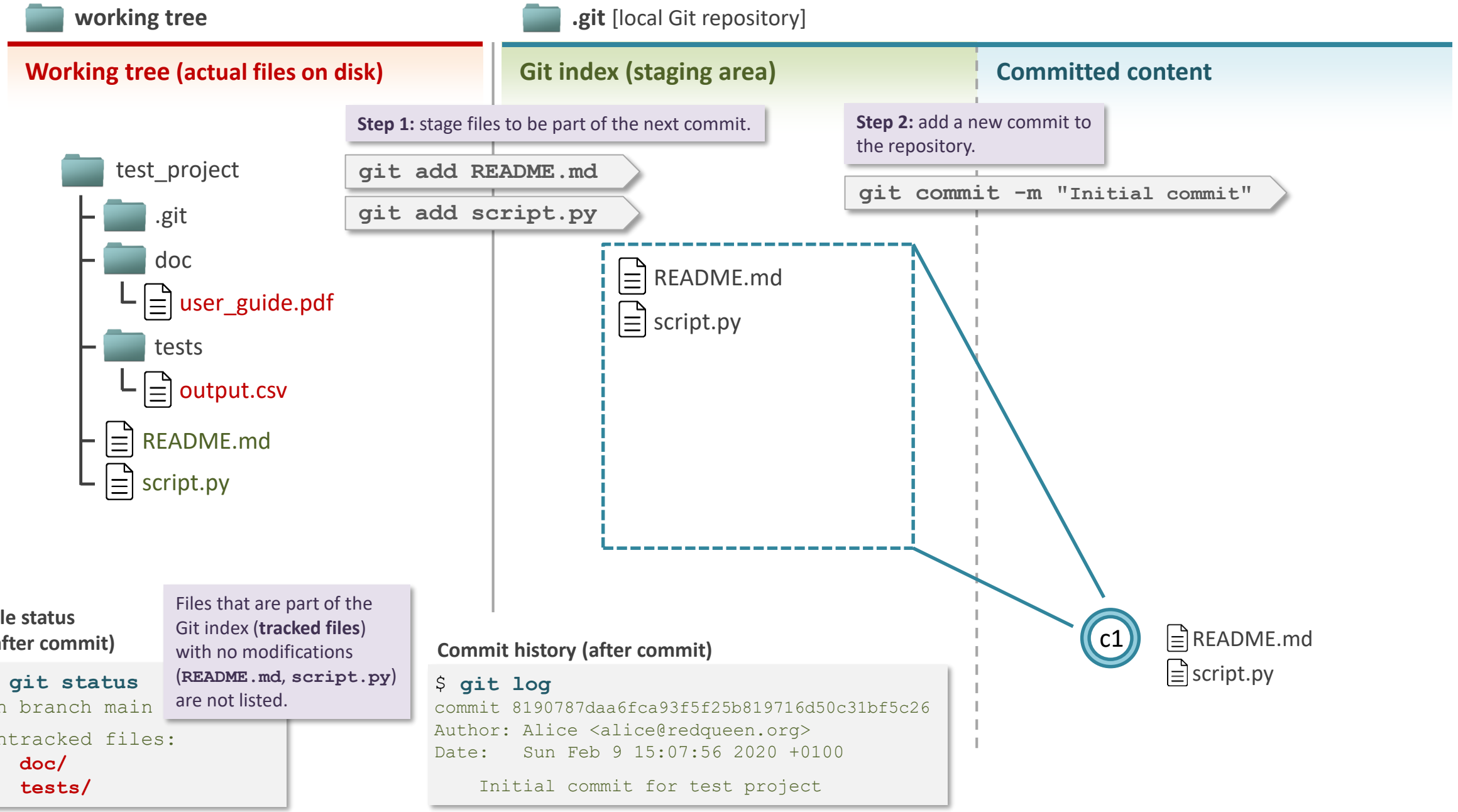
`git status` now indicates that 2 files have been newly added to the Git index.

Commit history

```
$ git log
fatal: your current branch 'main' does not have any commits yet
```

So far we only staged files, but no commit was made. This is why the output of `git log` is empty.

Staging and making a commit: a step-by-step example



Staging and making a commit: a step-by-step example

Important: once a version of a file was added to the Git index, it remains there and will be part of the next commit (unless we explicitly remove it). ***As long as a file is not modified***, there is ***no need to stage it again***.

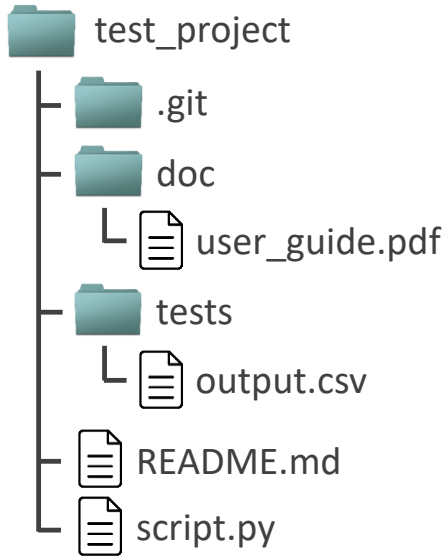
working tree | **.git** [local Git repository]

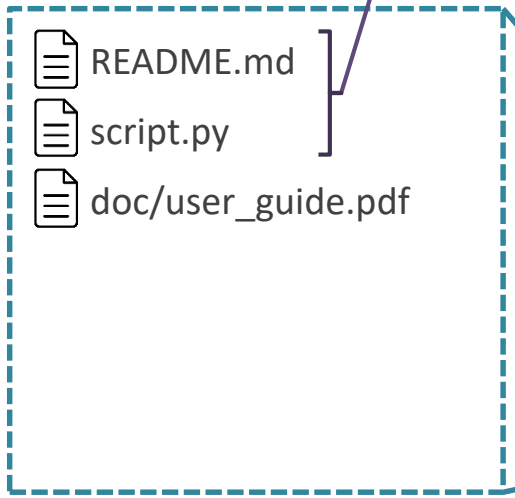
Working tree (actual files on disk) | **Git index (staging area)** | **Committed content**

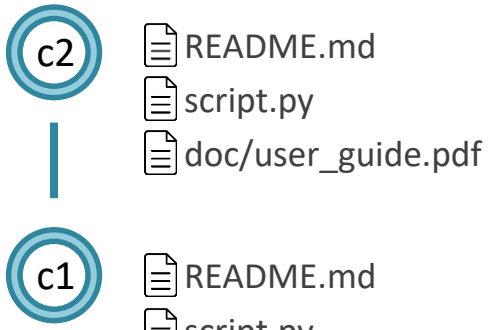
Let's add a new file to our repo: `user_guide.pdf`

```
git add doc/
```

```
git commit -m "Add user guide"
```







c2: README.md, script.py, doc/user_guide.pdf

c1: README.md, script.py

File status (after staging)

```
$ git status
On branch main

Changes to be committed:
  new file: doc/user_guide.pdf

Untracked files:
  tests/
```

Commit history (after commit)

```
$ git log
commit 04728026143ae57a71dcb7c1a503022041fb7d4d
Author: Alice <alice@redqueen.org>
Date:   Fri Oct 11 09:53:05 2024 +0200

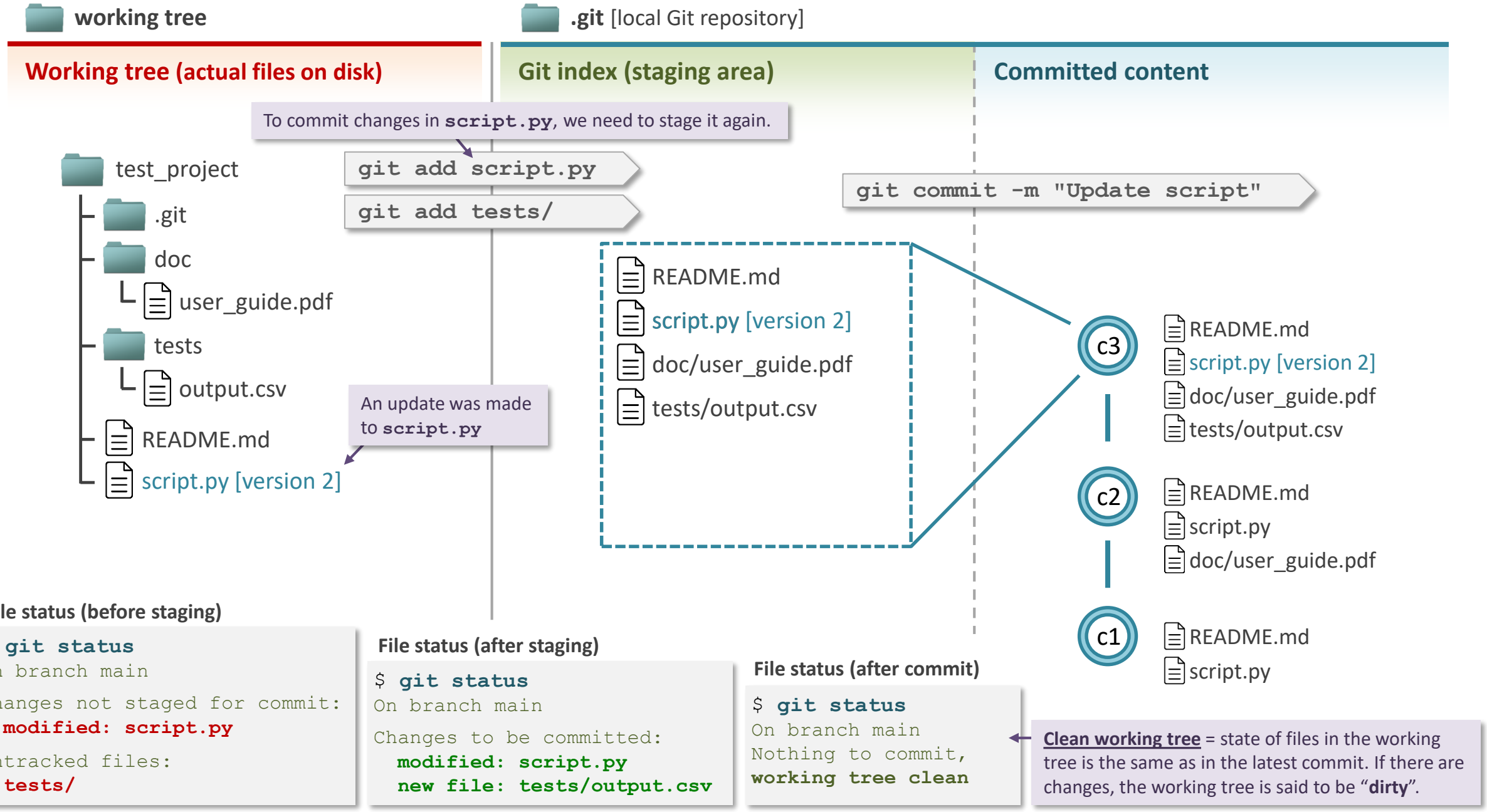
    Add user guide

commit 8190787daa6fca93f5f25b819716d50c31bf5c26
Author: Alice <alice@redqueen.org>
Date: Fri Oct 11 08:43:15 2024 +0200

    Initial commit for test project
```

Technical note: files that do not change across commits are stored only once in the data base (.git/objects). No space is wasted.

Staging and making a commit: a step-by-step example



Summary: staging (`git add`) and committing (`git commit`) files

- By default, files in a directory under Git control are **untracked**.
- To include a file (in its current state) – or a change in file content – in the next commit, the file **must be added to the Git index (staged)** with:

```
git add <file or directory>      # Adds the specified files/directories to the Git index.
```

  - Multiple files/directories can be added in a single command (by passing multiple file/directory names).
  - By default, the entire content of a file is staged. Files can be **partially staged** (only add some of the changes to the index) by using the `--edit` or `--patch` options of `git add` .

- **Modified files must be staged (added to the index) again**, if the new content is to be added to the next commit.
- Files that did no change do not need to be staged: once a file has been added to the index, it **remains in the index** (unless explicitly removed with `git rm` or `git rm --cached`).
- If a file was modified but is not staged again, those modifications will not be part of the next commit (this can often be a deliberate choice).

- Files that are in the index are called **tracked files**.
- Files that are not in the index are called **untracked files**.

Summary: staging (`git add`) and committing (`git commit`) files

- To **create a new commit**, the command is:

```
git commit -m/--message "your commit message"
git commit
```

If no commit message is given, Git will open the default editor and ask you to enter a message **interactively**.

Reminder

commit = snapshot of the Git index

The Git index (staging area) can be thought of as a "virtual space" where the content of the next commit is prepared.

- **Removing files** from the index is done with:

```
git rm --cached <file to remove>
```

← Removes the file from the Git index.

```
git rm <file to remove>
```

← Removes the file from the Git index and **deletes it** from your working tree (i.e. your local disk).

Making commits: some advice

Git does not impose any restrictions on what and when things can be committed. *

However, it's best to:

- Make commits at ***meaningful points*** of your code/script development, for instance:
  - When a new feature was added (or a few related functions).
  - When a bug was fixed.
- Make ***multiple small commits instead of a large one*** if you are making changes that affect different functionalities of your code (this can make it easier to e.g. revert changes).
- ***Don't commit broken code on your main/master branch***, as this is the branch that others might use to get the latest version of your code.

  If you have partial work, you can commit it to a *temporary/feature* branch, and later merge it into *main/master* (more on branch management will follow later).

* One exception being that, by default, commits with zero changes are not allowed, but they are possible by using the `--allow-empty` option: `git commit --allow-empty`

Shortcuts: for staging (`git add`) and committing (`git commit`) files

- `git add` **shortcuts**

```
git add -u / --update     # Stages all already tracked files, but ignore untracked files.
git add -A / --all        # Stages all files/directories in the working tree (except ignored files), including file deletions.
git add .                 # Stages entire content of the current directory, except file deletions.
```

- `git commit` **shortcuts**

This is a shortcut for:

```
git add <file or directory>
git commit -m "commit message"
```

```
git commit -m "commit message" <files or dirs>   # Stage and commit the files/directories in a single command.
git commit -a / -all -m "commit message“         # Stage and commit all modified tracked files in a single command.
```

`-a / --all` is a shortcut for:

```
git add -u
git commit -m "commit message"
```

It will not stage/commit untracked files.

Supplementary material...

Detail: understanding the output of `git commit`

```
$ git commit -m "Initial commit for test-project"
[main (root-commit) 8190787] Initial commit for test-project
3 files changed, 6 insertions(+)
create mode 100644 README.md
create mode 100644 script.py
create mode 100644 doc/quick_start.md
```

6 insertions = 6 lines added in total (across all files)

README.md

+ 1

```
# Quick-start guide for the test-project software
```

script.py

+ 1

```
#! /usr/bin/env python3
```

doc/quick_start.md

+ 4 (empty lines also count)

```
# Test project: testing version control with Git

A small test project to illustrate the use of Git.
Maybe I will add more content to it later.
```

Supplementary material...

Committing content: interactive commit message with the “vim” editor

```
$ git commit
```

When no commit message is specified, Git automatically opens a text editor. By default, this editor is “vim”.

```
Initial commit for test_project
# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch main
# Changes to be committed:
#       new file:   README.md
#       new file:   script.py
#       new file:   doc/quick_start.md
#
```

- In the “vim” editor, press on the key “**i**” to enter edit mode
- In edit mode, you can now type your commit message.

Supplementary material...

Committing content: interactive commit message with the “vim” editor

```
Initial commit for test_project

This is the very first commit in this Git repo.
Way to go!

# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch main
# Changes to be committed:
#       modified:   README.md
#       new file:   script.py
#       new file:   doc/quick_start.md
#
~
~
:wq
```

- Commit message can be entered over multiple lines.
- By convention, try to keep lines reasonably short (<= 80 chars)

- Press “**Esc**” to exit “edit” mode.
- Type “**:wq**” in the vim “command” mode.

Press “**Enter**” to exit vim and save your commit message.

```
[main (root-commit) 8190787] Initial commit for test_project
3 files changed, 6 insertions(+)
create mode 100644 README.md
create mode 100644 script.py
create mode 100644 doc/quick_start.md
```

- You are now back in the shell and your commit is done.

Supplementary material...

Demo

- Initializing a new Git repo.
- Adding content to the Git repo.
- Making a commit with interactive commit message.

exercise 1 – part A

Your first commit



This exercise has helper slides

Exercise 1 help: bash (shell) commands you may need during this course

`cd <directory>`	Change into directory (enter directory).
`cd ..`	Change to parent directory.
`ls -l`	List content of current directory.
`ls -la`	List content of current directory including hidden files.
`pwd`	Print current working directory.
`cp <file> <dest dir>`	Copy a file to directory “dest dir”.
`mv <file> <new name>`	Rename a file to <new name>.
`mv <file> <directory>`	Move a file to a different directory.
`cat <file>`	Print a file to the terminal.
`less <file>`	Show the content of a file (type “q” to exit).
`vim <file>`	Open a file with the “vim” text editor.
`nano <file>`	Open a file with the “nano” text editor.

Reminder...

Inspecting **file status**

```
git status
```

```
git diff
```

Display file status

`git status` Display the status of files in the working tree.

- *** Modified files**: files with changes in content as comparted to the latest commit.
- **** Staged files that have not been modified** since the last commit (**unmodified files**) are not listed, but they are still in the index and will be part of the next commit.
- **Ignored files** are also not listed.

```
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   LICENSE.txt
        modified:   README.md
        modified:   script.py
        deleted:    test/test_output.csv
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md
        modified:   doc/user_guide.md
        deleted:    test/log.txt
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        untracked_file.txt
```

new file = file is not present in latest commit.

modified = file is modified compared to latest commit.

deleted = file is present in latest commit and will now be removed

Staged files **

modified = file is modified compared Git index.

deleted = file is deleted on disk, but is still present in the Git index (and the latest commit).

Unstaged files

Modified files *

untracked files

Green = files with (changes in) content (compared to the latest commit) that has been staged and will be part of the next commit.

Red = files with (changes in) content (compared to the latest commit) that is not staged. These changes will not be part of the next commit.

Note: the (new) content of a file can be **partially staged**: some changes in the file are staged (added to the index), while some remain unstaged. This is the case in the example above for the `README.md` file (which is why it's listed in both the staged and unstaged sections). **Only the staged content will become part of the next commit**.

File status in Git: summary

Possible statuses for files in Git:

- **Tracked** – file that is currently under version control. More specifically, it is currently part of the Git index (staging area) and therefore also generally part of the latest commit *. Tracked files can be further categorized as:
  - **Unmodified** – the file is part of the latest commit * (and the Git index), and no change was made to the file since then. In other words, the content of the file in the working directory (working tree) is the same as in the latest commit. Unmodified files are not listed by the `git status` command.
  - **Modified** –the content of the file in the working directory (working tree) differs from the latest commit *. Modified files can be staged, unstaged, or partially staged.
    - **Staged**: the difference in content has been added the Git index (staging area), and will therefore be committed with the next commit.
    - **Unstaged**: the difference in content has not been staged (not part of the Git index), and will therefore not be part of the next commit.
    - **Partially staged**: some differences (but not all) have been staged (added to the Git index). Only the staged differences will be part of the next commit.
- **Untracked** - file present in the project directory (working tree), but not currently under version control by Git. More specifically, the file is not currently present in the Git index – but could be part of an earlier commit.
- Ignored - untracked file that is part of the repository's "ignore list" (`.gitignore` or `.git/info/exclude` file). Ignored files are not listed by the `git status` command.

* more precisely: the commit to which the HEAD pointer is currently pointing – this concept is explained later in the slides.

Supplementary material...

How do I know **what changed** and which changes are staged ?

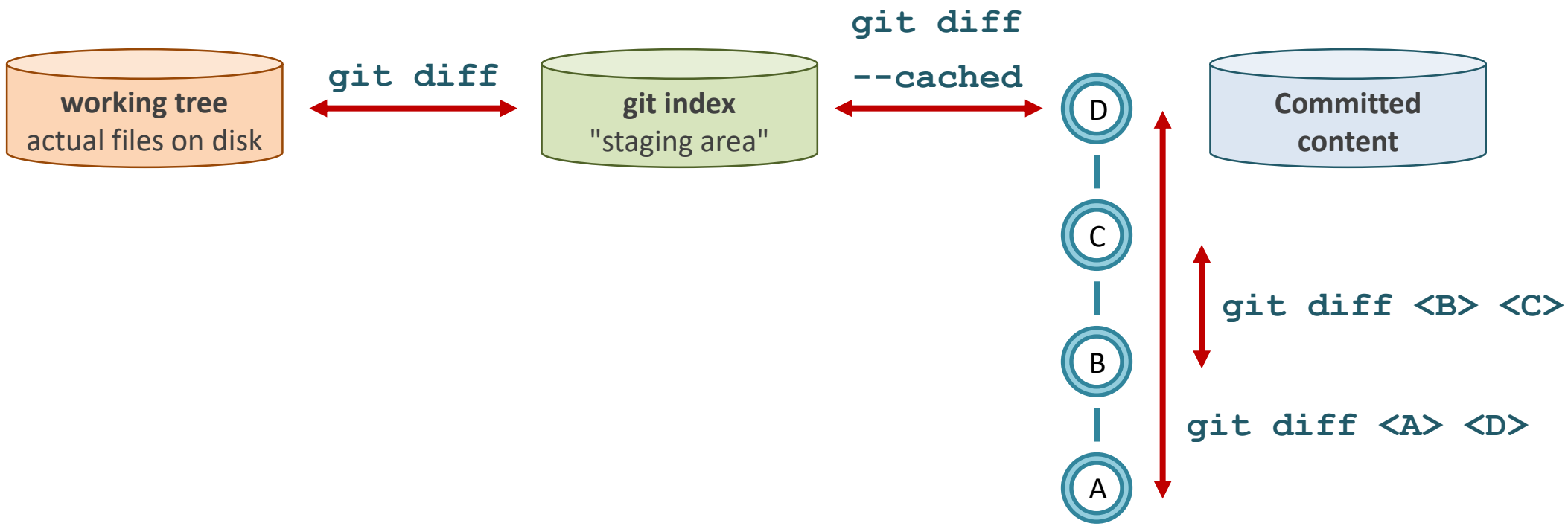
`git diff` Show differences between two states of a Git repo.

```
git diff                    # Show diff between working tree and Index.
git diff file               # Show diff for a specific file between working tree and index.
git diff --cached           # Show diff between index and the last commit (HEAD).
git diff <older commit> <newer commit>    # Show diff between 2 commits.

git diff --name-only        # Add --name-only to show only file names, not the changes.
```

Example output

```
❯ git diff
diff --git a/README.md b/README.md
index f5e333d..844d178 100644
--- a/README.md
+++ b/README.md
@@ -1,2 +1,3 @@
Project description:
-This is a test
+This is a demo project
+and it's pretty useless
```



Inspecting **commits** and **history**

```
git show
```

```
git log
```

Display the “content” of a commit

`git show` Display the changes in file content introduced by a commit.

```
git show <commit reference>
git show
```

with no argument, the latest commit on the current branch is shown (i.e. HEAD)

Examples of commit references:

- A commit ID (hash): **89d201f**
- A branch name: **develop**
- A tag name: **1.0.7**
- The **HEAD** pointer.
- A relative reference: **HEAD~3**

If no commit reference is given, **HEAD** is used as default.



The detail of changes can only be shown for plain text files.

Example:

```
$ git show 89d201f
commit 89d201fd01ead6a499a146bc6da5aa078c921ecf
Author: Alice <alice@redqueen.org>
Date:   Wed Feb 19 14:00:02 2020 +0100

    Add stripe color option to class CheshireCat

diff --git a/script.sh b/script.sh
index d7bfdc8..fa99250 100755
--- a/script.sh
+++ b/script.sh
@@ -7,13 +7,28 @@

  class CheshireCat():
-   def __init__(self, name, owner="red queen"):
+   def __init__(self, name, owner="red queen", stripe_color="orange"):
+       self.stripe_color = stripe_color
```

`git show --name-only <ref>`

Only display file names (without the changes)

```
$ git show --name-only 89d201f
commit 89d201fd01ead6a499a146bc6da5aa078c921ecf
Author: Alice <alice@redqueen.org>
Date:   Wed Feb 19 14:00:02 2020 +0100

    Add stripe color option to CheshireCat

script.sh
```

Display commit history

Print the commit history of the repository (newest commit at the top)

```
git log
git log --oneline
git log --all --decorate --oneline --graph
```



`git log` has many options to format its output.

See `git log --help`

Example: default view

```
$ git log
commit f6ceaac2cc74bd8c152e11b9c12ada725e06c8b9 (HEAD -> main, origin/main)
Author: Alice alice@redqueen.org
Date:   Wed Feb 19 14:13:30 2020 +0100

    Add stripe color option to class CheshireCat

commit f3d8e2280010525ba29b0df63de8b7c2cd7daeaf
Author: Alice alice@redqueen.org
Date:   Wed Feb 19 14:11:56 2020 +0100

    Fix off_with_their_heads() so it now passes tests

commit cfd30ce6e362bb4536f9d94ef0320f9bf8f81e69
Author: Mad Hatter mad.hatter@wonder.net
Date:   Wed Feb 19 13:31:32 2020 +0100

    Add .gitignore file to ignore script output
```

Example: compact view of current branch

```
$ git log --oneline
f6ceaac (HEAD -> main, origin/main) peak_sorter: add authors to script
f3d8e22 peak_sorter: display name of highest peak when script completes
cfd30ce Add gitignore file to ignore script output
f8231ce Add README file to project
821bcf5 peak_sorter: add +x permission
40d5ad5 Add input table of peaks above 4000m in the Alps
a3e9ea6 peak_sorter: add first version of peak sorter script
```

Example: compact view of entire repo (all branches)

```
$ git log --all --decorate --oneline --graph
* fc0b016 (origin/feature-dahu, feature-dahu) peak_sorter: display highest peak at end of script
* d29958d peak_sorter: add authors as comment to script
* 6c0d087 peak_sorter: improve code commenting
* 89d201f peak_sorter: add Dahu observation counts to output table
* 9da30be README: add more explanation about the added Dahu counts
* 58e6152 Add Dahu count table
| * f6ceaac (HEAD -> main, origin/main) peak_sorter: add authors to script
| * f3d8e22 peak_sorter: display name of highest peak when script completes
|/
* cfd30ce Add gitignore file to ignore script output
* f8231ce Add README file to project
| * 1c695d9 (origin/dev-jimmy, dev-jimmy) peak_sorter: add check that input table has the ALTITUDE and PEAK columns
| * ff85686 Ran script and added output
|/
* 821bcf5 peak_sorter: add +x permission
* 40d5ad5 Add input table of peaks above 4000m in the Alps
* a3e9ea6 peak_sorter: add first version of peak sorter script
```



Adding custom shortcuts to Git

Some git commands can be long and painful to type, especially when you need them often !
To shorten a command, you can create **custom aliases**:

```
git config --global alias.<name of your alias> "command to associate to alias"
```

Example:

```
git config --global alias.adog "log --all --decorate --oneline --graph"
```

With the alias set, you can now simply type:

```
git adog
```



Editing the **Git index**

(staging area)

Removing content from the Git index: example

Scenario: an update was made to `user-guide.pdf` and `script.py`. We want to commit the new version of `user-guide.pdf` (version 2), but not the changes to `script.py` and not `notes.md` .

working directory

Working tree (actual files on disk)

- test-project
  - .git
  - doc
    - user-guide.pdf [version 2]
  - tests
    - output.csv
  - README.md
  - script.py [version 2]
  - notes.md

`git add --all`

`git restore script.py`

Without `--staged` , **resets the file in the working tree** to its version from the index. Be careful not to lose work (there is no "undo").

.git, local Git repository

Git index (staging area)

- README.md
- script.py [version 2]
- doc/user-guide.pdf [version 2]
- tests/output.csv
- ~~notes.md~~

`git restore --staged script.py`

The version of `script.py` in the index is restored to the version from the latest commit*.

`git restore --staged notes.md`

Since `notes.md` is not present in the latest commit*, the whole file gets removed from the index.

Committed content

c3
- README.md
- script.py [version 2]
- doc/user-guide.pdf
- tests/output.csv

c2
- README.md
- script.py
- doc/user-guide.pdf

c1
- README.md
- script.py

File status after `git add --all`

```
$ git status
On branch main
Changes to be committed:
  modified: doc/user-guide.pdf
  modified: script.py
  new file: notes.md
```

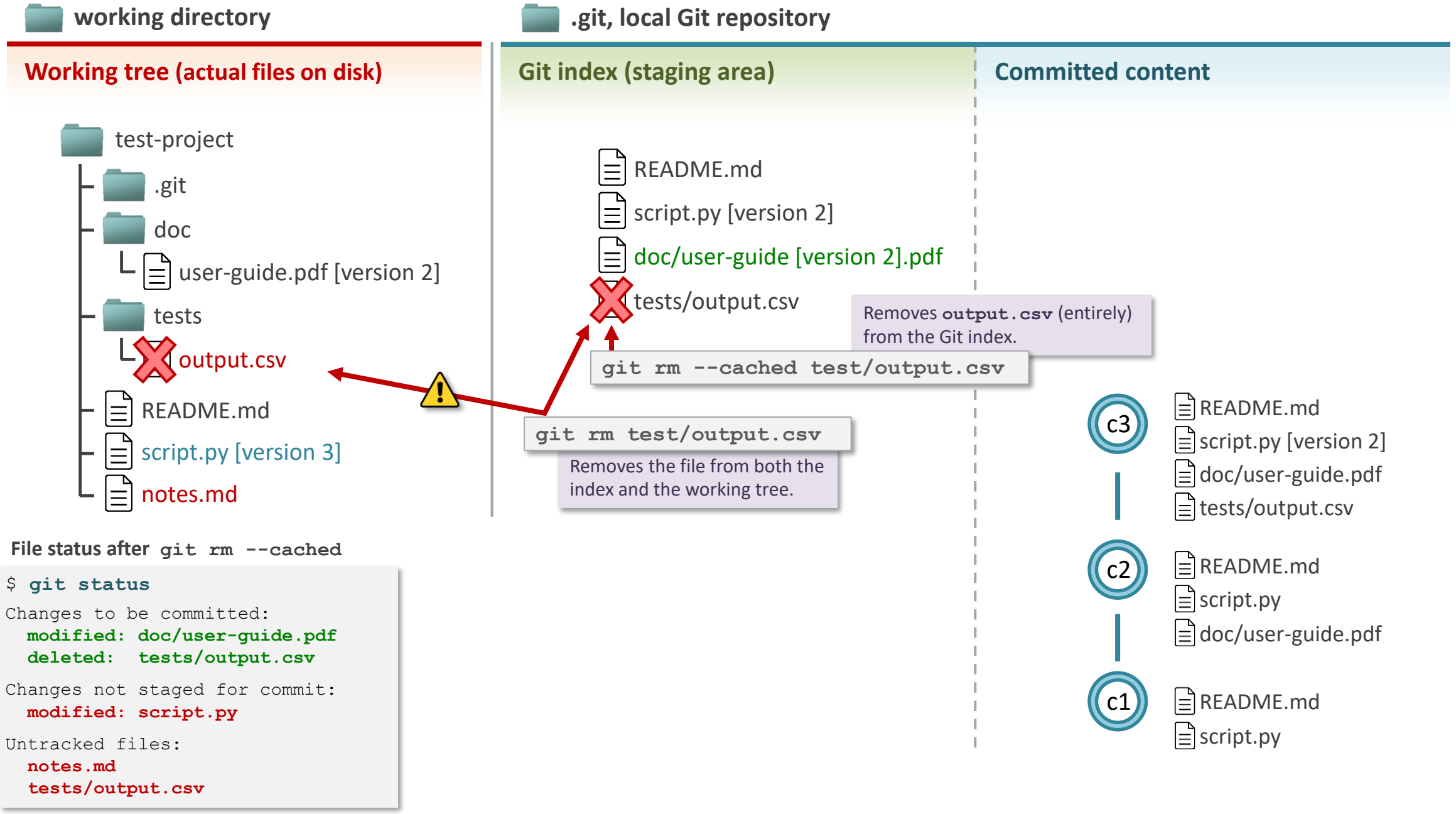
File status after `git restore`

```
$ git status
Changes to be committed:
  modified: doc/user-guide.pdf
Changes not staged for commit:
  modified: script.py
Untracked files:
  notes.md
```

* more precisely: the commit to which the HEAD is currently pointing, usually the latest commit on the current branch.

Removing content from the Git index: example

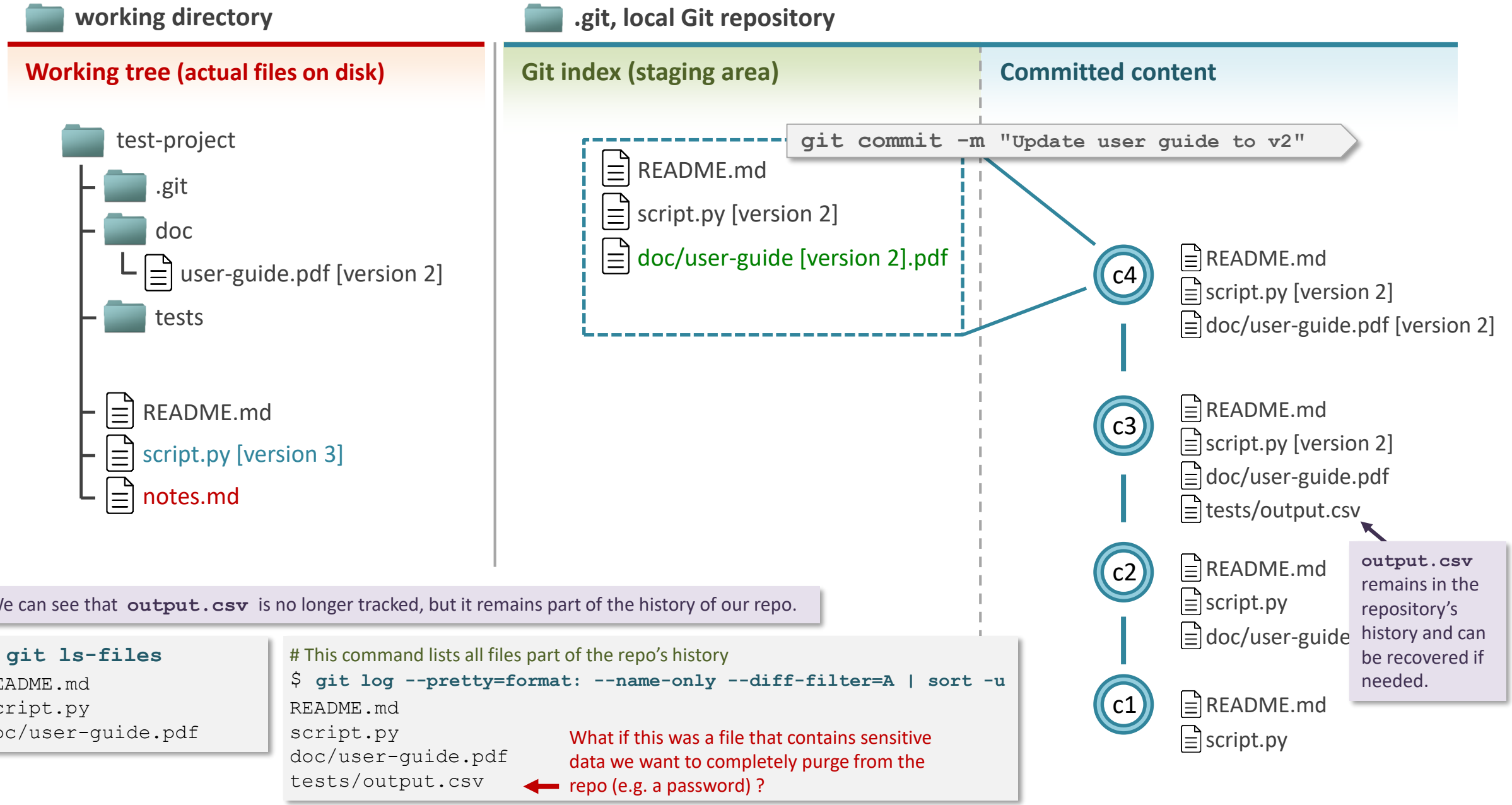
Scenario: at this point we realize that we would also like to stop tracking `tests/output.csv` in our repo.



File status after `git rm --cached`

```
$ git status
Changes to be committed:
  modified: doc/user-guide.pdf
  deleted:  tests/output.csv
Changes not staged for commit:
  modified: script.py
Untracked files:
  notes.md
  tests/output.csv
```

Removing content from the Git index: example



We can see that `output.csv` is no longer tracked, but it remains part of the history of our repo.

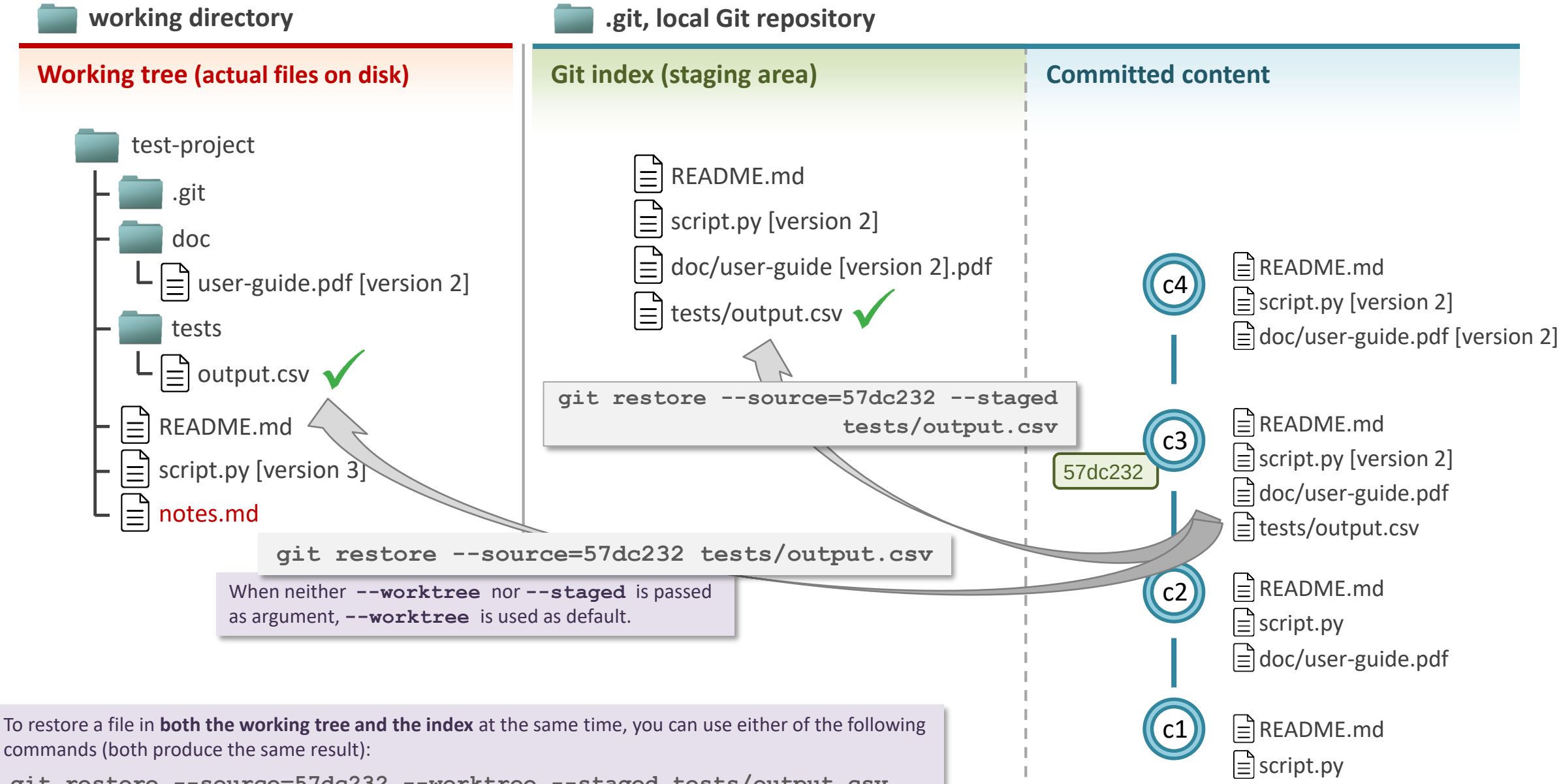
```
$ git ls-files
README.md
script.py
doc/user-guide.pdf
```

```
# This command lists all files part of the repo's history
$ git log --pretty=format: --name-only --diff-filter=A | sort -u
README.md
script.py
doc/user-guide.pdf
tests/output.csv
```

What if this was a file that contains sensitive data we want to completely purge from the repo (e.g. a password) ?

Retrieving a file from the Git repo: example

I would now like to **retrieve the file `output.csv`** from the Git repository.



To restore a file in **both the working tree and the index** at the same time, you can use either of the following commands (both produce the same result):

```
git restore --source=57dc232 --worktree --staged tests/output.csv
git checkout 57dc232 tests/output.csv
```



OMG ! How will I remember all these f****** commands ?

fantastic ?

The `git status` command provides helpful hints on how to stage/unstage files.

```
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   user-guide.pdf

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   script.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        notes.md
        tests/output.csv
```

Warning: without the `--staged` option, `git restore` will **reset (overwrite) the file in the working tree** with the version of the file from the Git index.

Only run it if you intend to delete the current version of your file – there is no "undo".



Summary: removing/renaming files from/in the Git index

- Remove newly staged content from the index (one file at a time).

```
git restore --staged <file>  # Remove newly staged content of the specified file.
```

Without the `--staged` option => **resets file in worktree to the its version in the Git index.**

The same can also be achieved using the `git reset` command. This is a specific use of the `reset` command, which has a wider scope.

```
git reset HEAD <file>    # Remove newly staged content of a specific file.
git reset HEAD           # Remove all newly staged content (all files).
```

Useful to unstage all changes in a single command.

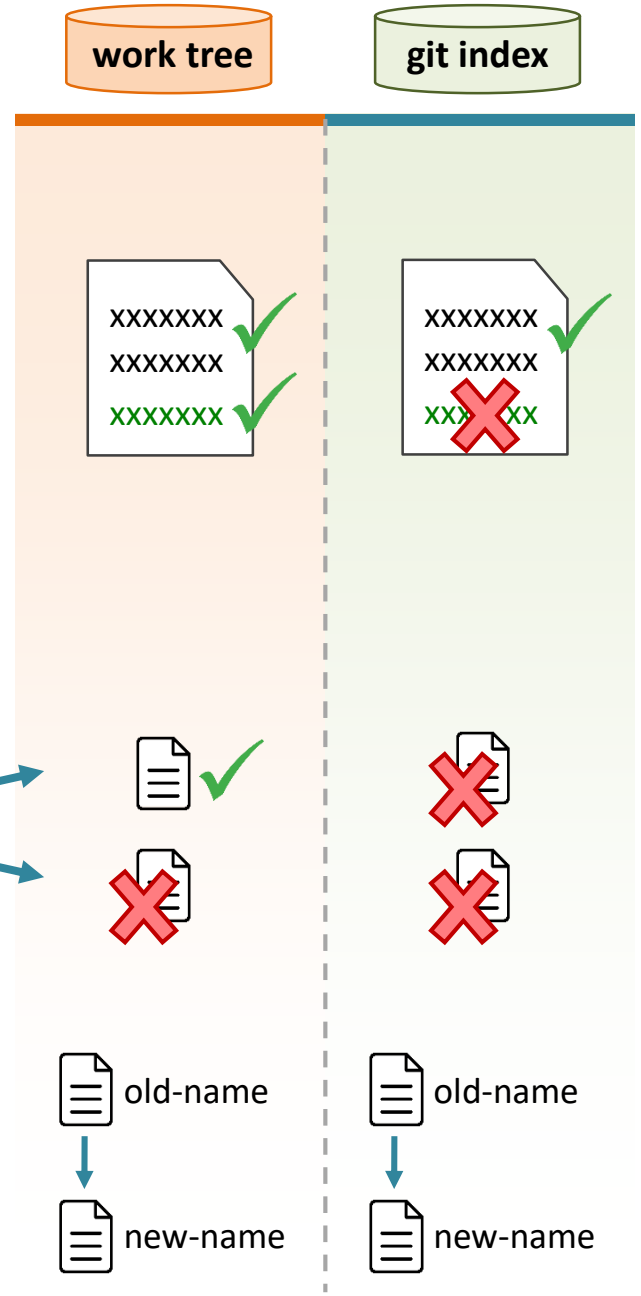
- Delete entire files from the index and the working tree.

```
git rm --cached <file>   # Delete file from index only.
git rm <file>            # Delete file from both index and working tree.
```

Without the `--cached` option => **deletes file in working tree (i.e. on disk) !**

- Rename and/or move files both in the working tree and the Git index.

```
git mv <file> <new location/new name>
```



Note: the `git restore` command is available from Git >= 2.23

ignoring untracked files

```
.gitignore
```

```
.git/info/exclude
```

Ignoring files

- By default, files that are not added to a Git repo are considered **untracked**, and are always listed as such by `git status` .
- To stop Git from listing files as **untracked**, they can be added to one of the following "ignore" files:

`.gitignore`

- For files to be **ignored by every copy of the repository**.
- Usually a single `.gitignore` file, placed a the root of the working tree is sufficient, but multiple files can be used (see next slide).
- `.gitignore` is meant to be shared, so track in in your repo.
- Examples of files to ignore:
  - outputs of tests
  - .Rhistory, .RData
  - .pyc (compiled version of python code)

Most of the time, this is the method you will want to use to ignore files.

`.git/info/exclude`

- For files that should be **ignored only by your own local copy of the repository**.
- Not versioned and not shared.
- Examples of files to ignore:
  - Files with some personal notes.
  - Files specific to your development environment (IDE).

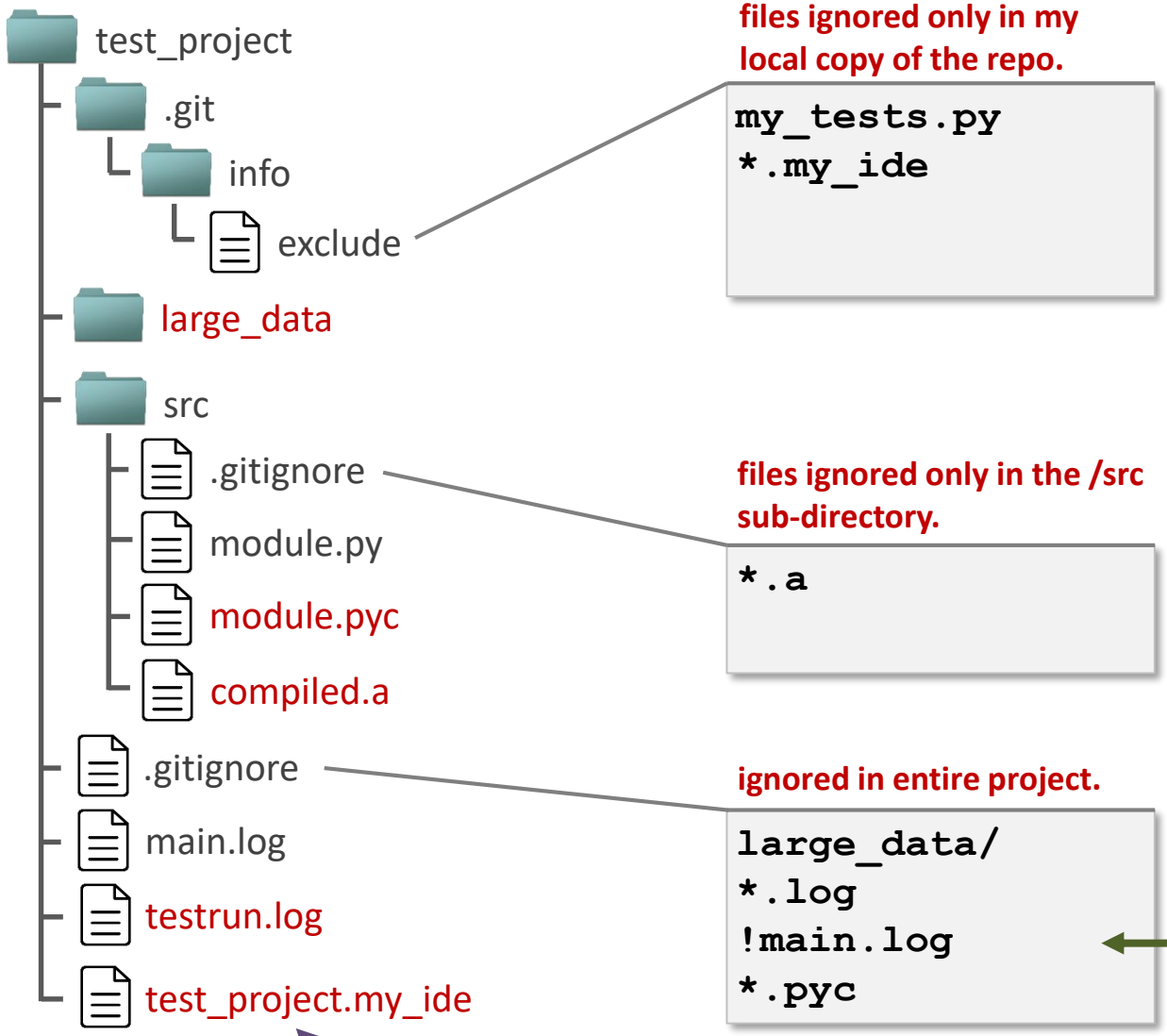
Use this method for **special cases** where a file should **only be ignored in your local copy of the repo.**

Example of a `.gitignore` file

```
my_tests.py
.Rdata
.Rhistory
*.pyc
test_outputs/
```

- Files to ignore are added by manually editing the two above-mentioned files.
- Files can be ignored based on their full name, or based on glob patterns (see next slide for examples).
  - `*.txt` ignore all files ending in ".txt"
  - `*.[oa]` ignore all files ending either in ".o" or ".a"
  - `logs/` appending a slash indicates a directory. The entire directory and all of its content are ignored.
  - `!dontignorethis.txt` adding a **!** In front of a file name means it should not be ignored (exception to rule).

Ignoring files: example



- There can be multiple `.gitignore` files per project, to create custom per-directory ignore rules.
- Ignore rules in sub-directories are inherited from the `.gitignore` of their parent directory(ies).
- The `.gitignore` files themselves should not be ignored: add them to the Git repo so they are tracked.

- **Order (sometimes) matters**: here the rule to not ignore main.log must be placed after the general rule to ignore *.log files.

red = ignored file.

Supplementary material...

Demo

- Ignoring files with `.gitignore`

exercise 1 – part B and C

Your first commit

A detailed look at **commits**

hashing algorithms

Hashing algorithms, such as SHA-1 or SHA-256 (**S**ecure **H**ashing **A**lgorithm) turns a binary input into an (almost) unique hexadecimal **hash**/**checksum value**.

hexadecimal = base 16 number (0-9 + a-f)
SHA-1 hashes are 40 characters long. SHA-256 are 64 characters long.

`e83c5163316f89bfbde7d9ab23ca2e25604af290` Example of a SHA-1 hash value.

- Important: hashing a given input always outputs the exact same hash.
- Example: running "This is a test" through the SHA-1 algorithm, will always produce the hash shown below:

```
echo "This is a test" | openssl sha1
```
→ `3c1bb0cd5d67dddc02fae50bf56d3a3a4cbc7204`

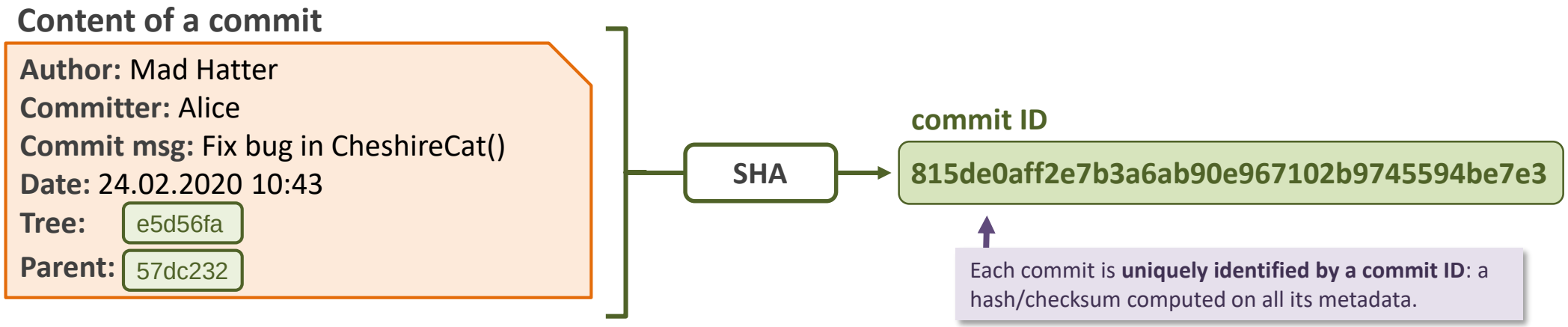
```
echo "This is a Test" | openssl sha1
```
→ `7500c6645cb9cdb20b32002cb82bbe067cc77d6e`

Notes:

- With current hardware, SHA-1 collisions can be reasonably easily created. SHA-1 is no longer considered secure for cryptographic purposes, but is good enough for usage in Git. It is also fast to compute.
- Recent Git versions can also use SHA-256 instead of SHA-1 as hashing algorithm, but SHA-1 remains the default for now.

Commits: immutable snapshots of a repository's state

- A commit represents the **state of a repository at a given time** => **snapshot of Git index + metadata**.
- A commit is **the only way to enter a change** into a Git repository.
  This **enforces accountability** as you cannot have untraceable modifications.
- Each commit has an associated author, committer, commit message and date - this **enforces documentation**.
- Commits are lightweight:
  - They **do *not* contain the tracked files' data**, only a **reference to the data** (specifically, a **Tree*** object that represents the state of the Git index at the time the commit was made).
- Commits contain a **reference to their parent** commit.



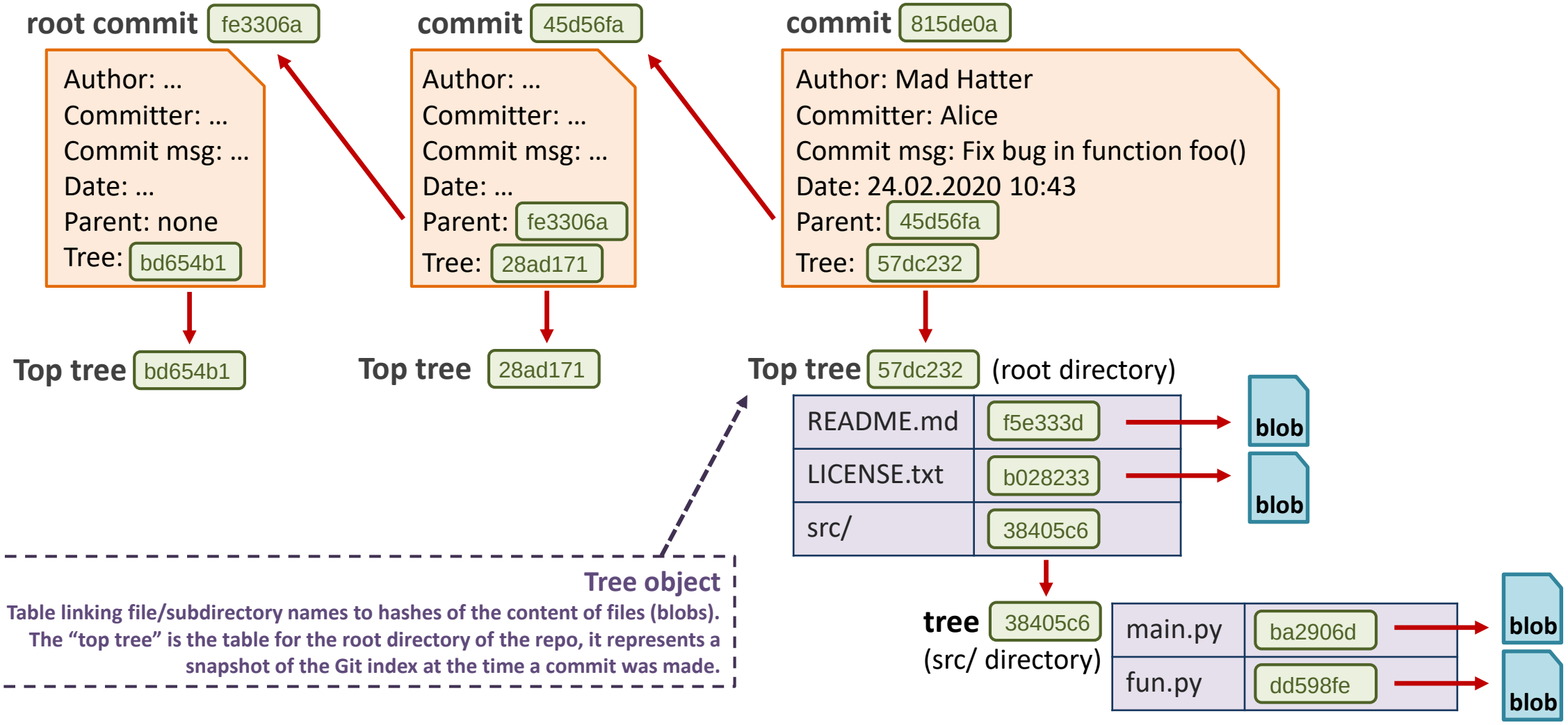
* Tree = reference to the state of all files at a given time point = snapshot of repository state.

- **Commits contain a reference to the top "Tree object"** – a table linking file names and hashes of the Git index at the time the commit was made. This is a "snapshot" of the index, and is how Git can retrieve the state of every file at a given commit.

If two commits have the same ID, their content is identical !

- **Commits point to their direct parent** – forming a DAG (directed acyclic graph) where no commit can be modified without altering all of its descendants.

If two commits have the same ID, their entire history is identical !



Supplementary material…

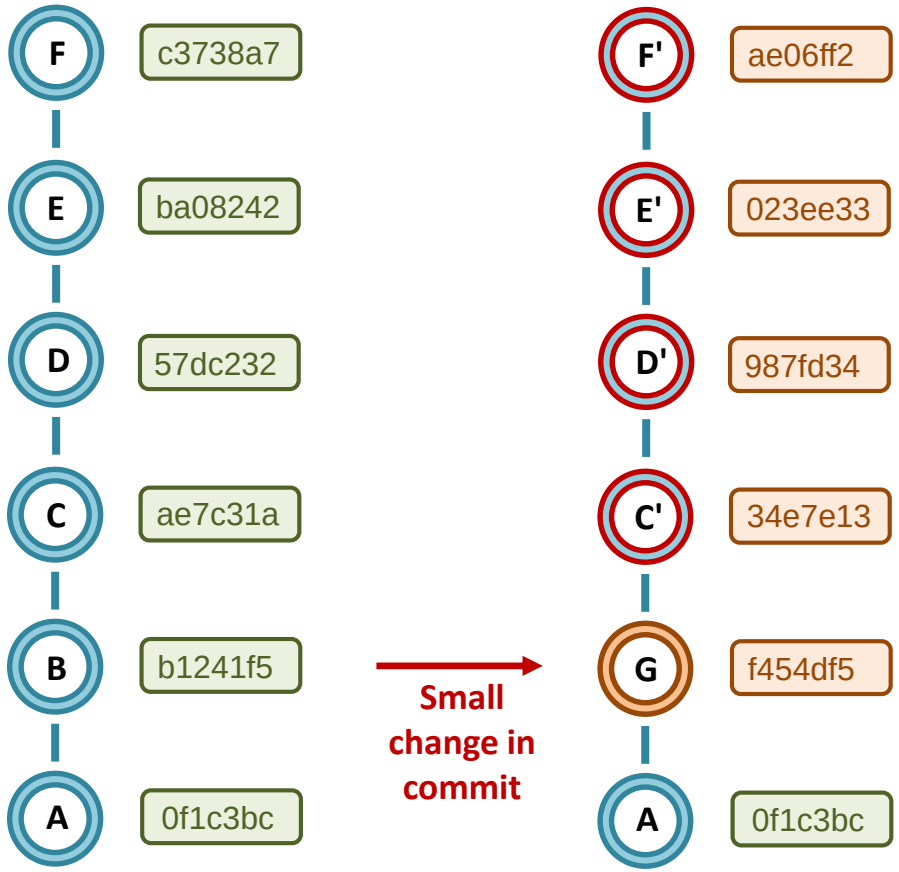
- Because of how a commit ID is computed, **commits are immutable**: once a commit is made, it cannot be modified without its commit ID being modified too - which would then make it a different commit !

- **Modifying a commit** will modify all of its descendants. It **creates a completely new history** of the Git repo.

- This ensures the **integrity of a Git repository's history**, something that **is important due to the distributed nature of Git**. It can be seen as a sort of blockchain.

Examples of things that change a commit's ID:

- **Changing the content** of a file.
- **Changing the time** a commit was made.
- **Changing the parent commit** of a commit.



The Git object store

- The "object store" is where Git stores the data and metadata of the tracked files and commits.
- Located in `.git/objects`

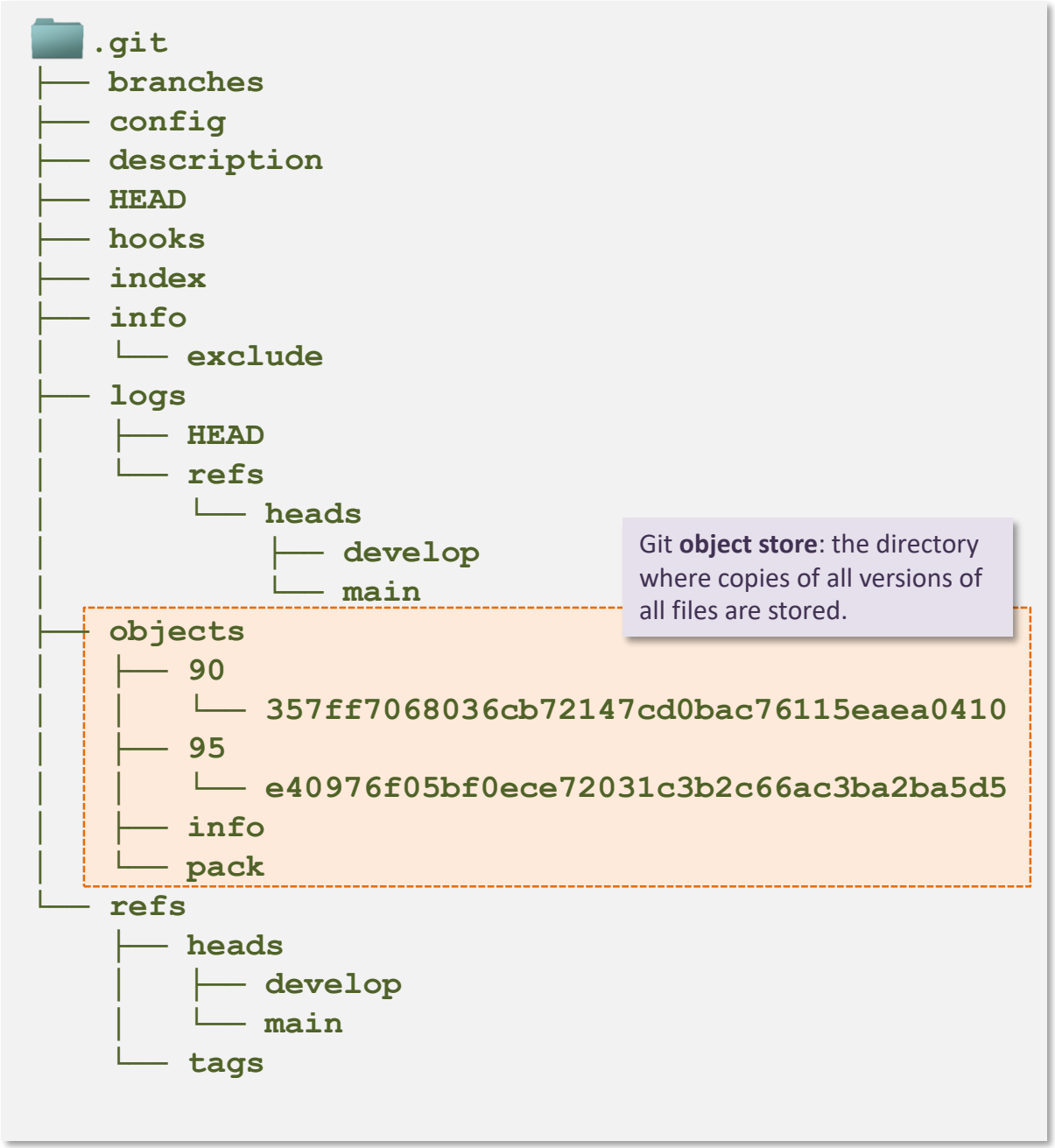
Git stores data in 4 object types, all saved in the object store [`.git/objects`]:

Blobs: binary, compressed, file that stores the content of a file. "blob" stands for "Binary Large OBject" (even if the object is not necessarily large)

Trees: Dictionary linking file names to blobs for a given directory.

Commits: metadata of each change introduced into the repository: author, commit message, state of files, etc …

Tags: name (e.g. software version) that points to a specific to a commit.



Supplementary material...

Blob objects (Binary Large OBjects)

- File that stores the content of a file (in a binary and compressed format).
- Does not store any metadata about the file, not even the file's name.
  - Two files with the same content have the same blob/SHA-1.
  - Two files with the same blob/SHA-1 have identical content. This allows fast comparison!
- Blobs are named after their content's SHA-1 hash*, and stored in the object store [`.git/objects`].

```
.git/objects/fa/263b8bb9291aaa5059dad78bb38b63f4318c62
.git/objects/4a/b7e6dbb9b1dd73a3e0292ef1d1b2909d107309
```

For performance reasons, the 2 first characters of the SHA-1 hash are used as sub-directory name (this avoids having too many files in the same directory). The remaining 38 characters are the name of the file.

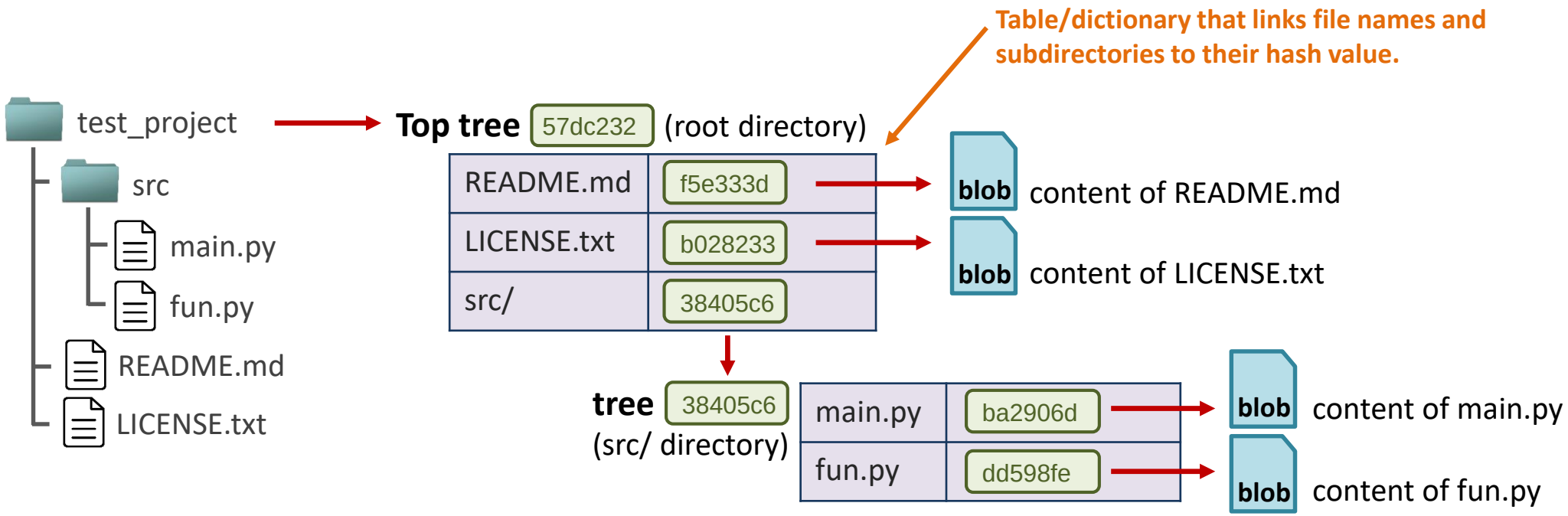
- Using a hash as file name creates so-called “content addressable” storage: the content of the file defines its location. This avoids any risk of losing content by overwriting files, since any change in a file will result in a new hash, and hence a new location.

* almost: Git adds a few header bytes to the content when computing file SHA-1 values.
you can get the SHA-1 hash computed by Git with: `git hash-object -t blob <file to hash>`

Supplementary material...

Tree objects

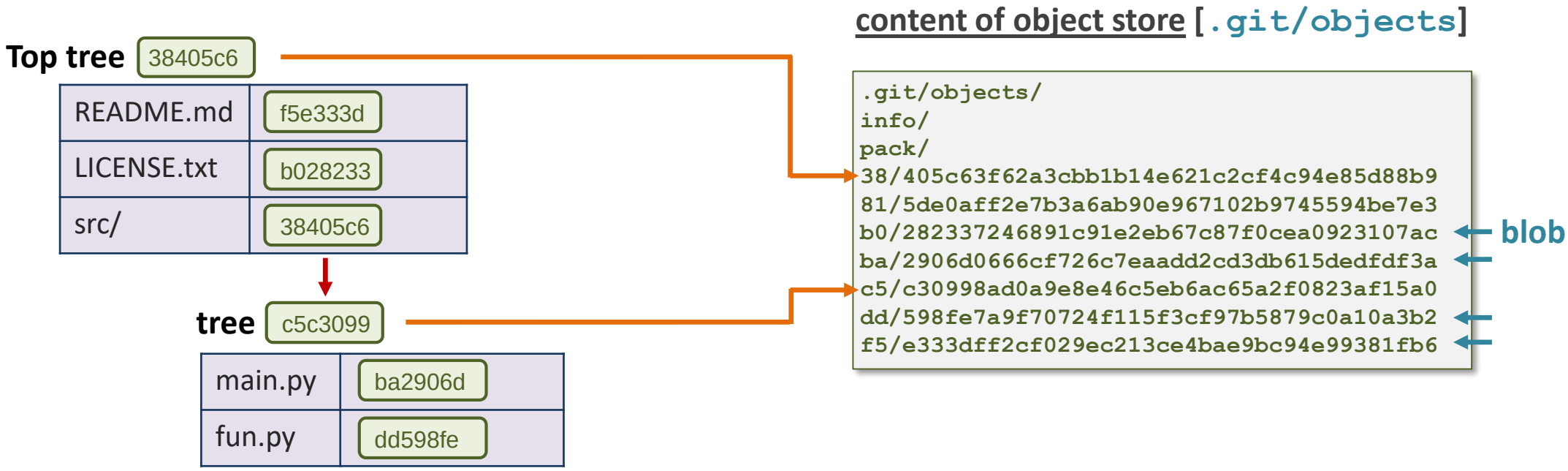
- Tree = dictionary/table linking filenames to blobs and directories to subtrees, at a given directory level.
- Sub-directories are also tree objects, referenced by their parent tree.
- **If two trees have the same hash, then their content is identical** – fast comparison as there is no need to look at individual files in the tree's sub-directory.
- The top tree (root of working tree) can be seen as a **snapshot of the entire repo** at a given time. It references blobs (file contents) and other trees (subdirectories).



Supplementary material...

Trees

- Trees are saved in the object store, as a file named after their hash – just like blobs.



Supplementary material...

Commit objects

- Commits are saved in the object store, as a file named after their hash – just like blobs and trees.

Content of commit

Author: Mad Hatter
Committer: Alice
Commit msg: Fix bug in function foo()
Date: 24.02.2020 10:43
Parent: 45d56fa
Tree: 57dc232

Commit saved in object store, named after its hash

content of object store [`.git/objects`]

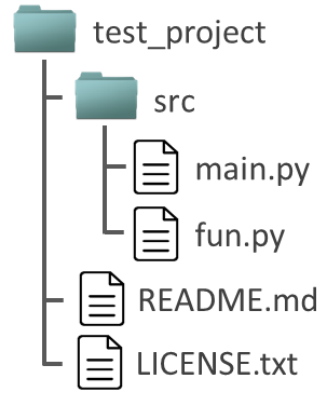
```
.git/objects/
info/
pack/
38/405c63f62a3cbb1b14e621c2cf4c94e85d88b9   ← tree
81/5de0aff2e7b3a6ab90e967102b9745594be7e3
b0/282337246891c91e2eb67c87f0cea0923107ac   ← blob
ba/2906d0666cf726c7eaadd2cd3db615dedfdf3a
c5/c30998ad0a9e8e46c5eb6ac65a2f0823af15a0
dd/598fe7a9f70724f115f3cf97b5879c0a10a3b2
f5/e333dff2cf029ec213ce4bae9bc94e99381fb6
```

`git commit` **triggers the creation of a commit object**

command in shell

```
$ git commit -m "Fix bug in function foo()"
[main (root-commit) 815de0a] Fix bug in function foo()
 4 files changed, 4 insertions(+)
 create mode 100644 LICENSE.txt
 create mode 100644 README.md
 create mode 100644 src/fun.py
 create mode 100644 src/main.py
```

- In our example, the object store has now 7 objects:
  - 4 blobs – one for each file tracked in the repo.
  - 2 trees – src/ and the root of the working dir.
  - 1 commit.



Supplementary material...

Git packfiles: compressing old history

- For older commits, Git uses a few tricks to decrease disk space usage:
  - Differences between similar files are stored as diffs.
  - Multiple files are compressed into a single “packfile” (`.pack` extension).
  - Each packfile has an associated packfile index (`.idx` extension), that associates filenames to blobs.

Supplementary material...

The **HEAD** pointer

HEAD: a pointer to the most recent commit on the currently active branch

Looking at the output of `git log`, we see a **HEAD ->** label: this **shows the position of the HEAD pointer**.

Commit ID (hash)
Here shown in a shortened form (7 first chars).

HEAD pointer

Local branch name

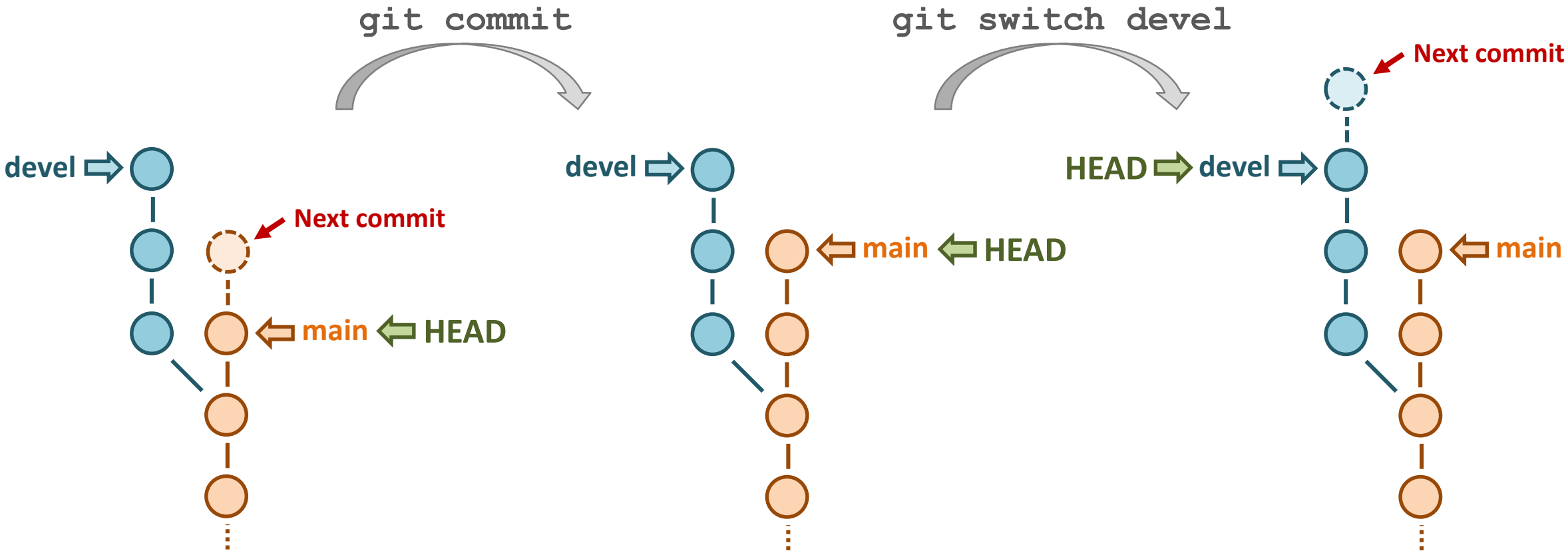
Remote branch name

First line of commit message

```
❯ git log --all --decorate --oneline --graph
* 351dca6 (HEAD -> main, origin/main, origin/HEAD) peak_sorter: added authors to script
* f3d8e22 peak_sorter: display name of highest peak when script completes
| * 076aa80 (origin/feature-dahu, feature-dahu) peak_sorter: display highest peak at end of script
| * d29958d peak_sorter: added authors as comment to script
| * 6c0d087 peak_sorter: improved code commenting
| * 89d201f peak_sorter: add Dahu observation counts to output table
| * 9da30be README: add more explanation about the added Dahu counts
| * 58e6152 Add Dahu count table
|/
* cfd30ce Add gitignore file to ignore script output
* f8231ce Add README file to project
| * 8e0d4fe (origin/dev-jimmy) peak_sorter: add check that input table has the ALTITUDE and PEAK columns
| * ff85686 Ran script and added output
|/
* 821bcf5 peak_sorter: add +x permission
* 40d5ad5 Add input table of peaks above 4000m in the Alps
* a3e9ea6 peak_sorter: add first version of peak sorter script
```

HEAD: a pointer to the currently checked-out branch/commit

- **HEAD** is – most of the time – a pointer to the latest commit on your current branch.
  (Sometimes it is also described as a pointer to the current branch – which is itself a pointer to the latest commit on the branch)
- The **HEAD** position is how Git knows what is the currently "active" branch.
- New commits are always added "under" the current **HEAD** (the new commit is the "child" of the commit pointed-to by **HEAD**.
- When a new commit is added, **HEAD** (and the branch it points to) is automatically moved to point to that new commit.



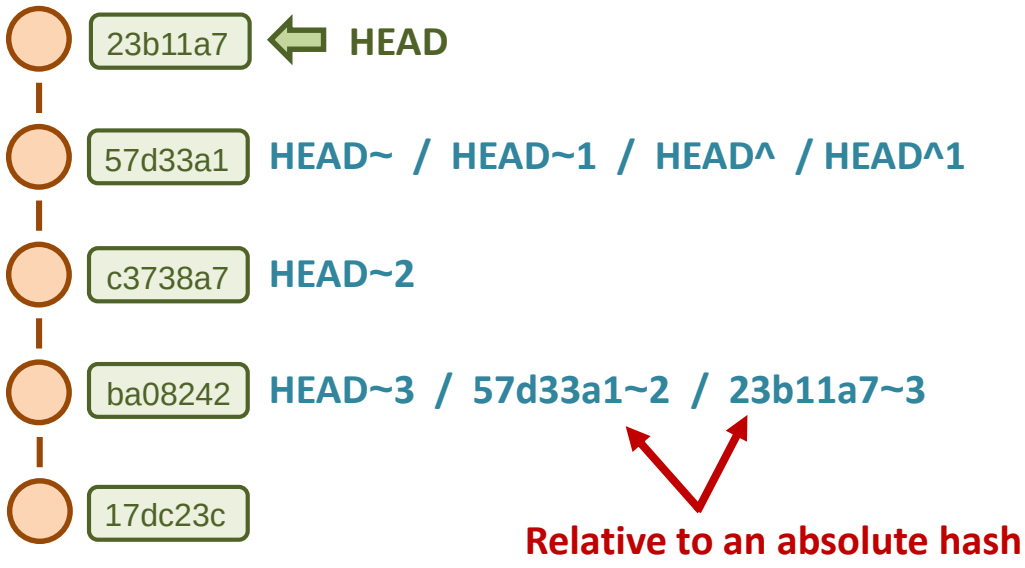
Another way to look at it, is that **HEAD always points to the parent of your next commit**.

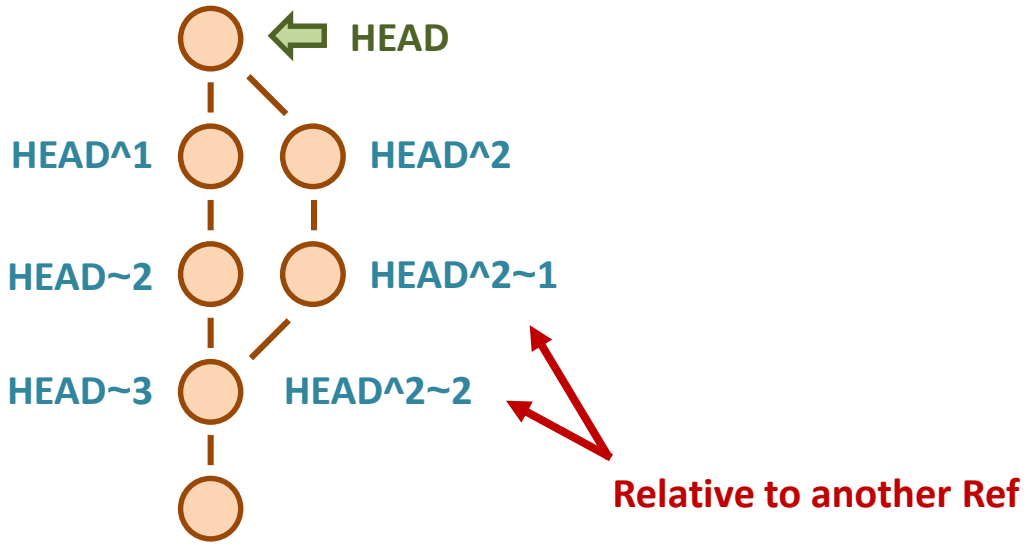
Relative references to commits

- Using `~` and `^` symbols, Git allows to refer to a commit by its position relative to another commit, rather than by its absolute hash.
- `Ref` can be any reference, such as **HEAD**, a commit hash, a branch name, or even another Ref.

`Ref~X`	refers to the **Xth generation before** the commit: ~1 = parent, ~2 = grand-parent, etc.
`Ref~`	is a shortcut for `Ref~1`

`Ref^X`	refers to the **Xth direct parent** of the HEAD commit (but most commits have only a single parent).
`Ref^`	is a shortcut for `Ref^1`





Supplementary material...

Part II

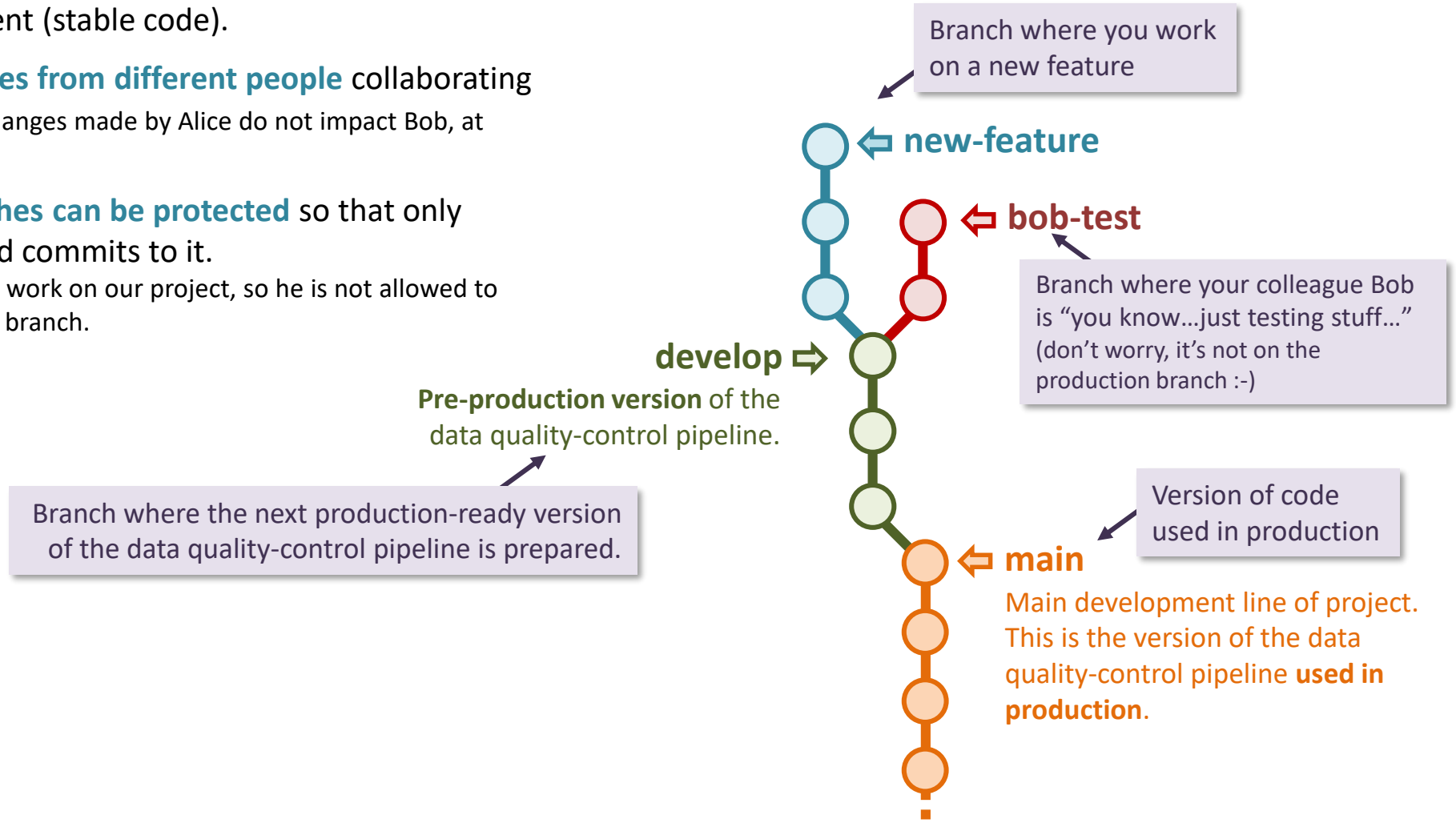
Git **branches**

Managing multiple lines of development

Why branches? An illustration with a data quality-control pipeline project

"Branching" means to **diverge from the main line of development**.

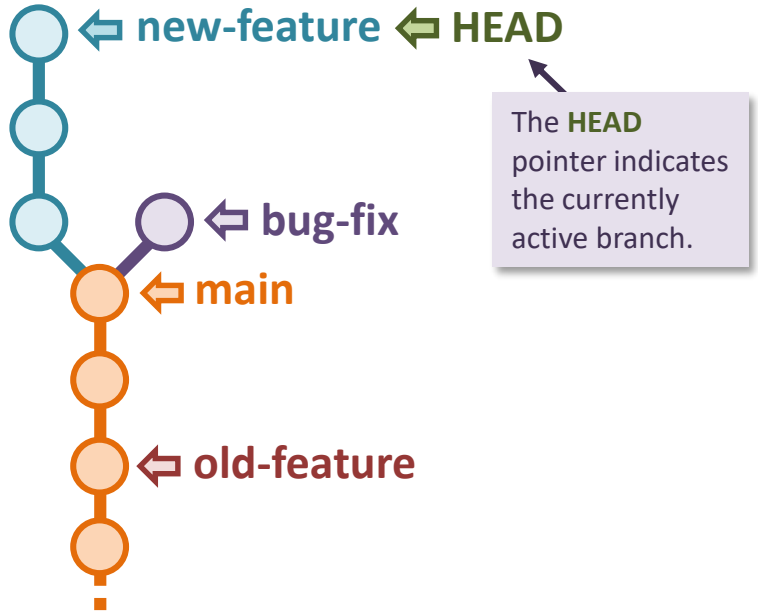
- Branches **isolate new changes** (work in progress) from the main line of development (stable code).
- Branches **isolate changes from different people** collaborating on a same project (so changes made by Alice do not impact Bob, at least not immediately).
- On online repos, **branches can be protected** so that only selected people can add commits to it.
  Use case: Bob just started to work on our project, so he is not allowed to make changes to the "main" branch.



Git is **designed to encourage branching**: branches are "cheap" (don't take much disk space) and switching between them is fast.

What are branches?

- A branch is just a pointer to a commit.
- A branch is **very lightweight** (41 bytes).
- By convention, the **main** branch is the branch representing the stable version of your work.
- To know which is the currently active branch, Git uses the **HEAD** pointer. The **HEAD** pointer always points to the currently active branch (except for the special case of "detached HEAD" mode, discussed later in the second part of this course).
- New commits are always added at the top of the currently active branch* (except for the special case of "detached HEAD" mode).



The **main** branch **is no special branch**. It is simply the default name given to the branch created when initializing a new repo [`git init`]. It has become a convention to use this branch as the stable version of a project.

Note: in earlier versions, the "**main**" branch used to be called the "**master**" branch.

Illegal characters in branch names

Spaces and some characters such as `,~^:?*[]\` are not allowed in branch names. It is strongly recommended to stick to lowercase letters, numbers and the "dash" character [**–**].

Behind the scenes: branches are lightweight pointers to commits

```
.git
├── branches
├── COMMIT_EDITMSG
├── config
├── description
├── HEAD
├── hooks
├── index
├── info
│   └── exclude
├── logs
│   ├── HEAD
│   └── refs
│       └── heads
│           ├── develop
│           └── main
├── objects
│   ├── 90
│   │   └── 357ff7068036cb72147cd0bac76115eaea0410
│   ├── 95
│   │   └── e40976f05bf0ece72031c3b2c66ac3ba2ba5d5
│   ├── info
│   └── pack
└── refs
    ├── heads
    │   ├── devel
    │   └── main
    └── tags
```

HEAD pointer: usually pointing at the latest commit of the current branch.

Branch pointers

```
❯ cat .git/refs/heads/main
8508bc698498861c036636dba40ac28b6c7f3a7a

❯ cat .git/refs/heads/devel
4aefde0735e0f95de9969fa660265f71d6a95ebd

❯ cat .git/HEAD
ref: refs/heads/main
```

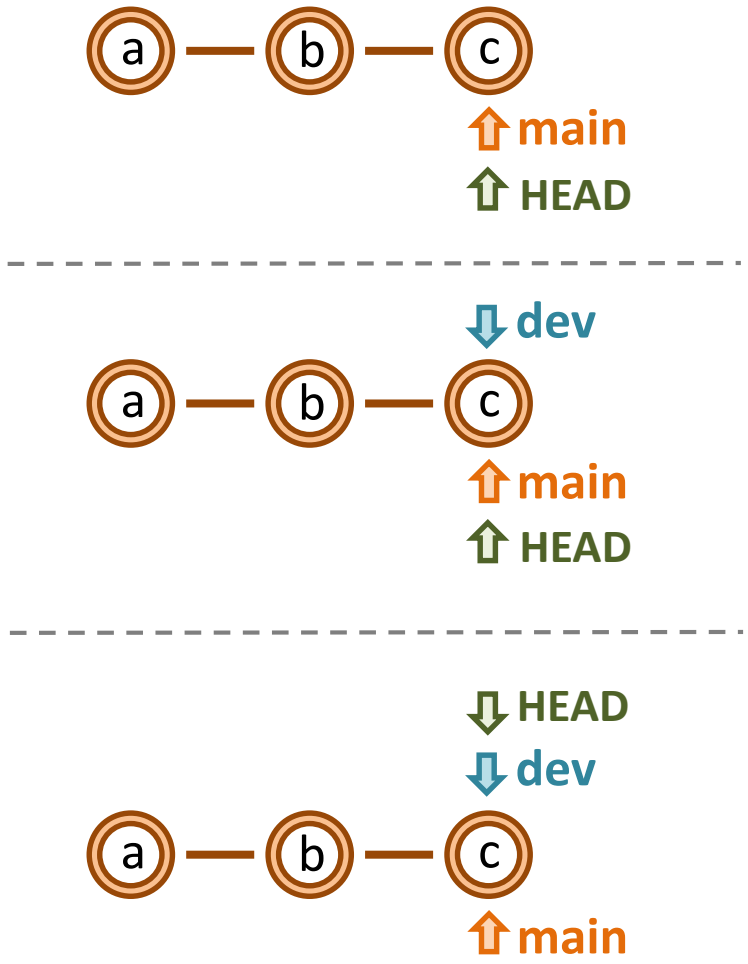
Supplementary material...

Create and **switch** branches

Create a new branch: `git branch <branch name>`

Switch to another branch: `git switch <branch name>`

Create a new branch **and switch** to it: `git switch -c <branch name>`



`git branch dev`

`git switch dev`

`git switch -c dev`

The `-c` option is to create and switch to the new branch immediately.

switch vs. checkout

On older Git versions the `git switch` command does not exist.
Instead, `git checkout` is used to switch branches:

```
git checkout <branch name>
git checkout -b <branch name>
```

The `git switch` command was introduced in Git version 2.23 as an replacement to `git checkout` for switching branches. This was done because the `checkout` command already has other uses (e.g. to extract older files from the Git database), and it was deemed confusing that a same command would have multiple usages. It remains nevertheless possible to switch branches with the `git checkout` command in recent Git versions.

Create and **switch** branches (continued)

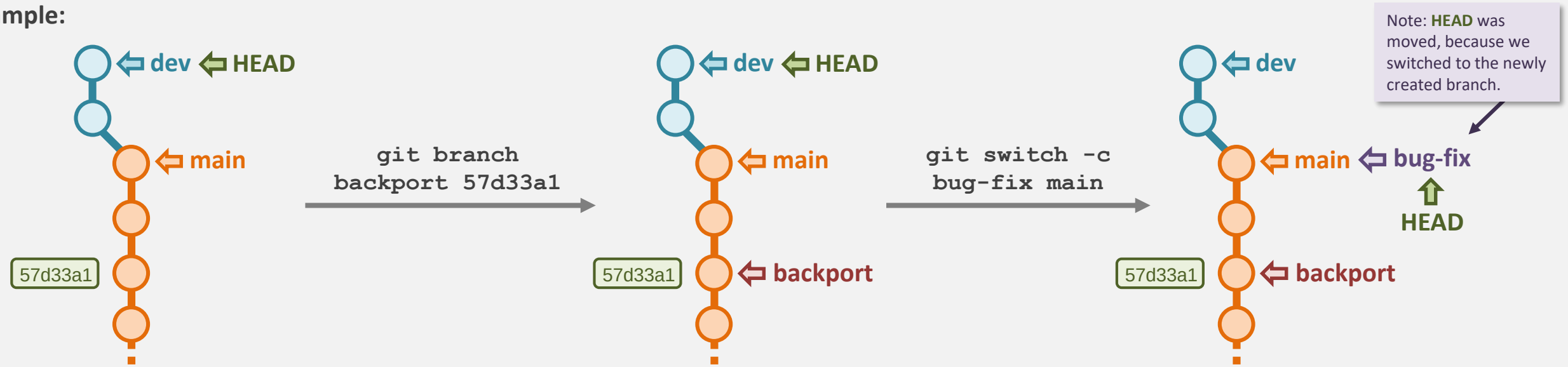
- By default new branches are created at the current position of the **HEAD** pointer (i.e. the current commit).
- But they can be created at any specified reference.

Reference to a commit, branch or tag.
The default reference is **HEAD**.

Create a new branch: `git branch <branch name> <reference>`

Create a new branch **and switch** to it: `git switch -c <branch name> <reference>`

Example:



List branches and identify the currently active branch

`git branch` List **local** branches

`git branch -a` List local **and remote** branches

Examples

```
❯ git branch
devel
* main
new-feature
```

The * denotes the currently checked-out (active) branch. Generally displayed in green.

```
❯ git branch -a
devel
* main
new-feature
remotes/origin/main
remotes/origin/devel
```

Remote branches (to be precise, pointers to remote branches) are shown in red and are named **remotes/<remote name>/<branch name>**

A convenient alternative: `git adog` (`git log --all --decorate --oneline --graph`) will also show all branches.
The currently active branch can be identified as it has the HEAD pointing to it.

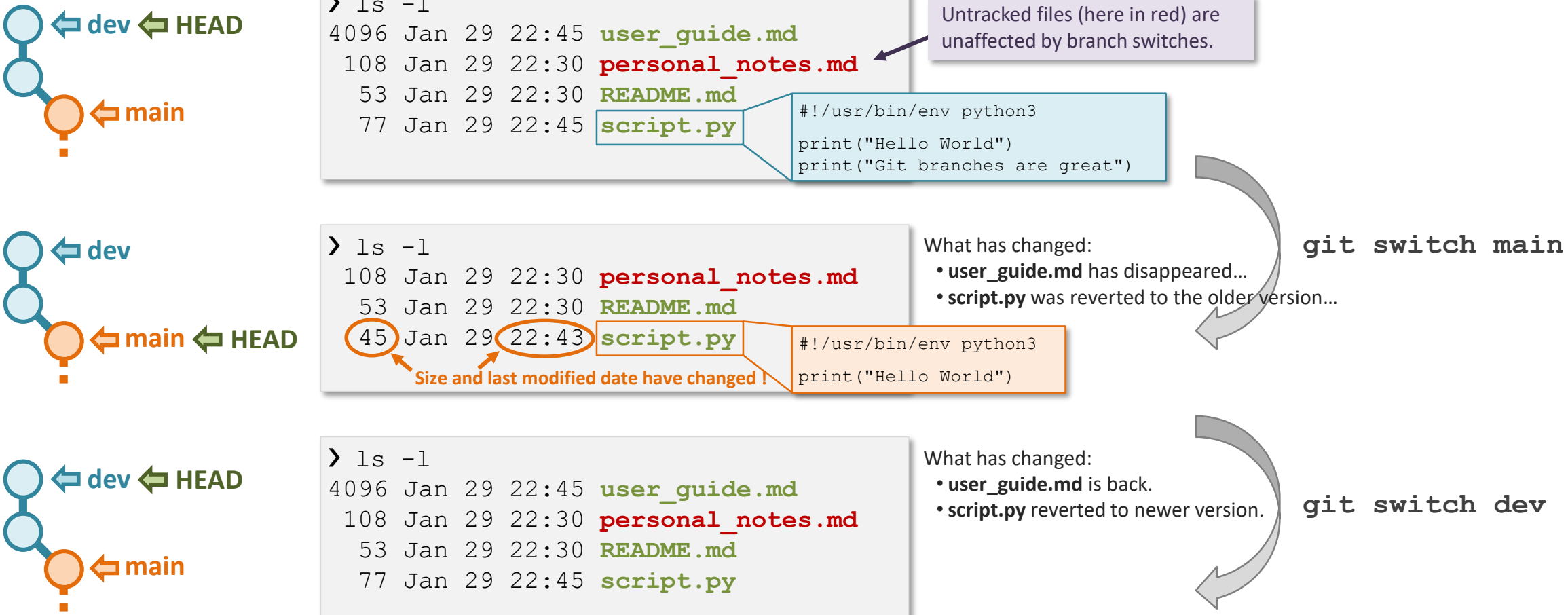
```
* 351dca6 (HEAD -> main, origin/main, origin/HEAD) peak_sorter: added authors to script
* f3d8e22 peak_sorter: display name of highest peak when script completes
| * 076aa80 (origin/feature-dahu, feature-dahu) peak_sorter: display highest peak at end of script
| * d29958d peak_sorter: added authors as comment to script
| * 6c0d087 peak_sorter: improved code commenting
| * 89d201f peak_sorter: add Dahu observation counts to output table
| * 9da30be README: add more explanation about the added Dahu counts
| * 58e6152 Add Dahu count table
|/
* cfd30ce Add gitignore file to ignore script output
```

Demo: `git switch`

- What happens in the working directory when switching branches

What happens in the working tree when switching branches

- When switching to different branch, **the content of your working directory (working tree) is updated** as to reflect the state of the commit the active branch (i.e. the branch you just switched to).
- This means that when switching branches, you can have files appear/disappear or be modified in your working directory.
- A copy of committed files is kept at all times in the **`.git`** repository, so they can be restored when switching branches.



Supplementary material…

What happens in the working tree when switching branches



What if you have uncommitted changes ?

- If the changes do not conflict between the branches, they are “carried-over” with the switch.

README.md (on main)

```
# Git demo project

Demo for the `git switch` command.
This is a new uncommitted line
```

README.md (on dev)

```
# Git demo project

Demo for the `git switch` command.
```

`git switch dev` ✓

README.md (on dev)

```
# Git demo project

Demo for the `git switch` command.
This is a new uncommitted line
```

The uncommitted changes are carried-over to the newly active branch.

- If the changes conflict between the branches, Git will not allow you to switch.

script.py (on main)

```
#!/usr/bin/env python3

print("Hello World")
print("This is uncommitted")
```

script.py (on dev)

```
#!/usr/bin/env python3

print("Hello World")
print("Git branches are great")
```

These two lines are conflicting.

`git switch dev` ✗

```
❯ git switch dev
error: Your local changes to the following
files would be overwritten by checkout:
        script.py
Please commit your changes or stash them
before you switch branches.
Aborting
```

Possible solutions

- **Make a commit on main** with your changes before switching to dev.
- **Use `git stash`** (see 2nd part of this course).
- **Revert changes** on script.py (**warning:** changes are lost for good!):
  `git restore script.py`

Supplementary material...

git merge

get branches back together

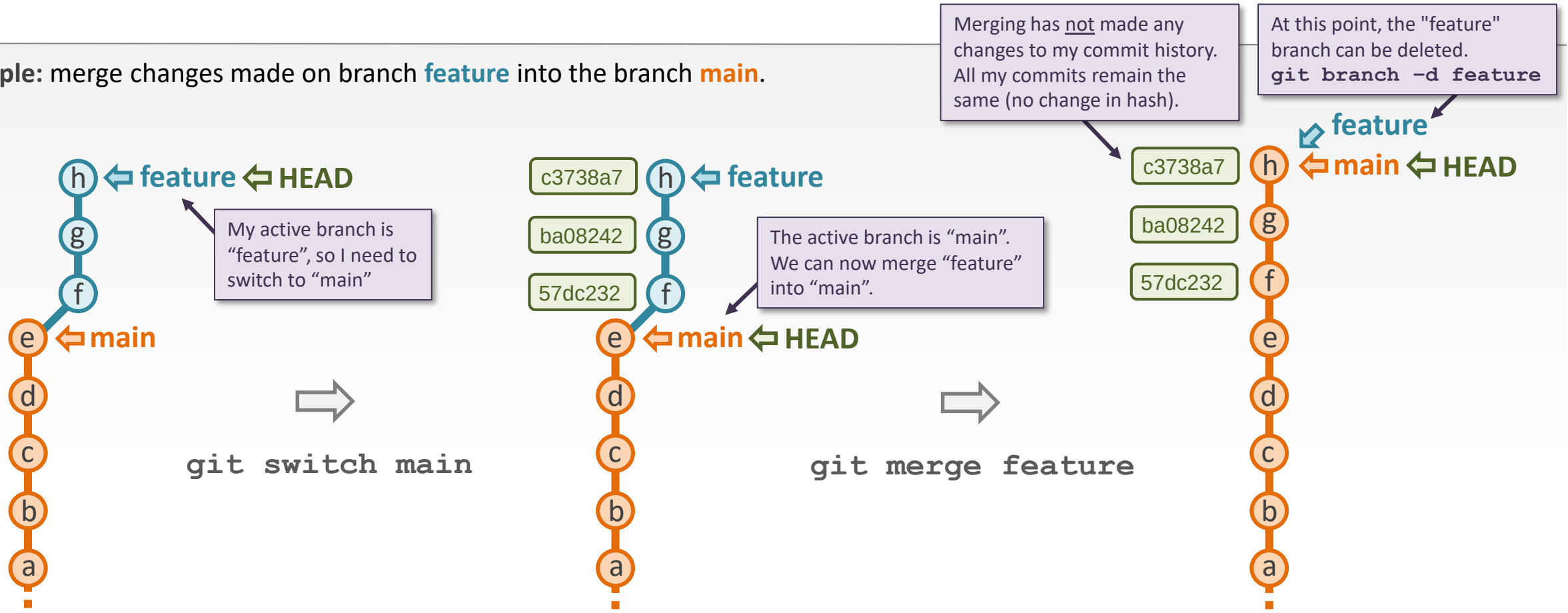
Merging branches

- **Merge:** incorporate changes from the specified branch into the currently active (checked-out) branch.

```
git merge <branch to merge into the current branch>
```

⚠ **Before running the command, make sure that the branch into which the changes should be merged is the currently active branch.** If not, use `git switch <branch>` to checkout the correct branch.

Example: merge changes made on branch **feature** into the branch **main**.



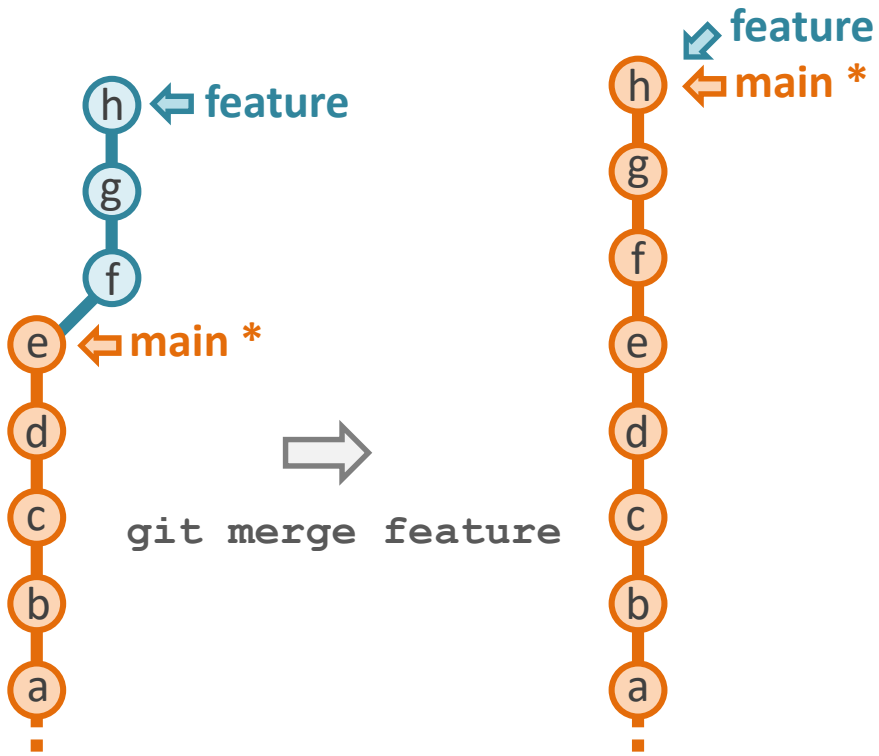
Two types of merges

- **Fast-forward merge:** when branches **have not diverged.**
  The branch that is being merged (here **feature**) is rooted on the latest commit of the branch that it is being merged into (here **main**).

- **3-way merge:** when branches **have diverged**. This introduces an extra "**merge commit**".
  The **common ancestor** of the 2 branches is not the last commit of the branch we merge into (here **main**).

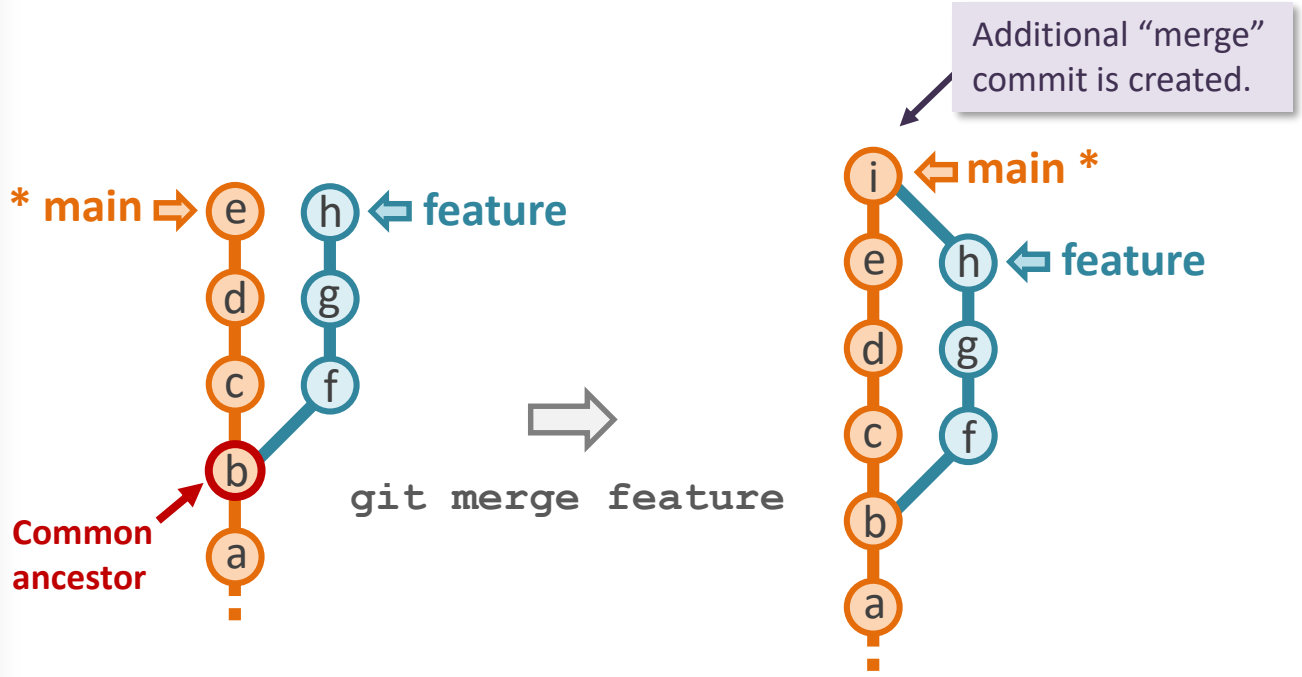
Fast-forward merge

- Guaranteed to be conflict free.



3-way merge (non-fast-forward)

- Creates an additional "merge commit" (has 2 parents).
- Conflicts may occur.



* denotes the currently active (checkout-out) branch.

Conflicts in 3-way merges (non fast-forward)

If **a same file is modified at (or around) the same place** in the two branches being merged, Git cannot decide which version to keep. There is a conflict, and **you need to manually resolve** it.

Story background: the Red Queen has just merged changes from her branch **dev-redqueen** into **main**.
Now Alice wants to merge her branch **dev-alice** into **main**.

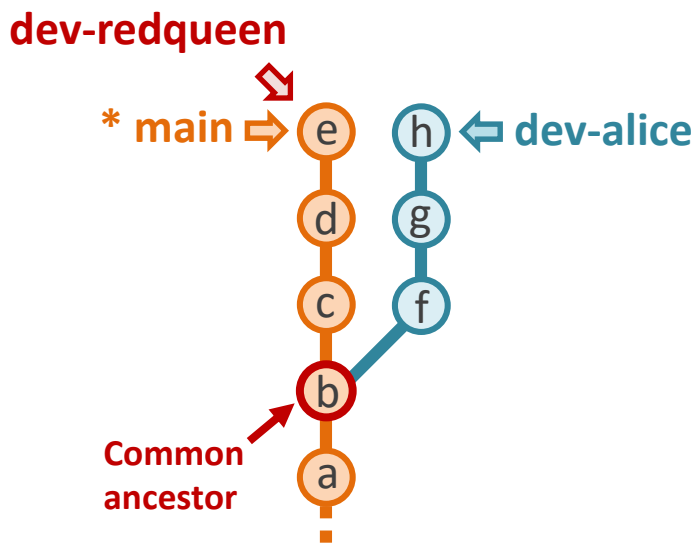
README.md version of main branch.

```
# Tea pot quality-control pipeline
Check and approve tea pots for use in
unbirthday parties.

Authors: Mad Hatter, Red Queen
Date modified: 2022 Oct 10

## Step 1: physical integrity check
* Check exterior for cracks and uneven
  painting.
* Check for mice inside of pot.
* Verify the Mad Hatter is on time.

## Step 2: tea-brewing integration test
* Brew tea for 7 min.
* Add 2 cubes of sugar.
* Probe tea.
```



README.md version of dev-alice branch.

```
# Tea pot quality-control pipeline
Check and approve tea pots for use in
unbirthday parties.

Authors: Mad Hatter, Alice
Date modified: 2022 Oct 11

## Step 1: physical integrity check
* Check exterior for cracks and uneven
  painting.
* Check for mice inside of pot.

## Step 2: tea-brewing integration test
* Brew tea for 7 min.
* Add 2 cubes of sugar.
* Probe tea.
* Make sure we still have no idea why
  a raven is like a writing desk.
```

Let's merge dev-alice into main...

```
$ git merge dev-alice
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
```

← File with conflicts that need to be manually solved.

Resolving conflicts

```
$ git merge dev-alice
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
```

File with conflicts

1. **Open the conflicting files** in the text editor of your choice.
2. **Look for the text between `<<<<<<<` and `>>>>>>>` .**

   There can be more than one of such sections, if there is more than one conflict in the file.

   - The text between `<<<<<<<` and `=======` is the version of the current branch, i.e. the branch into which we merge (**main**, in this example).
   - The text between `=======` and `>>>>>>>` is the version from the branch we are merging (**dev-alice**, in this example).

Version from the current branch (here **main**).

Version from branch being merged into the current branch (here **dev-alice**).

Note: there is no conflict for these 2 lines, because the edits were made at different locations in the file. Git is able to auto-merge such changes.

```
# Tea pot quality-control pipeline
Check and approve tea pots for use in
unbirthday parties.

<<<<<<< HEAD
Authors: Mad Hatter, Red Queen
Date modified: 2022 Oct 10
=======
Authors: Mad Hatter, Alice
Date modified: 2022 Oct 11
>>>>>>> dev-alice

## Step 1: physical integrity check
* Check for mice inside of pot.
* Verify the Mad Hatter is on time.

## Step 2: tea-brewing integration test
* Brew tea for 7 min.
* Add 2 cubes of sugar.
* Probe tea.
* Make sure we still have no idea why a
  raven is like a writing desk.
```

3. Manually edits the file(s)…

```
# Tea pot quality-control pipeline
Check and approve tea pots for use in
unbirthday parties.

Authors: Mad Hatter, Red Queen, Alice
Date modified: 2022 Oct 11

## Step 1: physical integrity check
* Check for mice inside of pot.
* Verify the Mad Hatter is on time.

## Step 2: tea-brewing integration test
* Brew tea for 7 min.
* Add 2 cubes of sugar.
* Probe tea.
* Make sure we still have no idea why a
  raven is like a writing desk.
```

4. Stage the conflict-resolved file(s).
5. Commit

```
$ git add README.md
$ git commit
[main a317d38] Merge branch 'dev-alice'
```

An editor will open with a pre-set commit message. You can accept it as is, or modify it.

Hash of the added "merge" commit.

Resolving conflicts: if you get lost…

- If you are lost at some point, run `git status` and it will give you some hints and commands.
- A merge can be aborted at anytime with `git merge --abort`
- Completed merges can be reverted (with the `git reset` commands – see the "git advanced" slides).

Examples

```
$ git status
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   README.md
```

Git tells you what to do and reminds you of commands.

Output of `git status` before conflicts are resolved in the file.

```
$ git status
On branch main
All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

Changes to be committed:
        modified:   README.md
```

Git tells you what to do and reminds you of commands.

Output of `git status` after conflicts are resolved in the file and the file was staged.

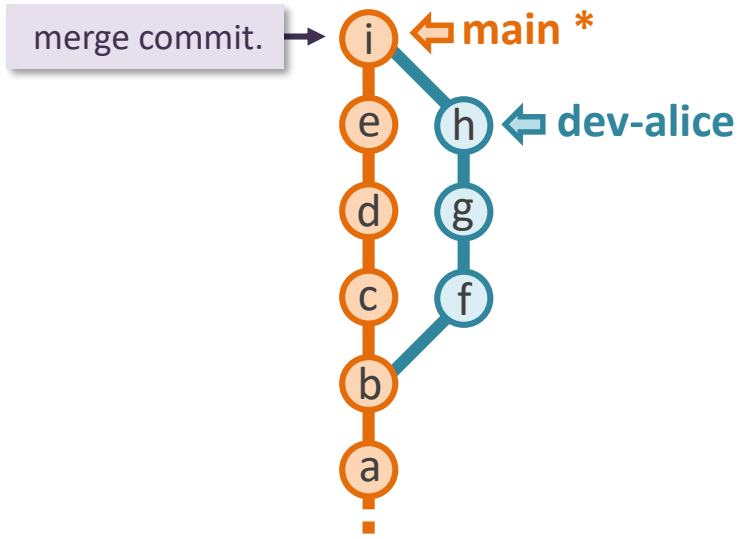
What's in a merge commit ?

If there was no conflict, the **merge commit** contains nothing but the commit message (and other metadata).

```
$ git show HEAD
commit 10fa3ad505821b0ea628b811143af47343a4d8dc (HEAD -> main)
Merge: 7446b3e b4fb462
Author: Red Queen <off.with.their.heads@wonder.org>
Date:   Tue Oct 11 15:16:39 2022 +0200

    Merge branch 'dev-redqueen'
```

If there was a conflict, the **merge commit** contains the conflict resolution changes made to the conflicted file(s).




```
$ git show HEAD
commit a317d38448dae4e6bd9b4862dcaccf4e416cc46c (HEAD -> main)
Merge: 10fa3ad 7999c7c
Author: Alice <alice@redqueen.org>
Date:   Tue Oct 11 15:27:35 2022 +0200

    Merge branch 'dev-alice'

diff --cc README.md
index 647be0c,74edef5..3ce8aa7
--- a/README.md
+++ b/README.md
@@@ -1,8 -1,8 +1,8 @@@
  # Tea pot quality-control pipeline
  Check and approve tea pots for use in unbirthday parties.

- - Authors: Mad-Hatter, Red Queen
- - Date modified: 2022 Oct 10
- - Authors: Mad-Hatter, Alice
++Authors: Mad-Hatter, Red Queen, Alice
+ Date modified: 2022 Oct 11

  ## Step 1: physical integrity check
  * Check exterior for cracks and uneven
```

Supplementary material...

Demo

- Merging branches (fast-forward and 3-way merge)

Deleting branches

Branches that are merged and are not used anymore can (should) be deleted.

`git branch -d <branch name>` ← **safe option:** only lets you delete branches that are fully merged.

`git branch -D <branch name>` ← **YOLO option:** lets you delete any branch.

- Note: A currently active (checked-out) branch cannot be deleted. You must switch to another branch before deleting it.

Examples

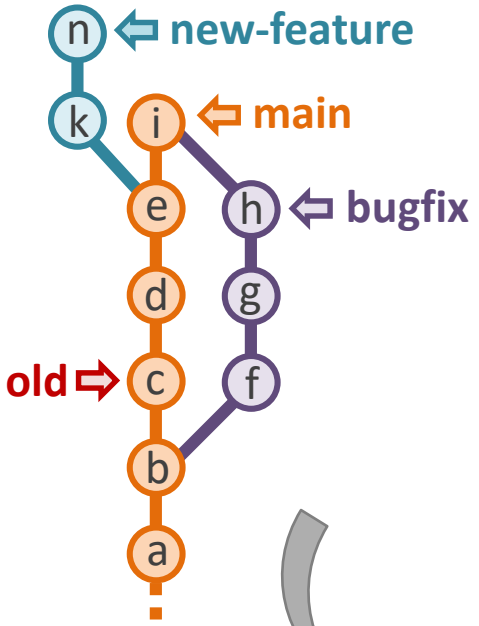
```
# The 'bugfix' and 'old' branches are fully merged. We can use -d.
› git branch -d bugfix
Deleted branch bugfix (was bd898dc)

› git branch -d old
Deleted branch old (was 75d3fed)

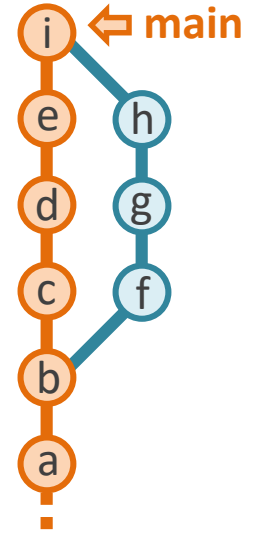
# Trying to delete a non-merged branch with -d will fail.
› git branch -d new-feature
error: The branch 'testing' is not fully merged.
If you are sure you want to delete it, run 'git branch -D testing'.

# -D allows deletion of a non-merged branch.
› git branch -D new-feature
Deleted branch new-feature (was f2a898b)
```

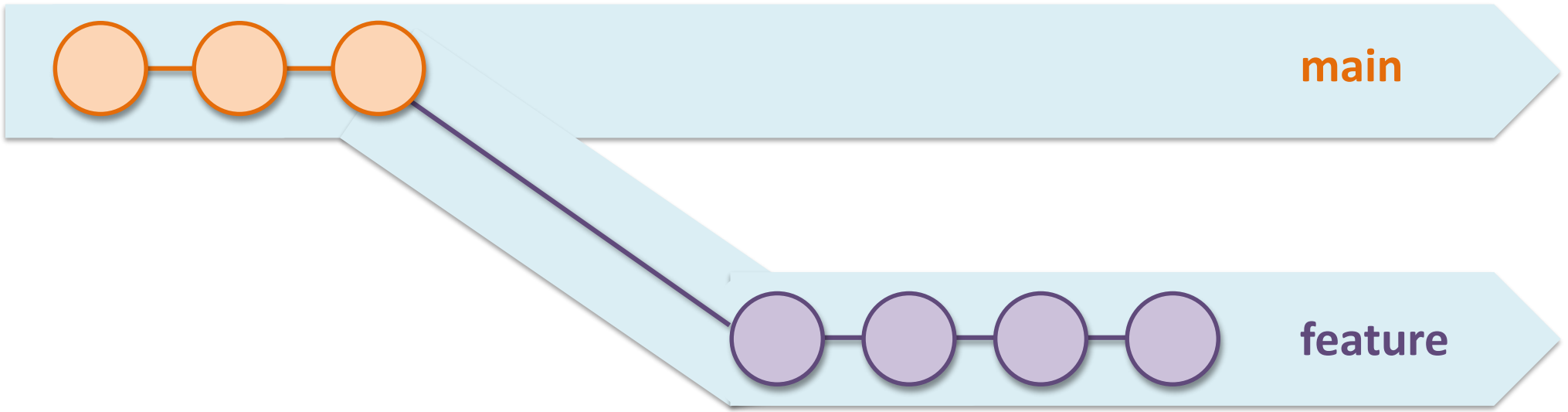
Deleted a branch by mistake ? – no panic !
This hash can be used to re-create it:
`git branch new-feature f2a898b`



Commands from "Example" box



Branch management: best practices



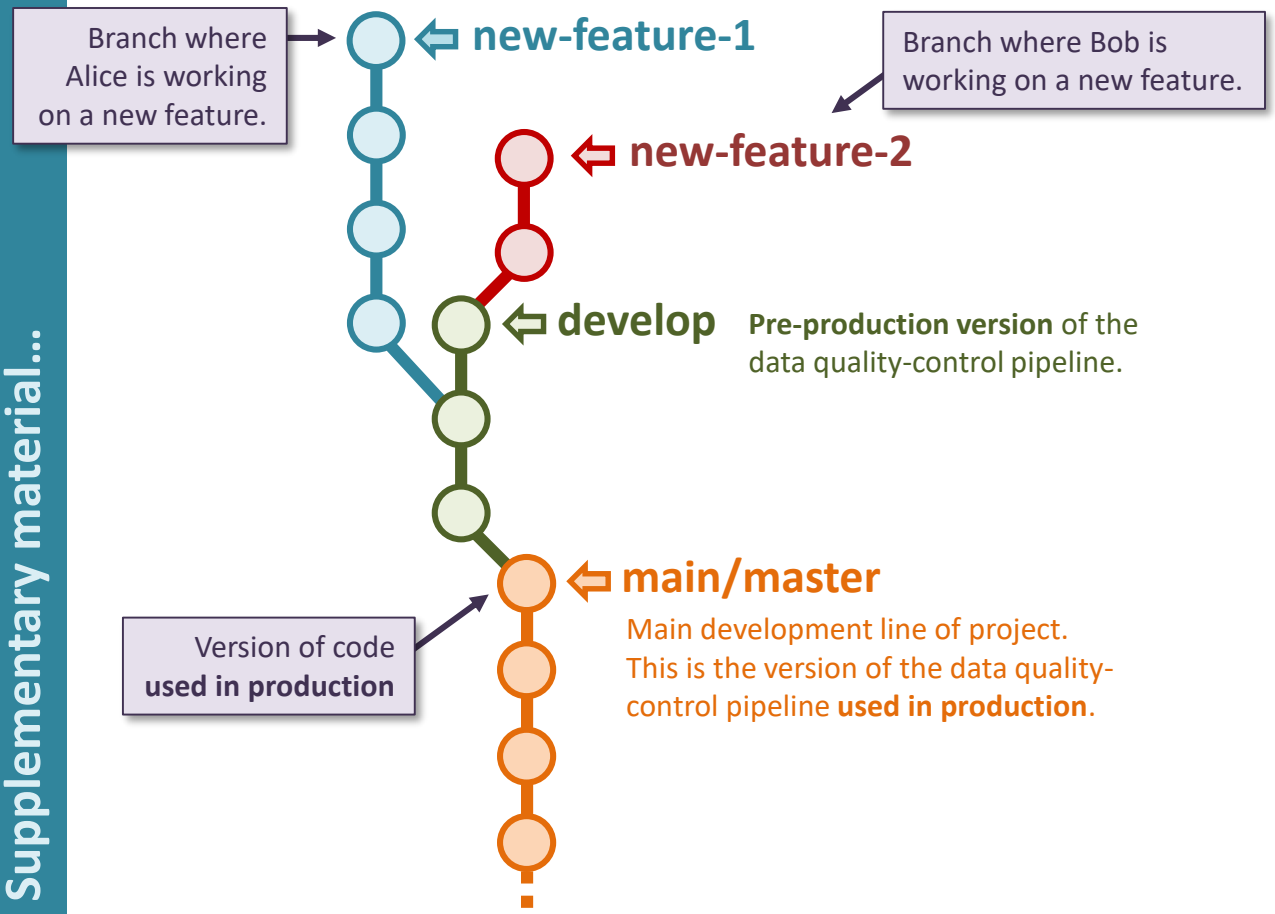
- **Use branches** to develop and tests new changes to your code/scripts - don't test directly on main.
- **Don't hesitate to create branches**, they are "cheap" (they don't add much overhead to the git database).
- Delete branches that are no longer used.

⚠ **Don't change the history on the main branch** if your project is used by others.

Branch management strategies: GitFlow vs. Trunk-based development

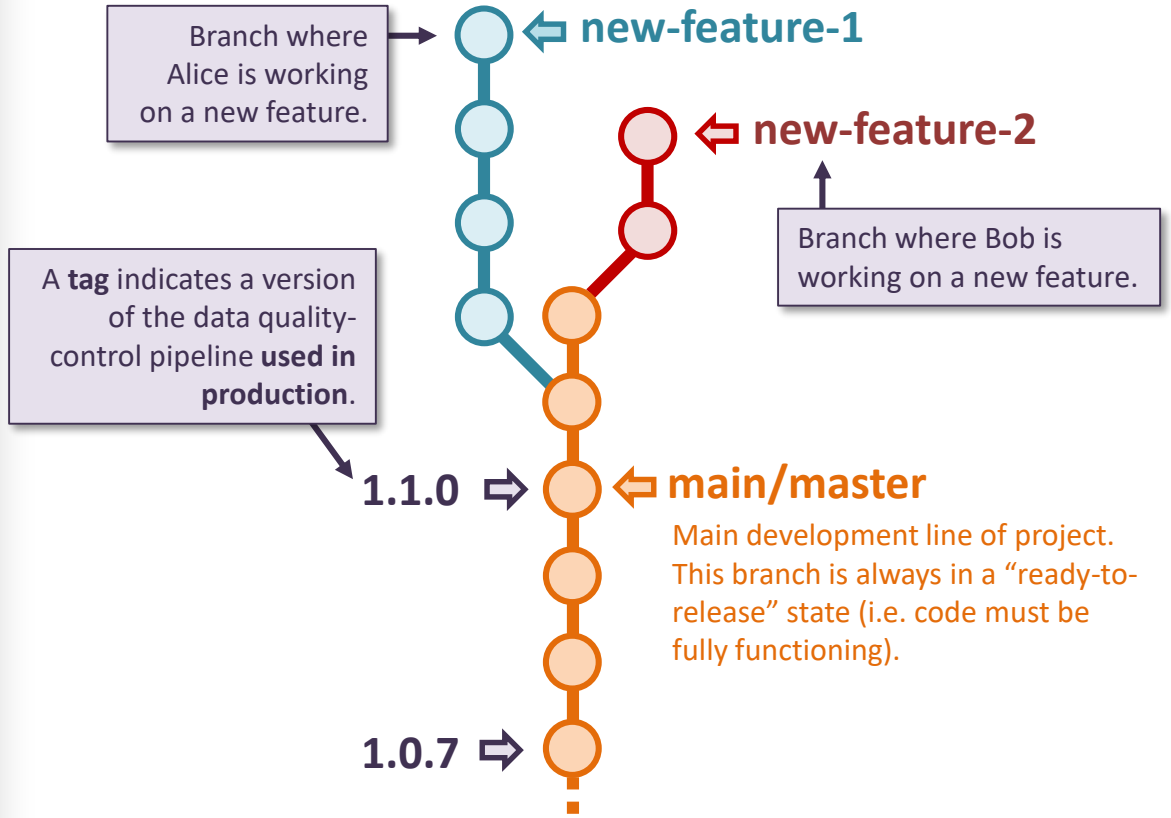
GitFlow: the idea is to have a long-lived **pre-production branch** (here "**develop**"), on which new features are added until ready for a new release, at which point the pre-production branch is merged into **main**.

- Useful if you distribute your code via the **main** branch of the Git repo, without making formal releases, i.e. your end-users use the latest version of **main** in production.



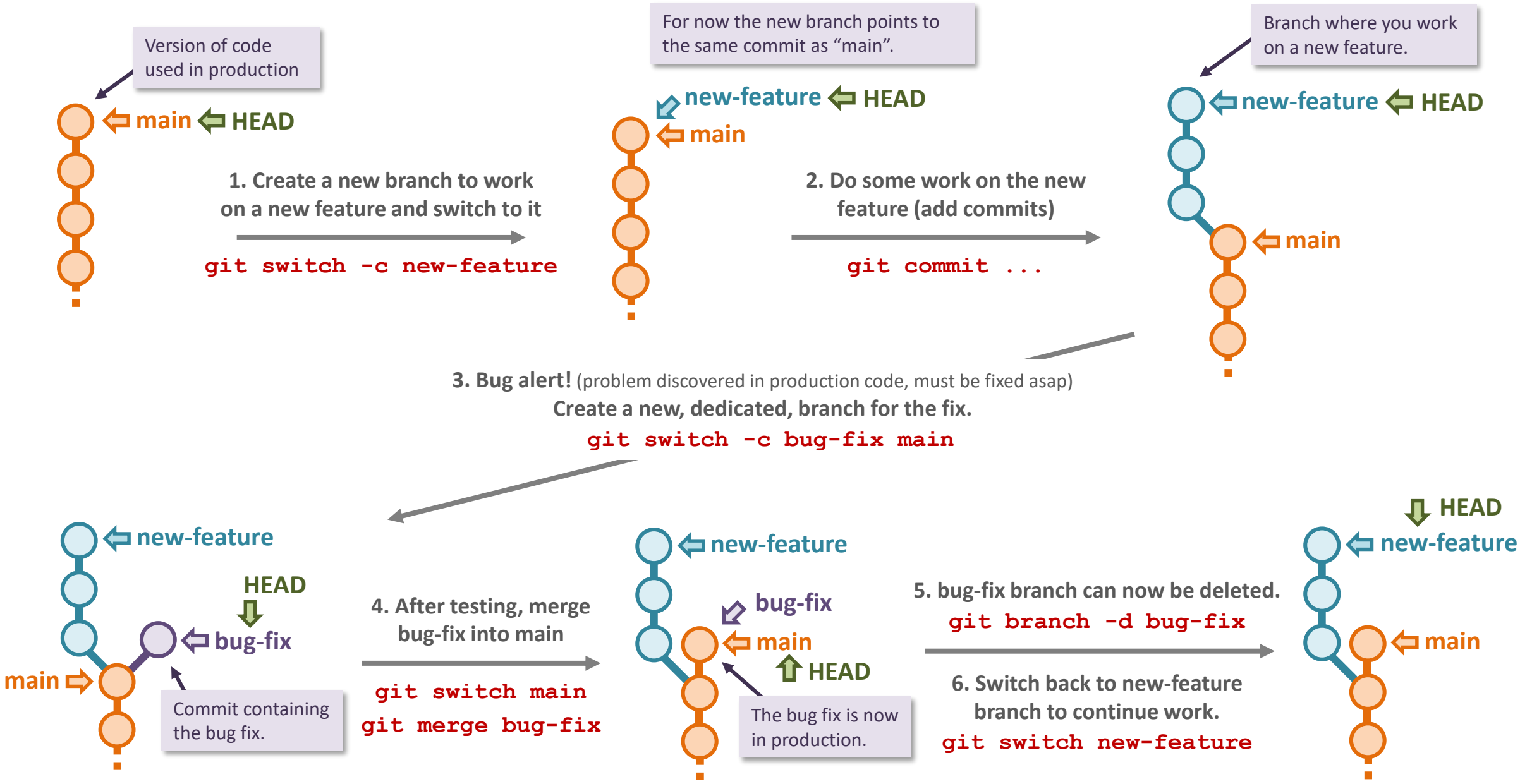
Trunk-based development: there is no long-lived branch outside of the **main** branch. All feature branches are directly merged into **main** once they are completed, and **main** should always be "production-ready". **Tags** are generally added to denote commits corresponding to versions used in production.

- If you **distribute your code via formal releases**, then this strategy makes more sense as it avoids the overhead of managing an extra long-lived branch (the pre-release breach in GitFlow).



Supplementary material...

Recap - example of branched workflow: adding a new feature to an application and fixing a bug



Summary...

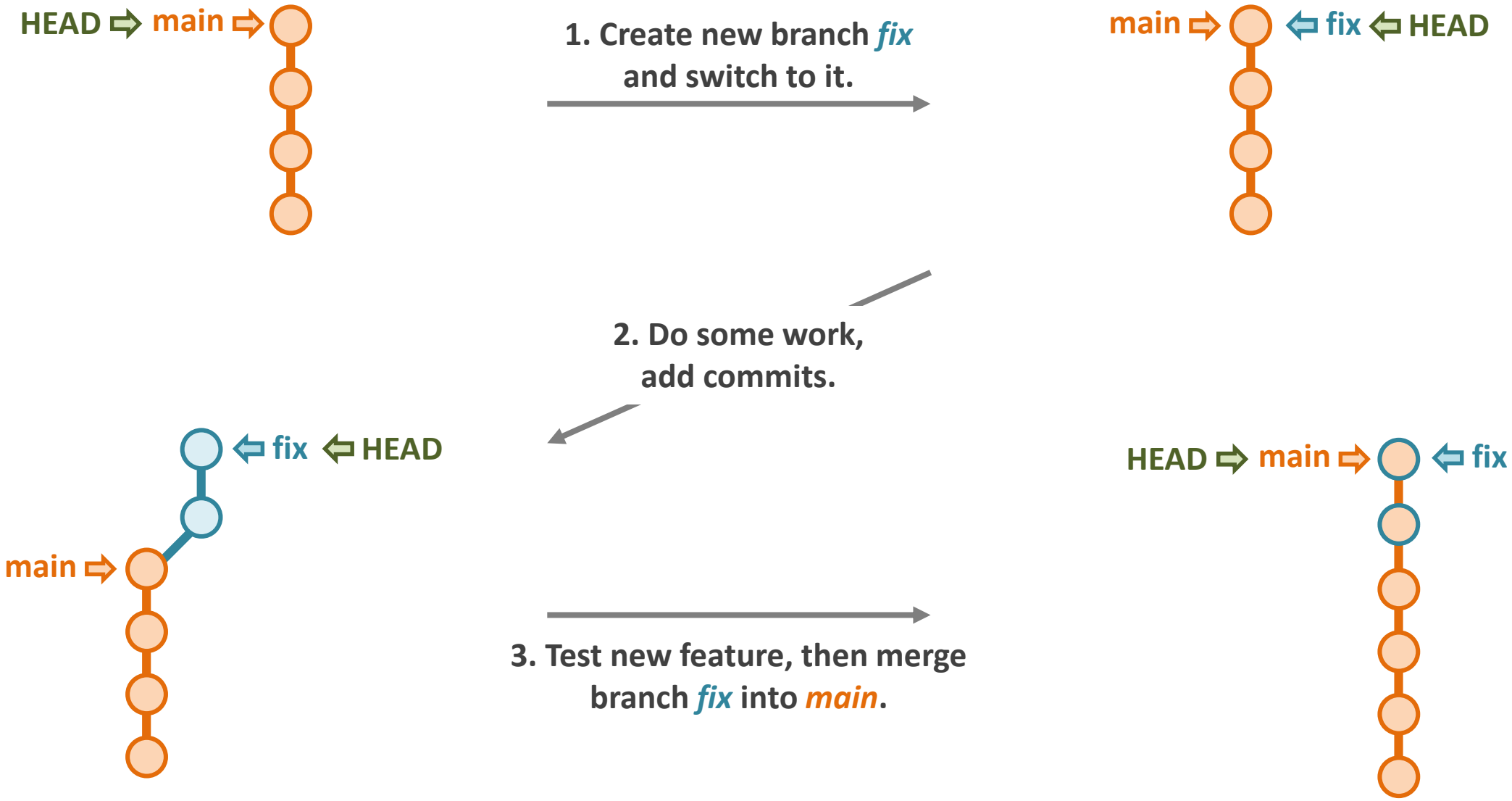
exercise 2

The Git reference webpage



This exercise has helper slides

Exercise 2 help: workflow summary



git rebase

make a linear history

git rebase: replay commits* onto a different base

* To be completely correct, we should say that we replay **differences between commits** (i.e. the changes that commits introduce to our code base), not the commits themselves (a commit is a state of the repo at a given time, it does not directly contain the information of changes to the codebase).

- `git rebase`: move/re-root a branch onto a different base commit.
- **Important**: the rebase command must be executed when on the branch to rebase, not the branch you rebase on.

```
git rebase <branch to rebase on>
```

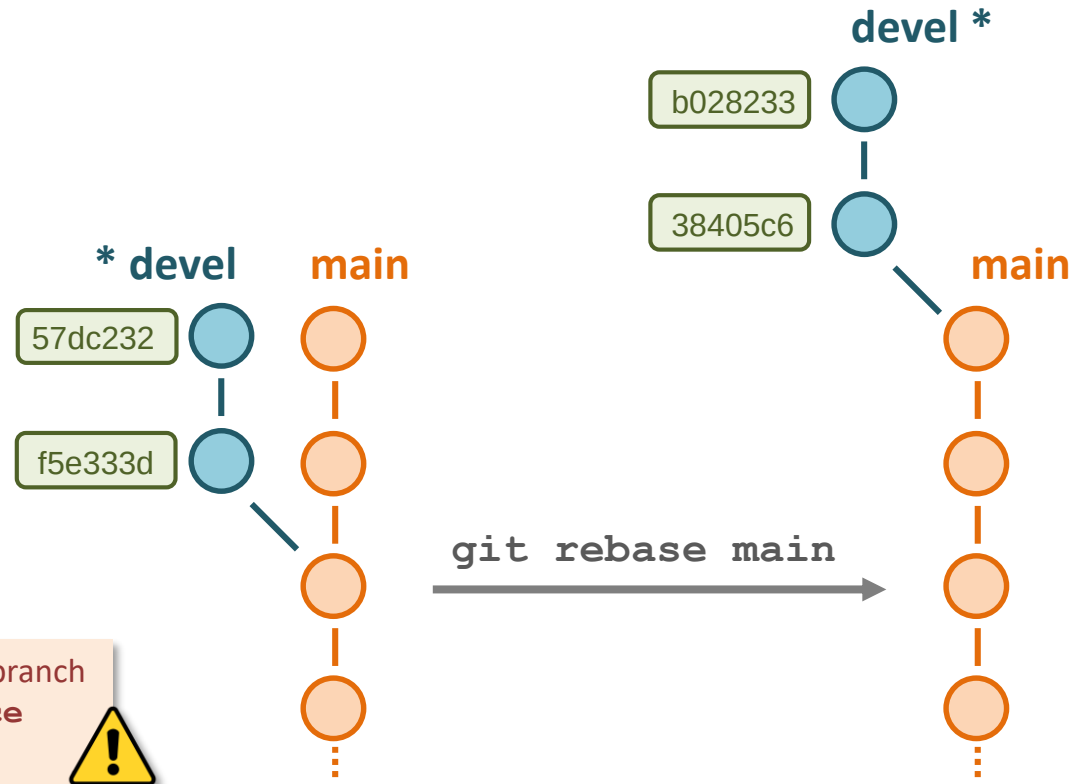
Example:

```
$ git branch
* devel
main

$ git rebase main
```

Make sure you are on the branch you want to rebase !

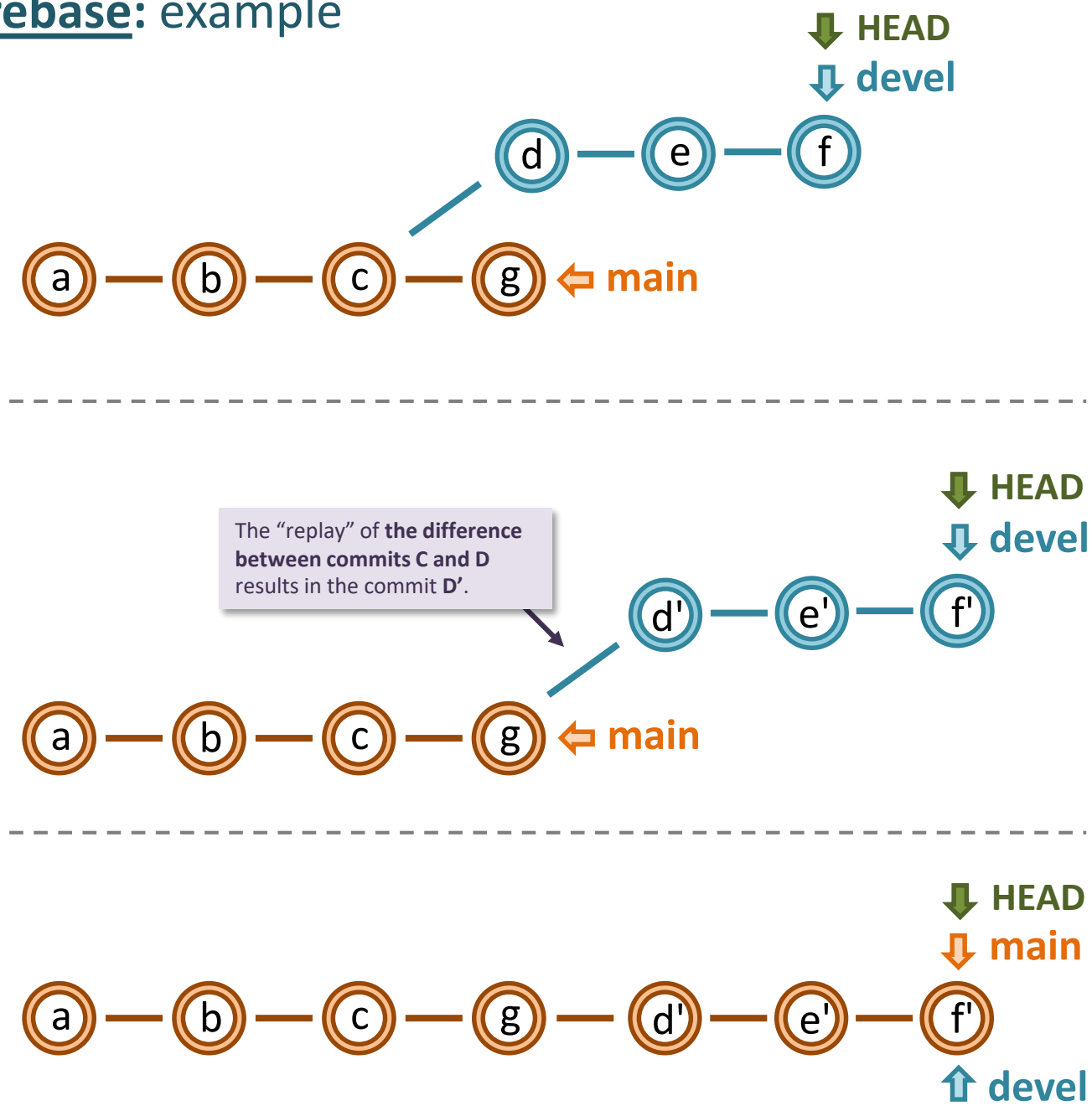
The branch you want to rebase on.



Rebase modifies the history of your branch (new commit ID values). Pushing a rebased branch to a remote requires to overwrite the branch on the remote using `git push --force`

This is not a problem on personal branches (or branches used by few people, if used with parsimony), but should be used with caution on branches used by many people.

git rebase: example



Before starting the rebase: make sure you are on the branch to rebase!

In this case, if were not already on branch *devel* we would run:

```
git switch devel
```

```
git rebase main
```

We can now fast-forward merge. Guaranteed to be conflict free :-)

```
git switch main
git merge devel
```

Resolving conflicts with rebase

- Rebase re-applies all commit to rebase sequentially: **at each step** there is a potential for conflict…
- To resolve conflicts, you will have to (same as for conflict resolution during merges):

1. **Edit the conflicting files**, choose the parts you want to keep, then remove all lines containing `<<<<<<<`, `=======` and `>>>>>>>`.

2. **Mark the files as resolved** with `git add <file>`

1. **Continue the rebase** with `git rebase --continue`

When a conflict arises, Git will provide guidance:

```
$ git rebase main
First, rewinding head to replay your work on top of it...
Applying: first commit on new branch
Using index info to reconstruct a base tree...
 M       new.txt
Falling back to patching base and 3-way merge...
Auto-merging new.txt

CONFLICT (content): Merge conflict in new.txt
error: Failed to merge in the changes.
Patch failed at 0001 first commit on new branch
Use 'git am --show-current-patch' to see the failed patch

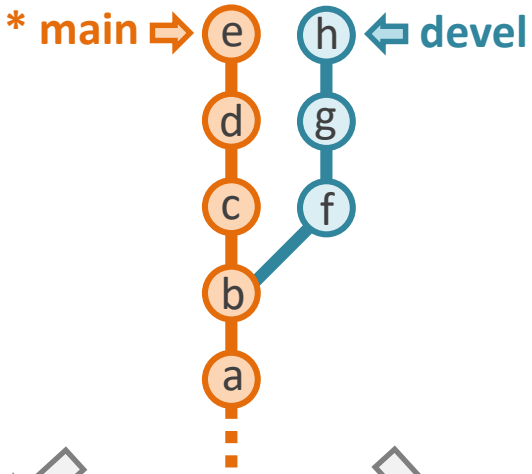
1. → Resolve all conflicts manually,
2. → mark them as resolved with "git add/rm <conflicted_files>"
3. → , then run "git rebase --continue".

You can instead skip this commit: run "git rebase --skip".
To abort and get back to the state before "git rebase",
run "git rebase --abort".
```

Branch reconciliation strategies when history has diverged: merge vs. rebase

merge (3-way merge)

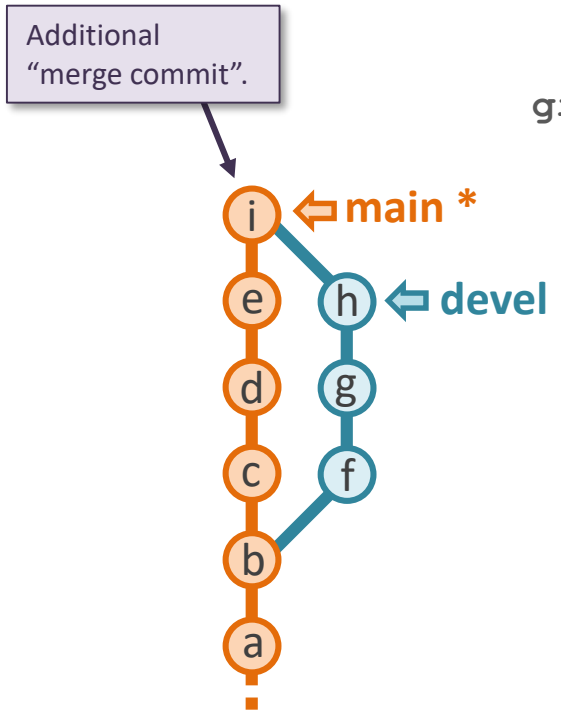
+ **Preserves history perfectly**.
+ Potentials conflicts must be solved only once.
- Creates an additional merge commit.
- Often leads to a "messy" history.



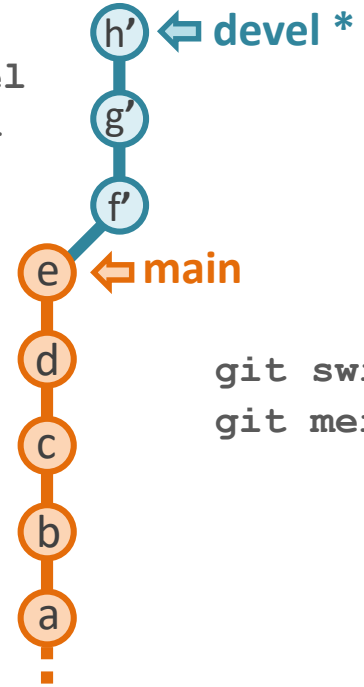
rebase + fast-forward merge

+ **Cleaner history** = easier to read and navigate.
- Conflicts may have to be solved multiple times.
- Loss of branching history.
  History of rebased branch is rewritten, not a problem in general.

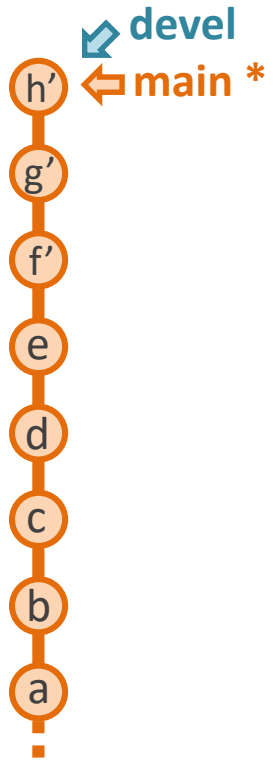
`git merge devel`



`git switch devel`
`git rebase main`



`git switch main`
`git merge devel`



Spoiler-alert: the end result is the same, i and h' have the same content.

Readability vs. history preservation tradeoff

Screenshots of two versions of a **same repository** (in the sense that it contains the exact same content with mostly the same commits).

```
*   a317d38 (HEAD -> main) Merge branch 'dev-alice'
|\
| * 7999c7c (dev-alice) improvement: add success message to QC pipeline
| * da96caa fix: update README
| * ccea24b improvement: better tea-brewing checks
* |   10fa3ad Merge branch 'dev-redqueen'
|\ \
| |/
|/|
| * b4fb462 (dev-redqueen) update: add timing module
| * d37df05 improvement: check that Mad Hatter is on time
|/
* 7446b3e update: add tea-brew integration test
* b82c9c9 update: add physical integrity check to pipeline
* 96d19d4 Initial commit
```

Here, history has been fully preserved, by always using merges and forcing extra merge commits (--no-ff) when needed.

Here, having a linear history has been prioritized (better readability), by rebasing branches before (fast-forward) merging them.

```
* 77d8354 (HEAD -> main, dev-alice) improvement: add success message to QC pipeline
* e48c71a fix: update README
* 51ae05e improvement: better tea-brewing checks
* b4fb462 (dev-redqueen) update: add timing module
* d37df05 improvement: check that Mad Hatter is on time
* 7446b3e update: add tea-brew integration test
* b82c9c9 update: add physical integrity check to pipeline
* 96d19d4 Initial commit
```

Supplementary material...

Supplementary material...

```
*   2501d8d417 (origin/test_node, test_node) Merge pull request #14830 from migueldiascosta/20220124105343_new_pr_EasyBuild452
*   a7f24f6f0c adding easyconfigs: EasyBuild-4.5.2.eb
| * aaa77532dc resume running test suite with Python 3.5 by using actions/setup-python@v2
| * 925cfc73a7 add quotes to avoid that Python 3.10 is interpreted as Python 3.1 ...
| * d780bb7cae stop running easyconfigs test suite with Python 3.5, also test with Python 3.8-3.10
* | 2585d099b8 sync with main + bump version to 4.5.3dev
* |   f1de981545 (tag: easybuild-easyconfigs-v4.5.2, origin/main, eb-source/main) Merge pull request #14829 from easybuilders/4.5.x
|\ \
| * | 4440893abe (eb-source/4.5.x) Merge pull request #14828 from migueldiascosta/eb452
| |/
* | de2a8651cc minor tweak release notes for v4.5.2
* | 0a205792b7 prepare release notes for EasyBuild v4.5.2 + bump version to 4.5.2
|/
*   5f7f1e103e Merge pull request #14821 from branfosj/20220121150125_new_pr_X1120210518
|\
| * 9390faedad add libXfont2 patch to fix build when libbsd is present
|/
*   f497a23162 (origin/scicore) Merge pull request #14743 from sib-swiss/20220117153155_new_pr_RDKit2021034
|\
| * 33c378c1be Update RDKit-2021.03.4: update comic-neue-checksum patch checksum
| * 1c52b44bf3 Update RDKit-2021.03.4: add comic-neue-checksum patch description and author
| * cf0bacffd7 Add patch for hard-coded checksum value of downloaded source file in the source code
| * 73275792fb add missing binutils build dependency to namedlist easyconfig
| * 55ec7565aa adding easyconfigs: namedlist-1.8-GCCcore-11.2.0.eb
|/
*   0ea31891d9 Merge pull request #14806 from boegel/20220120190948_new_pr_R-bundle-Bioconductor314
|\
| * bbfa623c8c add pathview extension to R-bundle-Bioconductor 3.14
|/
*   45acf59f55 Merge pull request #14711 from ItIsI-Orient/20220113183646_new_pr_Short-Pair20170125
|\
| * 70bfefbaca Added required changes
| * be0006d48b Fixed error + edited patch desc
| * 6f85ffb535 adding easyconfigs: Short-Pair-20170125-foss-2021b.eb and patches: Short-Pair-20170125-Python3fix.patch
*   ab3099a9df Merge pull request #14792 from branfosj/20220119163605_new_pr_Pillow-SIMD832
|\
| * c2f8c0ee7c the Pillow v8 patch also works for Pillow-SIMD v7
| * 9e333911ca fix CVE-2021-23437 in Pillow-SIMD v8 + add Pillow-SIMD v8.3.2 in easyconfigs using a 2021b toolchain
|/
*   aa46b3ecf1 Merge pull request #14548 from shot0829/20211213195043_new_pr_elbencho203
|\
```

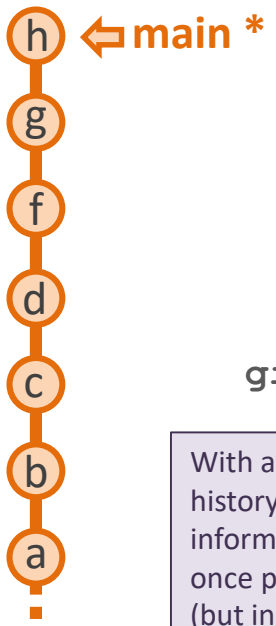
Never rebasing your changes before merging can lead to a hard to read history…

Ultimate history preservation: force the addition of a merge commit with `--no-ff`

When keeping an exact record of how the history of a Git repo came into existence is of prime importance, some people like to **add a merge commit even if a fast-forward merge is possible**.

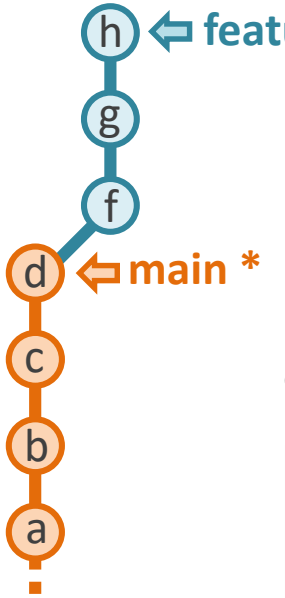
This is possible by adding the `--no-ff` option ("no fast-forward") to git merge.

```
git merge --no-ff <branch to merge>
```



`git merge feature`

With a regular fast-forward merge, the history is cleaner. However, the information that "f", "g" and "h" were once part of a different branch is lost (but in most cases this doesn't matter).

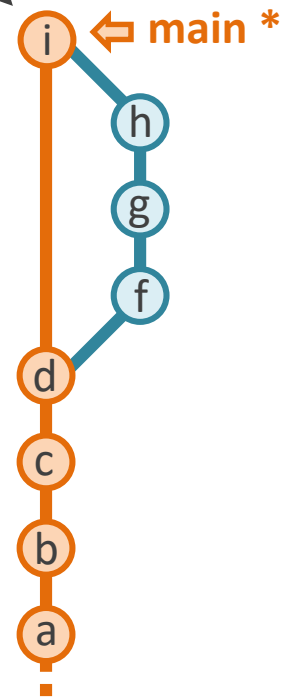


`git merge --no-ff feature`

The merge commit " i " is added for the sole purpose of allowing us to reconstruct the exact history of the repo: it tells us that commits "f", "g" and "h" were once part of a different branch, which was then merged into "main".

```
$ git show 10fa3ad
commit 10fa3ad505821b0ea628b8
Merge: 7446b3e b4fb462
Author: Alice <alice@redqueen.org>
Date: Tue Oct 11 15:16:39 2022 +0200

    Merge branch 'feature'
```



Supplementary material...

Demo

- Rebasing a branch (feat. manual conflict resolution)

Cherry-picking: copy-pasting commits

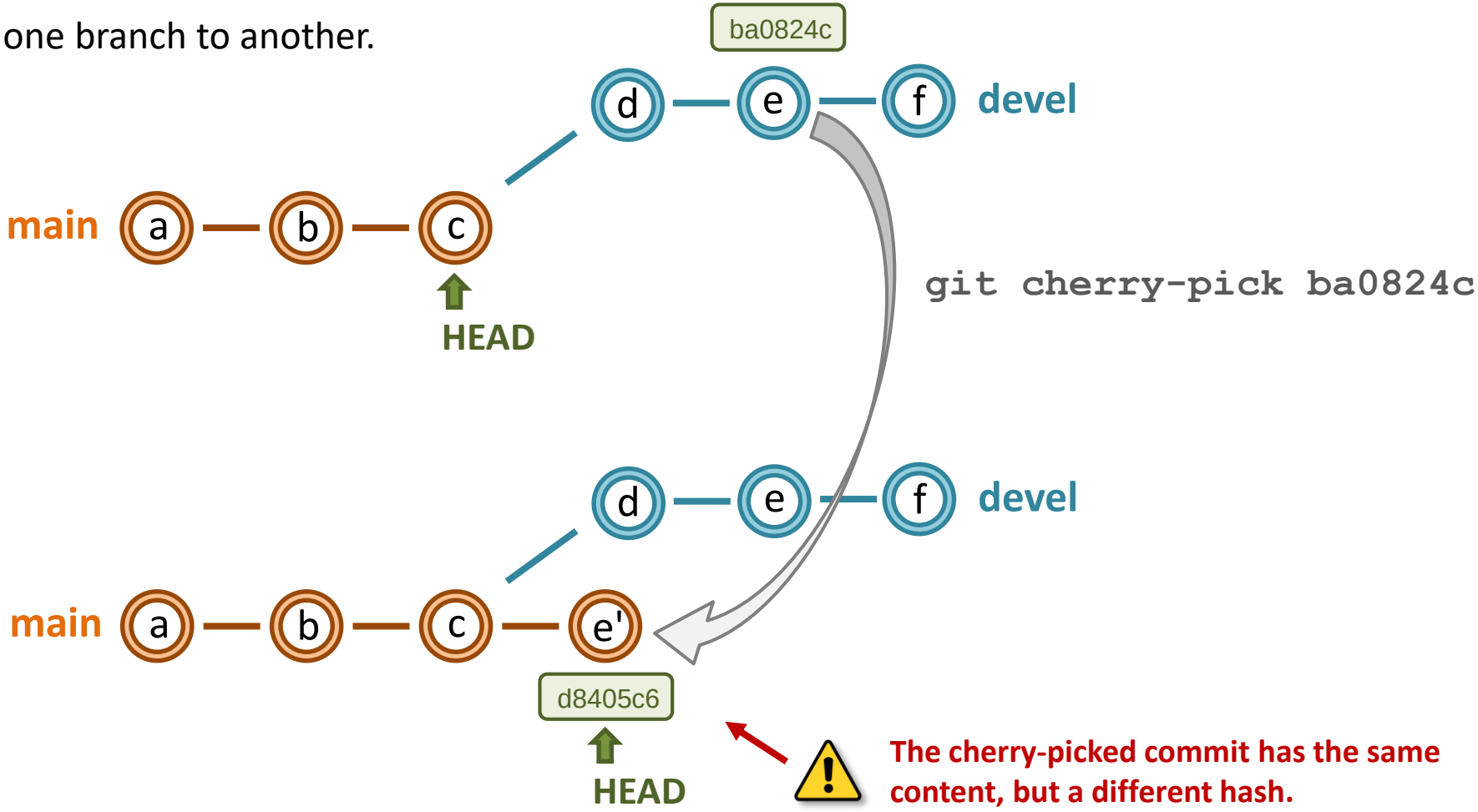
```
git cherry-pick
```

Cherry-pick: merge a single commit into the current branch

- `git cherry-pick`: "copy" a commit (or several) to the current branch.

```
git cherry-pick <commit to pick>
```

Example:
"copy" a fix from one branch to another.



Cherry-pick use case: simplify history

```
* 0cc45cf (HEAD -> master) Update and fix bugs in exercises and slides
| * 4e08672 (origin/master, origin/HEAD) Add newline before list in Cleanup.
| *   5935b22 Merge branch 'master' of gitlab.sib.swiss:rengler/git_course
| |\
| |/
|/|
* | 9164384 Update slides
| * 8672e4b Hopefully make the list in ex4 look good now.
|/
* 7fdb6b1 Update exercises
* 4ad684b Rename files
* b7a33b1 Add exercise for git stash
```

All we really need is this commit

Situation:

- The local and remote `main` branches have a complicated history.
- But in the end, all that is really needed from the `origin/main` branch is `8672eb4`.
- Should I merge my 2 branches ?

`git cherry-pick 8672eb4`

```
* 8473b64 (HEAD -> master) Hopefully make the list in ex4 look good now.
* 0cc45cf Update and fix bugs in exercises and slides
| * 4e08672 (origin/master, origin/HEAD) Add newline before list in Cleanup.
| *   5935b22 Merge branch 'master' of gitlab.sib.swiss:rengler/git_course
| |\
| |/
|/|
* | 9164384 Update slides
| * 8672e4b Hopefully make the list in ex4 look good now.
|/
* 7fdb6b1 Update exercises
* 4ad684b Rename files
* b7a33b1 Add exercise for git stash
```

`git push --force`

overwrites remote branch `origin/main` with content from local `main`.

```
* 8473b64 (HEAD -> master, origin/master, origin/HEAD) Hopefully make the list in ex4 look good now.
* 0cc45cf Update and fix bugs in exercises and slides
* 9164384 Update slides
* 7fdb6b1 Update exercises
* 4ad684b Rename files
* b7a33b1 Add exercise for git stash
```

✓ Clean history :-)

Supplementary material...

Retrieve data from earlier commits

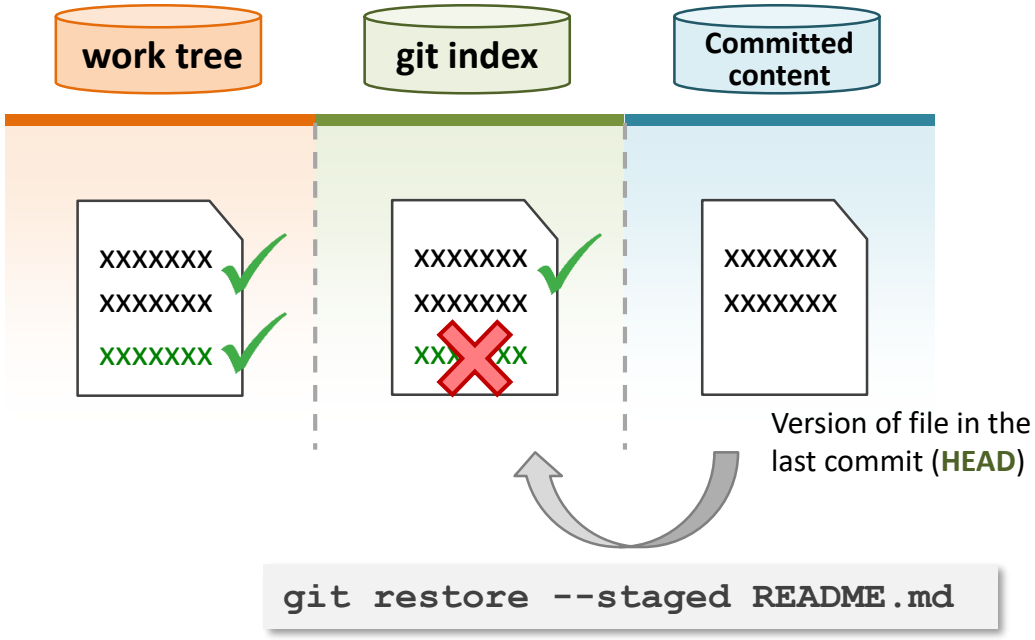
```
git restore
```

```
git checkout
```

Un-stage file modifications (restore file in index)

```
git restore --staged <file name>
```

- Restores the content of a file in the Git index back to the latest commit (**HEAD** commit).
- Does not modify files in the working tree.



Example: un-stage changes to README.md file.

```
$ git status
On branch main
Changes to be committed:
(use "git restore --staged <file>..." to unstage)
          modified:   README.md
```

```
$ git restore --staged README.md
```

```
$ git status
On branch main
Changes not staged for commit:
(use "git add <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
          modified:   README.md
```

The file is still modified in the working directory, but the changes are no longer staged.

Reminder...

Restore / checkout of individual files

Retrieving the content of a file from an earlier commit can be done with either:

```
git restore -s/--source <commit reference> <file name>
```

or

If no commit references is specified, the file is retrieved from the index.

```
git checkout <commit reference> <file name>
```

Warning: these commands will overwrite existing versions of the retrieved file in your working tree (without any sort of warning). Make sure you don't have uncommitted changes you want to keep.

Examples: the `<commit reference>` can be e.g. a commit ID, a relative reference, a tag or a branch name.

```
$ git restore -s ba08242 output.txt
$ git restore -s HEAD~10 output.txt
$ git restore -s v2.0.5  output.txt
$ git restore -s devel-branch output.txt
```

using a branch name, implicitly refers to the latest commit on the branch.

```
$ git checkout ba08242 output.txt
$ git checkout HEAD~10 output.txt
$ git checkout v2.0.5  output.txt
  Updated 1 path from 2a7fac8
$ git checkout devel-branch output.txt
  Updated 1 path from e55fa6f
```

A small difference between these two commands is that `restore` updates the file only in the working tree (i.e. the files in your working directory), while `checkout` updates both the working tree and the index.

```
$ git restore --source ad26560 README.md
$ git status
Changes not staged for commit:
(use "git restore <file>..." to discard changes
in working directory)
        modified:   README.md
```

```
$ git checkout ad26560 README.md
Updated 1 path from e55fa6f
$ git status
Changes to be committed:
(use "git restore --staged <file>..." to unstage)
        modified:   README.md
```

Checkout of the entire repo state at an earlier commit

- Checking out a commit will restore both the working tree and the index to the exact state of the specified commit.
- It will also move the **HEAD** pointer to that commit.

```
git checkout <commit reference>
```

Examples:

```
$ git checkout ba08242
$ git checkout HEAD~10
$ git checkout v2.0.5
```

Make sure to have a clean working tree before doing a checkout!

```
$ git checkout ad26560
error: Your local changes to the following files would be
overwritten by checkout:
        README.md
Please commit your changes or stash them before you switch branches
```

- After a checkout, you enter a "detached HEAD" state….

→

```
$ git checkout ba08242
Note: checking out 'ba08242'.

You are in 'detached HEAD' state. You can look
around, make experimental changes and commit them,
and you can discard any commits you make in this
state without impacting any branches by performing
another checkout.
```

- To get back to a “normal” state you should go back to a regular branch:

`git switch <branch>` or `git checkout <branch>`

exercise 3

The crazy peak sorter script



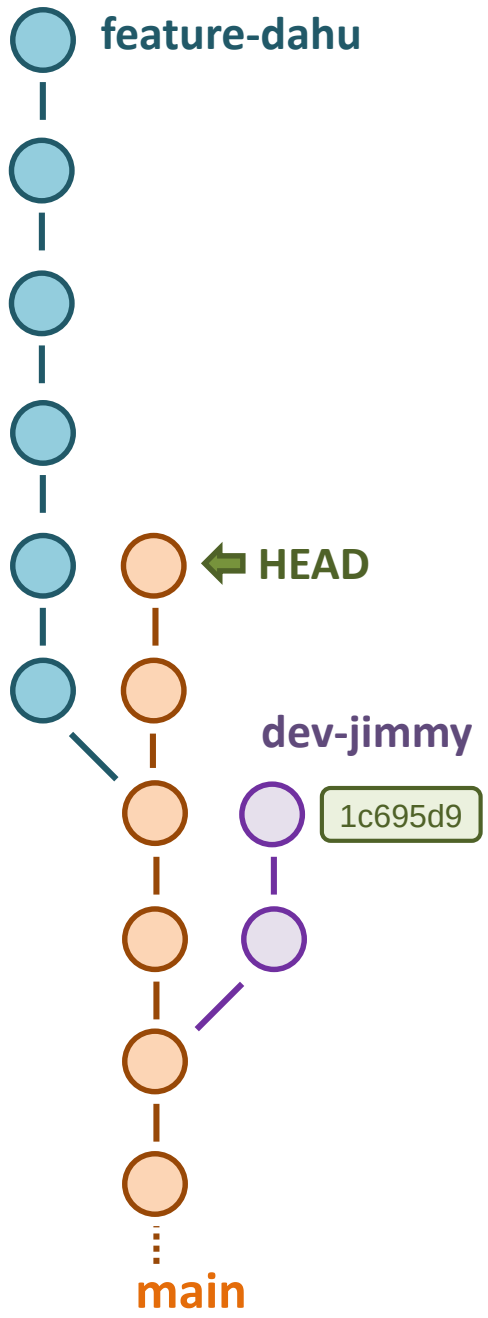
This exercise has helper slides

Exercise 3 help: history of the peak-sorter repo

This slide shows the history of the repo for exercise 3, both as the command line output and as a schematic representation (on the right).

This can help you understand the command line representation of a repo's history.

```
[rengler@local peak_sorter]$ git log --all --decorate --oneline --graph
* fc0b016 (origin/feature-dahu) peak_sorter: display highest peak at end of script
* d29958d peak_sorter: added authors as comment to script
* 6c0d087 peak_sorter: improved code commenting
* 89d201f peak_sorter: add Dahu observation counts to output table
* 9da30be README: add more explanation about the added Dahu counts
* 58e6152 Add Dahu count table
| * f6ceaac (HEAD -> master, origin/master, origin/HEAD) peak_sorter: added authors to script
| * f3d8e22 peak_sorter: display name of highest peak when script completes
|/
* cfd30ce Add gitignore file to ignore script output
* f8231ce Add README file to project
| * 1c695d9 (origin/dev-jimmy) peak_sorter: add check that input table has the ALTITUDE and PEAK columns
| * ff85686 Ran script and added output
|/
* 821bcf5 peak_sorter: add +x permission
* 40d5ad5 Add input table of peaks above 4000m in the Alps
* a3e9ea6 peak_sorter: add first version of peak sorter script
```



Part III

Working with **remotes**

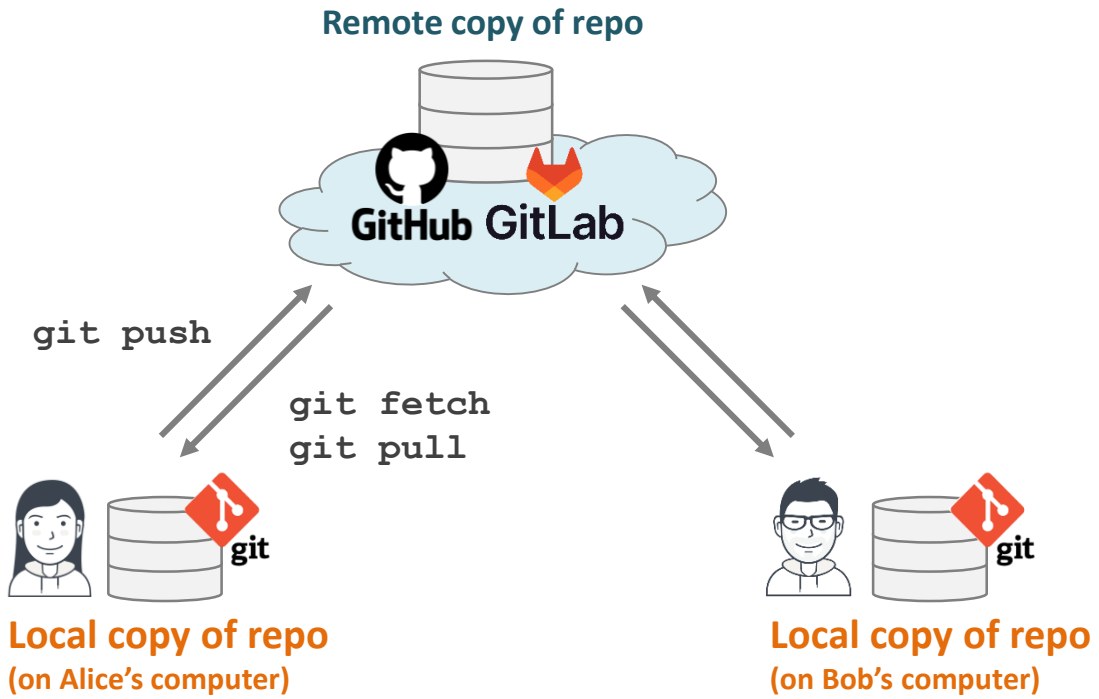
Linking your local repo with an online server

What is a “remote” ?

A **remote** is a copy of a Git repository that is stored on a server (i.e. online).

Remotes are very useful, as they allow you to:

- **Backup** your work.
- **Collaborate** and synchronize your repo with other team members.
- **Distribute** your work – i.e. let other people clone your repo (e.g. like the repo of this course).



Good to know:

- Each copy of a Git repo (local or online) is a **full copy of the entire repo’s history** (provided it has been synced).
- Git does not perform any automatic sync between the local and remote repos. All **sync operations must be manually triggered**.

Remotes are generally hosted on dedicated servers/services, such as GitHub, GitLab (either gitlab.com or a self-hosted instance), BitBucket, ...

Add a remote to an existing project (or update a remote's URL)

- **Case 1:** your local **repo was cloned** from a remote – *nothing to do* (the remote was automatically added by Git).
- **Case 2:** your local **repo was created independently** from the remote – it must be linked to it.

Add a new remote: `git remote add <remote name> <remote url>`

Change URL of remote: `git remote set-url <remote name> <remote url>`

Note: by convention, the `<remote name>` is generally set to `origin`.

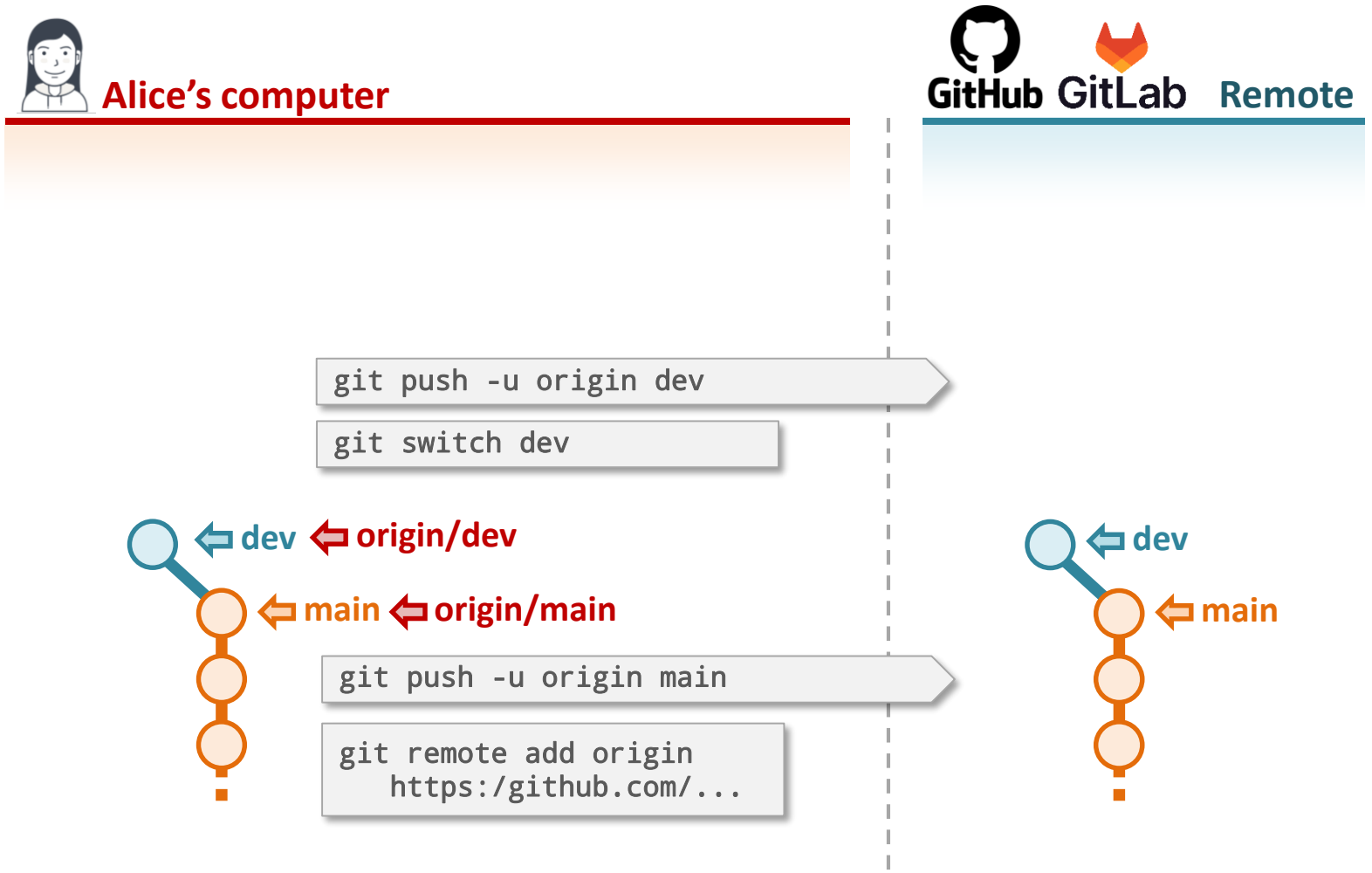
Examples

```
# Add a new remote (named origin) to the local repo.
❯ git remote add origin https://github.com/sibgit/test.git

# Update the URL of the existing origin remote. In this example, the remote was moved GitLab.
❯ git remote set-url origin https://gitlab.sib.swiss/sibgit/test.git
```

Supplementary material...

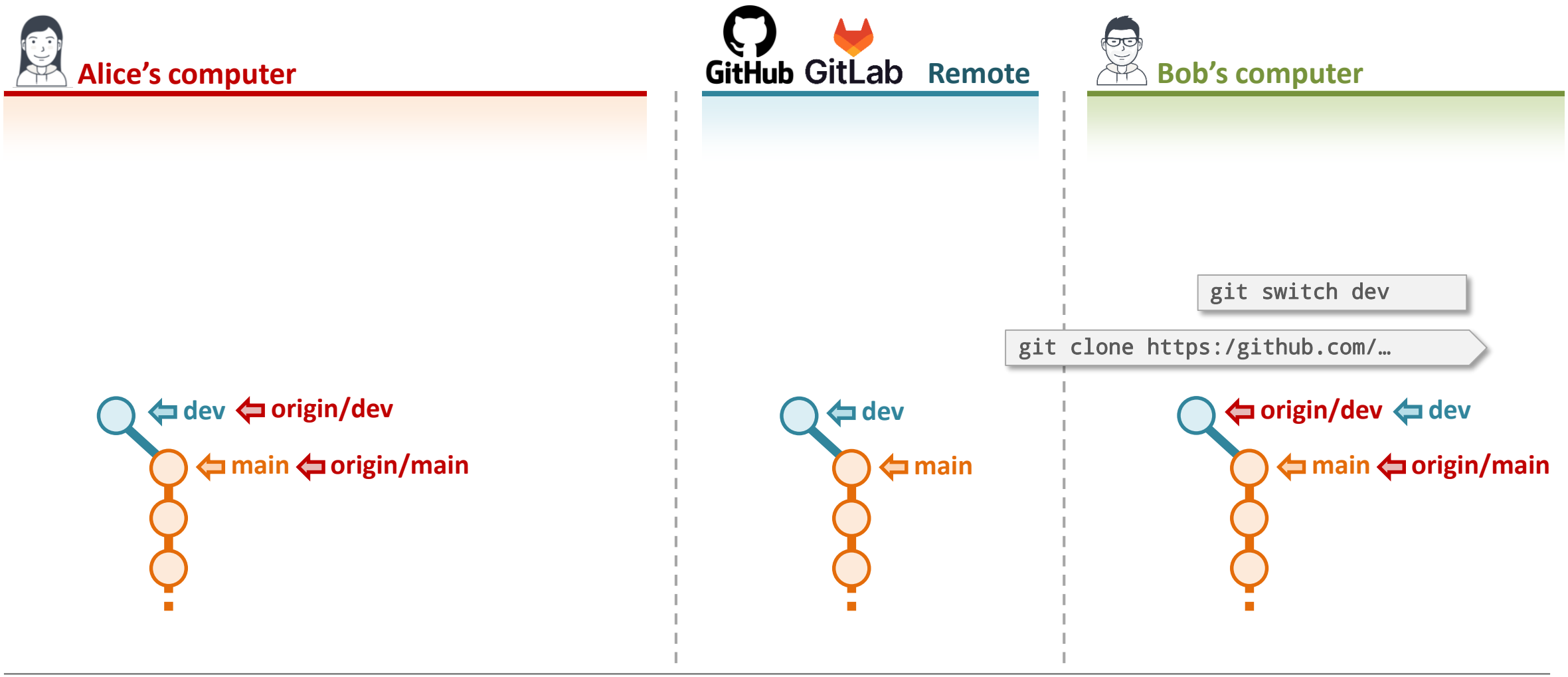
Example – part 1: creating a new remote and pushing new branches



Alice has a Git repo with 2 branches: **main** and **dev**. She now wants to store her work on GitHub, to collaborate and have a backup.

1. She creates a remote on GitHub and links it to her local repo using `git remote add origin <URL of remote>`
2. She pushes her branch **main** to the remote using `git push -u origin <branch name>`
   (at this point the branch has no upstream, so the `-u/--set-upstream` option must be used).
3. She pushes her branch **dev** to the remote.

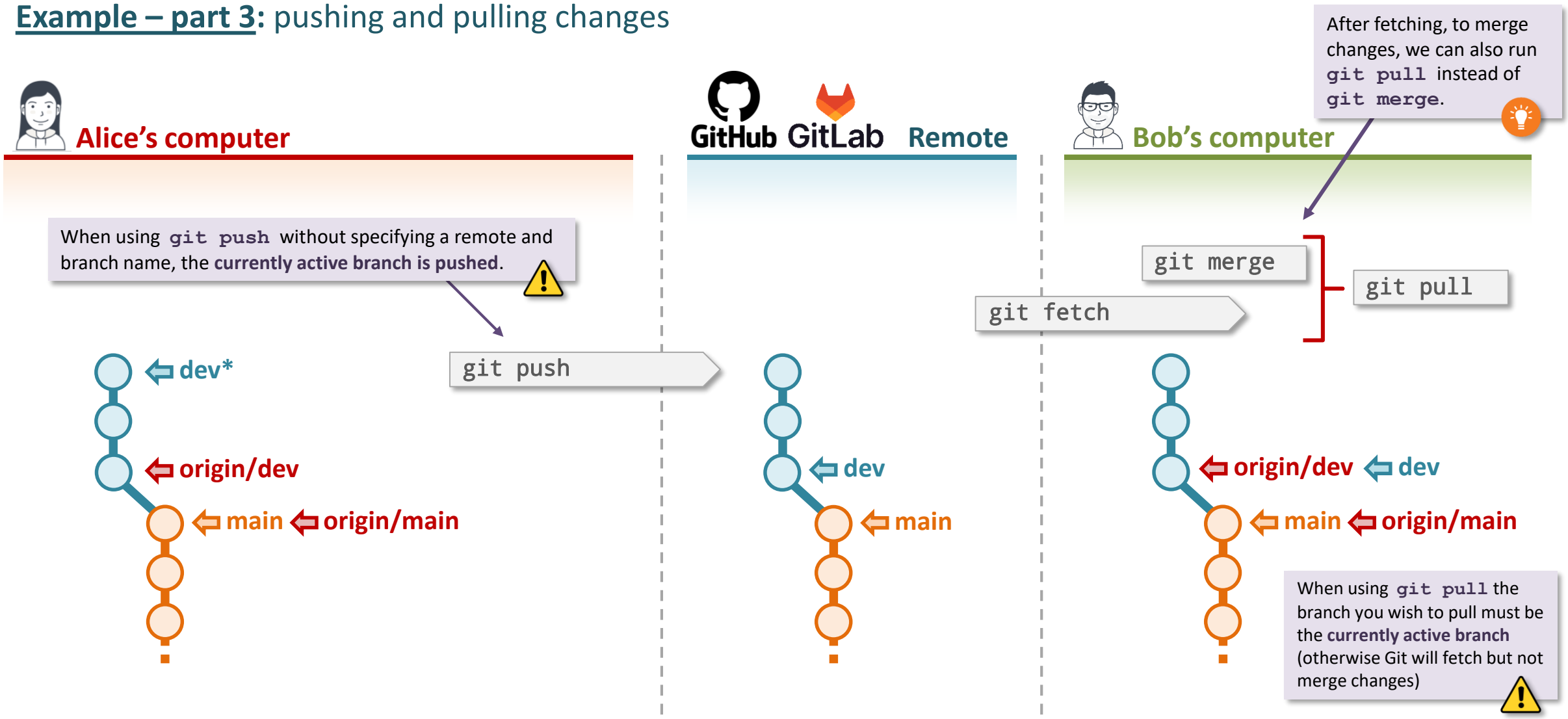
Example – part 2: cloning a remote and checking-out branches



Bob has now joined the team to work with Alice.

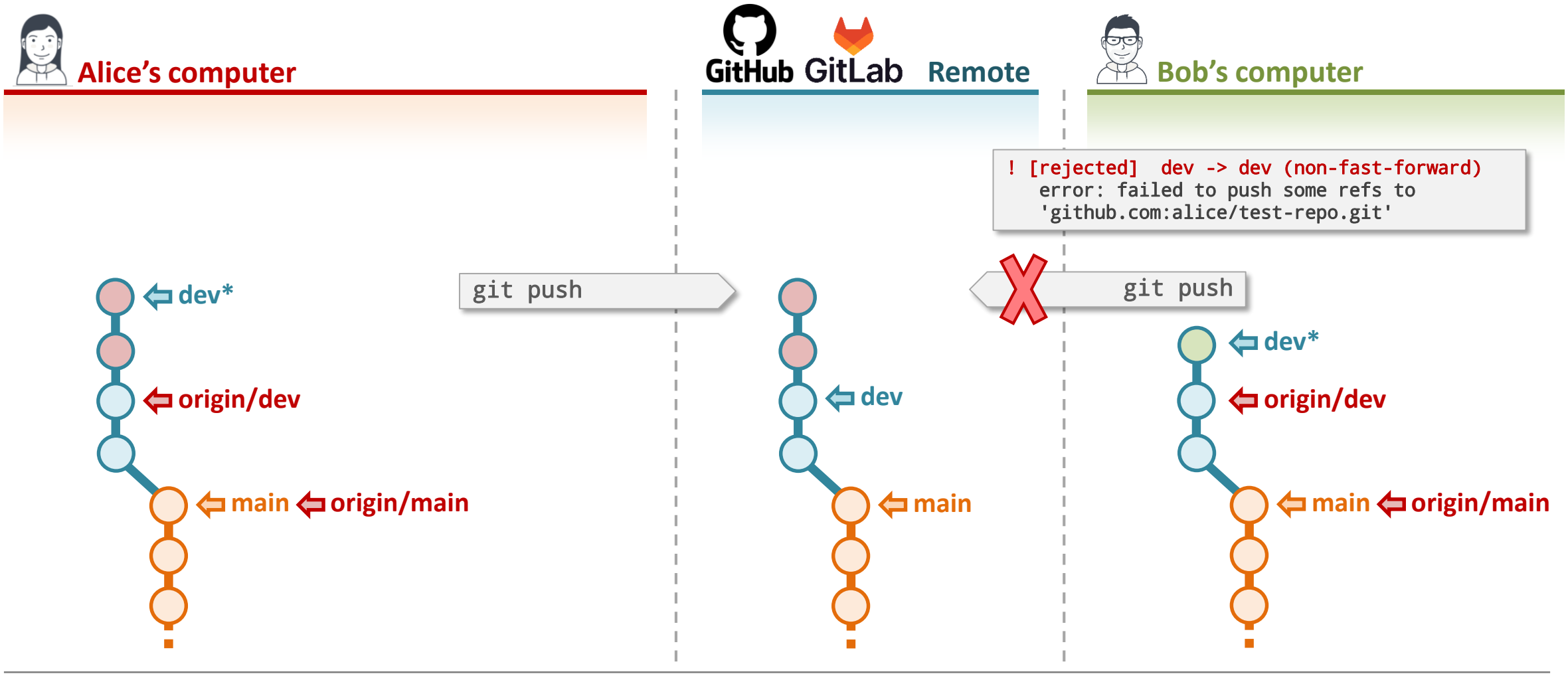
1. He **clones** the repo from GitHub using `git clone <URL of remote>`. At this point, Bob has no local **dev** branch - only a pointer to **origin/dev**.
2. Bob checks-out the **dev** branch to work on it. Because there is already a remote branch **origin/dev** present, Git automatically creates a new local branch **dev** with **origin/dev** as upstream (no need add the `--create/-c` option to `git switch`).

Example – part 3: pushing and pulling changes



1. In the mean time, Alice added 2 new commits to **dev**. She pushes her changes to the remote using `git push` (since her **dev** branch already has an upstream, there is no need to add the `-u/--set-upstream` option this time).
2. To get Alice's updates from the remote, Bob runs `git pull`, which is a combination of `git fetch` + `git merge`.
   Important: `git fetch` downloads all new changes/updates from the remote, but does not update your local branches.

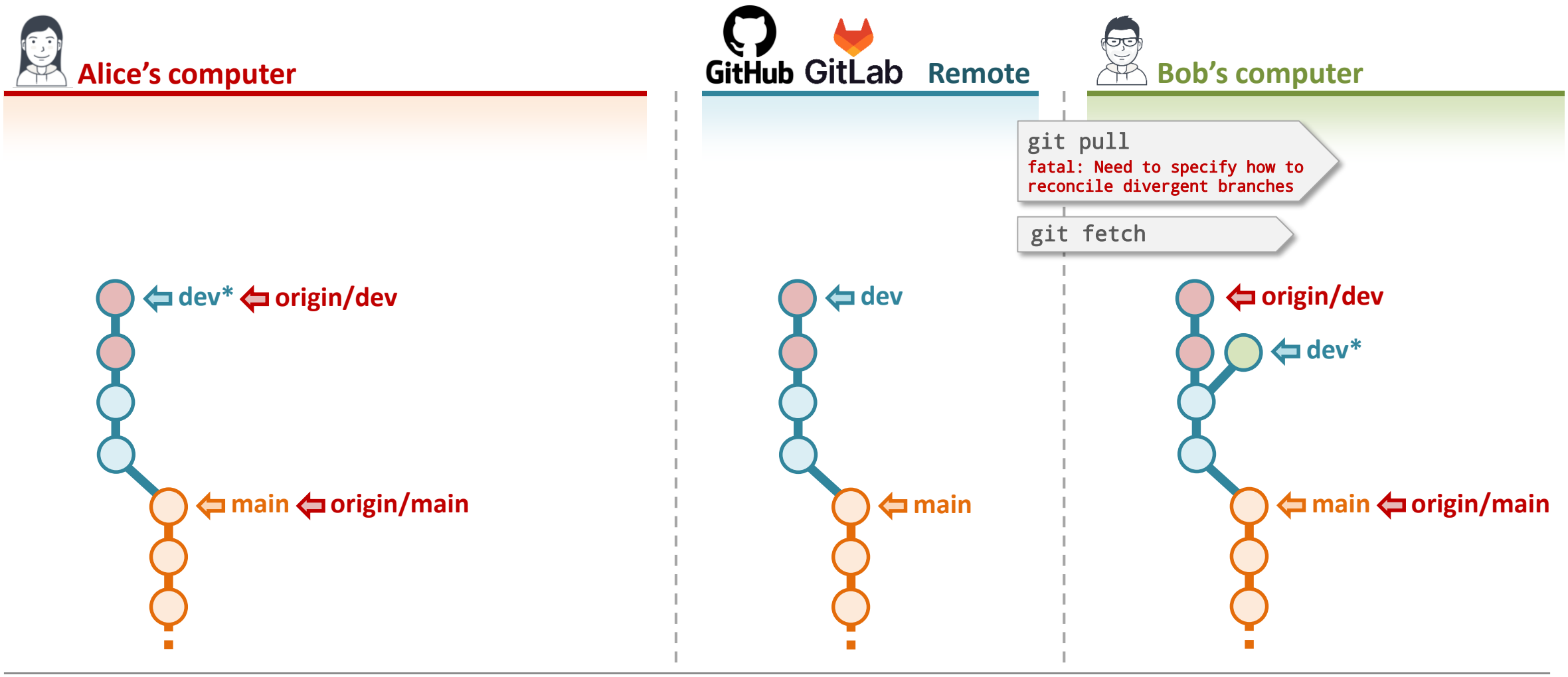
Example – part 4: reconciliation of a diverging history



Both Alice and Bob have now both added some commits to their local **dev** branch. As a result, **the history of their branches has diverged**.

1. Alice pushes her changes to the remote with `git push`, as usual.
2. When Bob tries to `git push`, his changes are **rejected** because the history between his local **dev** branch and the remote have **diverged**!

Example – part 4: reconciliation of a diverging history (continued)



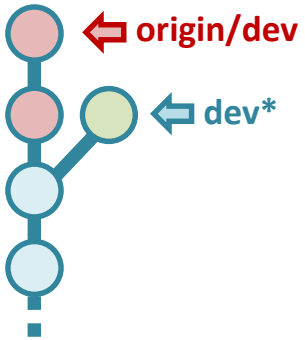
In order to be able to push his changes to the remote, Bob must first reconcile his local **dev** branch with the remote…

1. Bob starts by performing a `git fetch`, just to get the new commits from the remote and see how his local branch diverges from the remote.
   Important: this operation does not impact/update his local **dev** branch.
2. If Bob tried to `git pull`, the operation would fail* because Git wants to know how the branch reconciliation should be carried-out.
   * On recent Git versions (>= 2.33), the default pull behavior is to abort if history diverged. On older versions, the default behavior is to merge (as in **git pull --no-rebase**, see the next slide).

Reconcile local and remote repos

Rebase, merge or overwrite history

To reconcile a diverging branch between a local repo and a remote, 4 options exist.

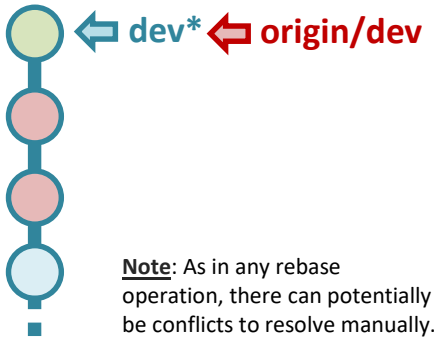


Initial situation: a local branch **dev** has diverged from its upstream branch **origin/dev**

Rebase your local branch

```
> git pull --rebase

# Which is the same as:
> git fetch
> git rebase origin/dev
```



Note: As in any rebase operation, there can potentially be conflicts to resolve manually.

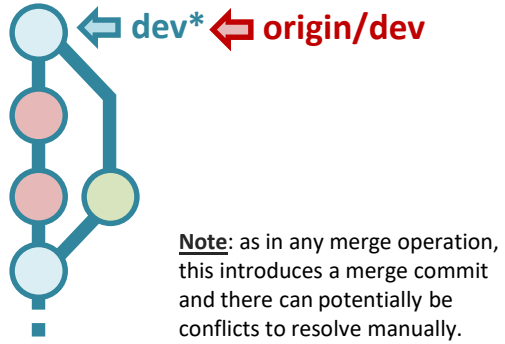
Option 1: **rebase** your local branch on the remote.

- Linear, clean, history.
- No changes are lost.

Merge the remote branch

```
> git pull --no-rebase

# Which is the same as:
> git fetch
> git merge origin/dev
```



Note: as in any merge operation, this introduces a merge commit and there can potentially be conflicts to resolve manually.

Option 2: **merge** the remote into your local branch.

- Perfect history preservation (no commit is re-written).
- No changes are lost.

If you can't remember the `--no-rebase` and `--rebase` options of `git pull`, simply `fetch` and then `merge` or `rebase` on **origin/dev** .

⚠ Overwrite the remote branch

```
git push --force-with-lease
```

```
git push --force
```

`--force-with-lease` overwrites the remote branch only if it is still in the expected state. **Safer** because it prevents from accidentally overwriting someone else's commits.

`--force` overwrites the remote branch regardless of what is currently there.



Option 3: **overwrite** the remote with your local branch.

- Needed when you make a history-rewriting change locally, typically a rebase of a branch.
- **Permanently deletes the data on the remote** – so make sure no-one else has pushed new commits in the meantime..

⚠ Reset (overwrite) the local branch

```
git reset --hard origin/dev
```

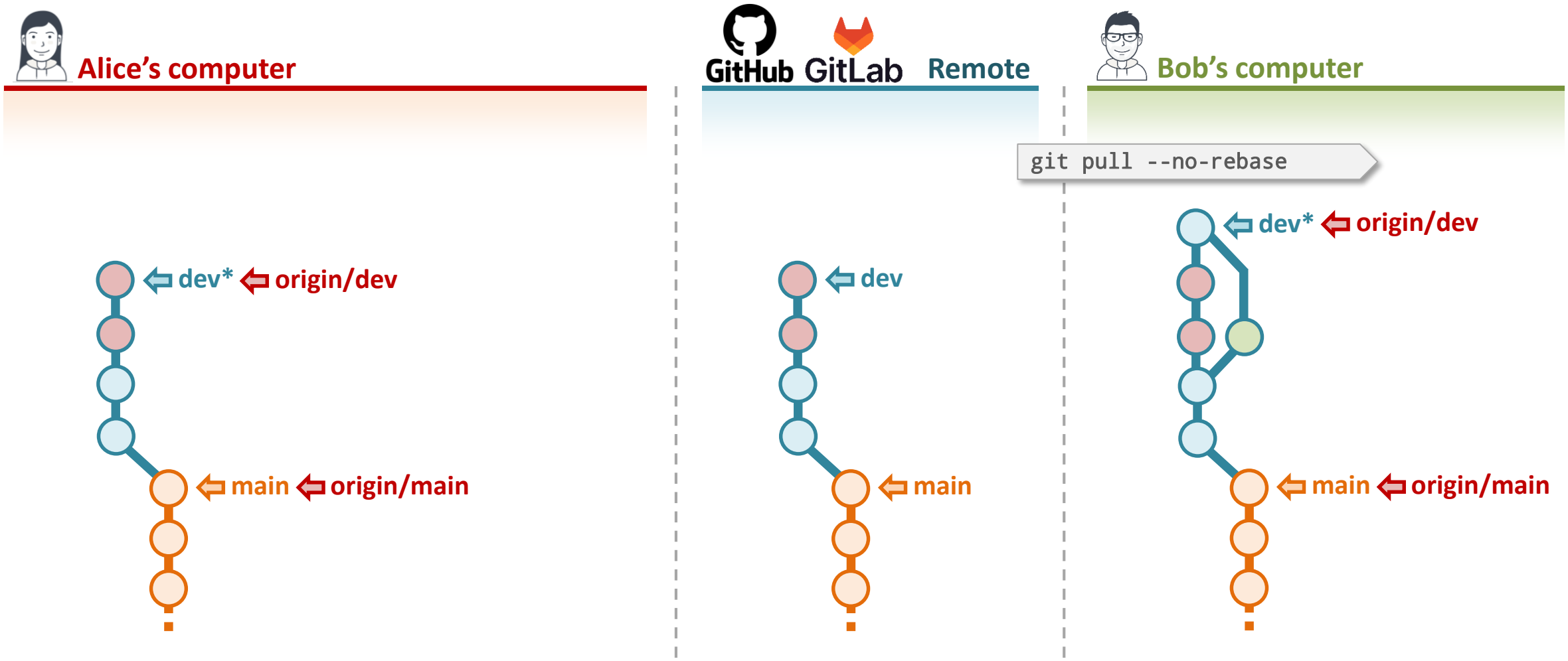
Warning: this will **discard all staged and unstaged changes** to tracked files, resetting them to the HEAD commit. Make sure your working tree is clean before running this command. Untracked files are not affected.



Option 4: **reset (overwrite)** your local branch to the remote.

- Needed when someone else changed history on the remote.
- **Permanently deletes your diverging local commits** !

Example – part 4: reconciliation of a diverging history (continued)



Bob decides to merge without rebase and runs `git pull --no-rebase`.

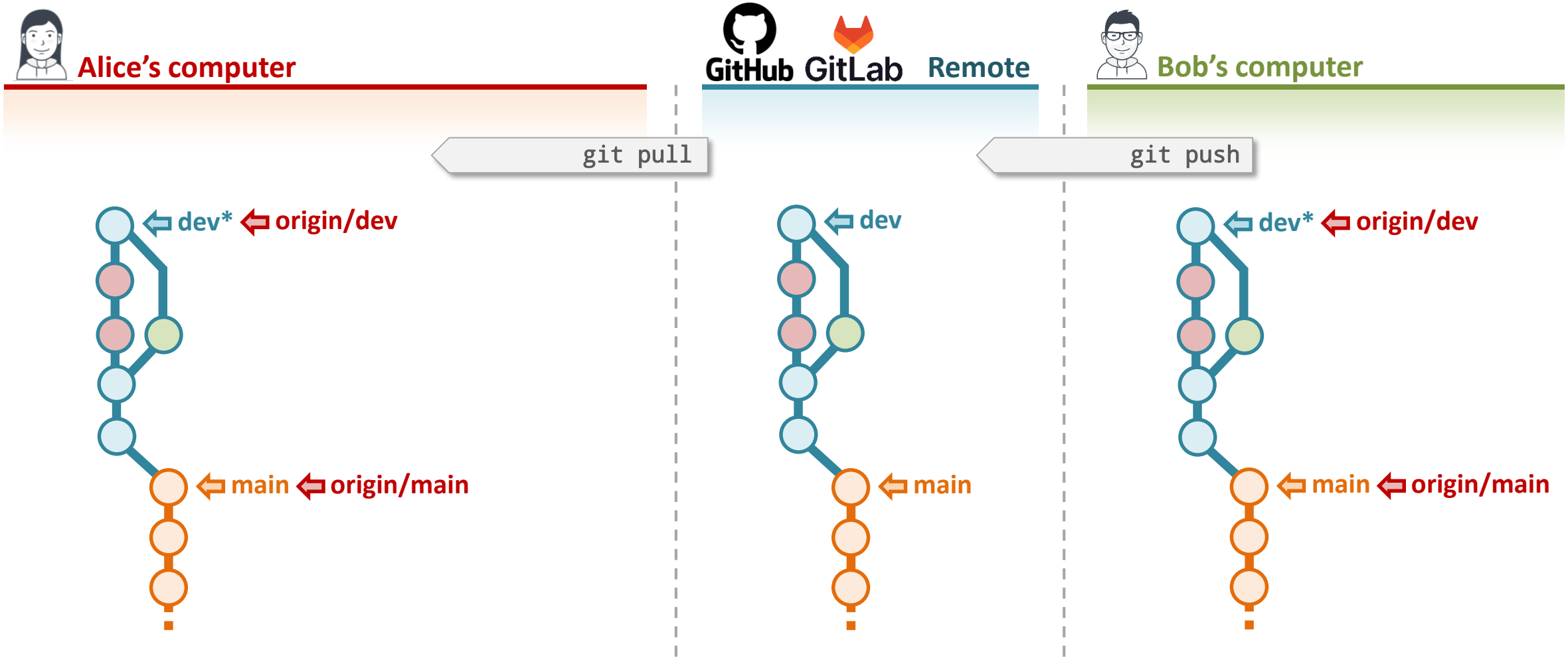
Note: depending on the version of Git, the default behavior of `git pull` is different:

- Newer versions default to `git pull --ff-only` (i.e. raise an error if a fast-forward merge is not possible)
- Older versions default to `git pull --no-rebase` (i.e. the automatically merge)

The default behavior can be modified in the git config.

```
git config pull.rebase false  # merge
git config pull.rebase true   # rebase
git config pull.ff only       # fast-forward only
```

Example – part 4: reconciliation of a diverging history (the end!)

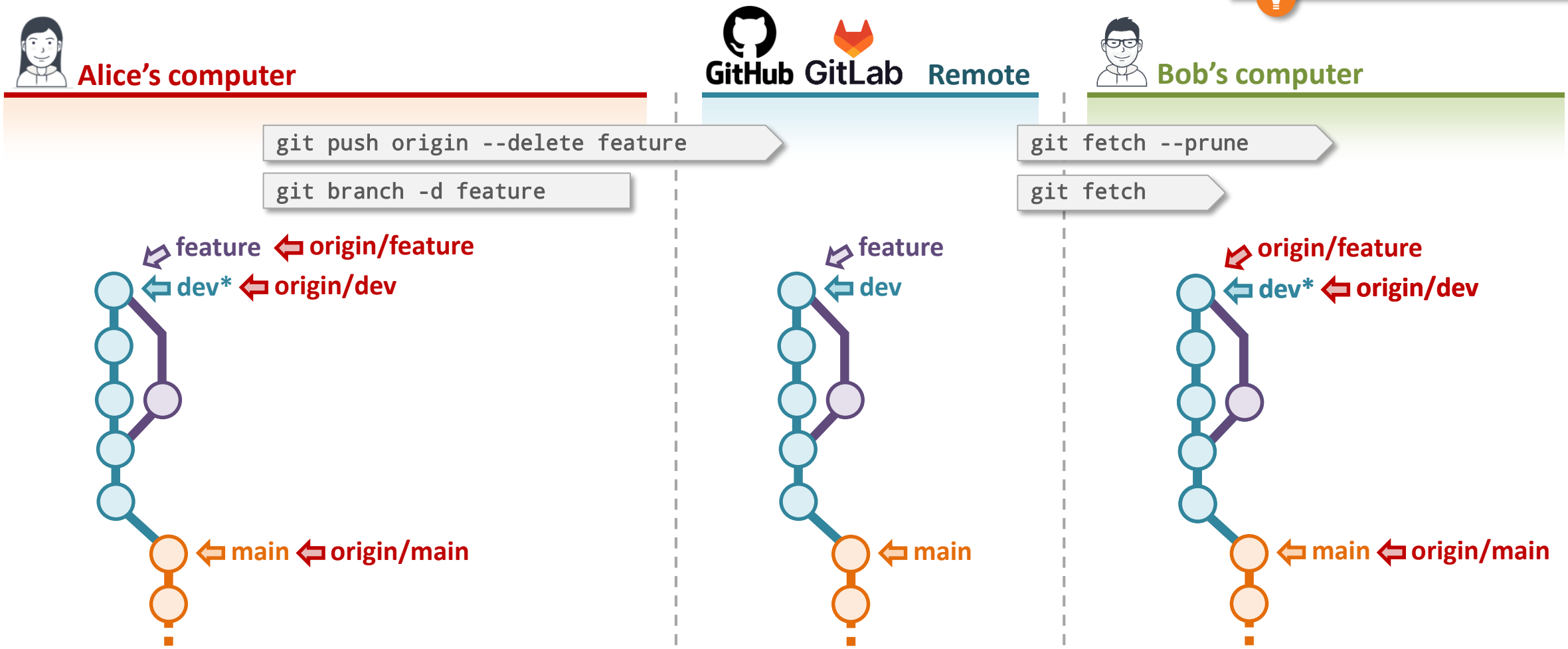


Finally, Bob can `git push` his changes to the remote - there are no more conflicts.

Alice can then `git pull` them.

Example – part 5: deleting branches on the remote

The `--prune` option also works with `git pull --prune`.



We are now at a later point in the development... Alice has just completed a new feature on her branch **feature**, and merged it into **dev**. She now wants to delete the **feature** branch both locally and on the remote.

1. Alice deletes her local branch with `git branch -d <branch name>`.
2. Alice deletes the feature branch on the remote with `git push origin --delete <branch name>`. This also deletes her **origin/feature** pointer.
3. Bob runs `git fetch`, but this does not delete references to remote branches, even if they no longer exist on the remote.
4. To delete his local reference to the remote feature branch (**origin/feature**), Bob has to use `git fetch --prune`.

Interacting with remotes: commands summary

Command	What it does
`git clone <URL>`	**Create a local copy** from an existing online repo. Git automatically adds the online repo as a remote.
`git remote add origin <remote url>`	**Add a new remote** to an existing local repo.
`git remote set-url origin <remote url>`	**Change/update the URL of a remote** associated to a local repo.
`git remote -v`	**Display the remote(s)** associated to a repo.
`git branch -vva`	**List branches of repo and their associated upstream** (if any).

By convention, the **`<remote name>`** is generally set to **`origin`**, but it could be anything.

```
$ git remote -v
origin   https://github.com/alice/test-project.git (fetch)
origin   https://github.com/alice/test-project.git (push)
```

The fetch and push URLs should be the same.
To use different URLs (different remotes) for push and fetch, add two different remotes.

```
$ git branch -vva
  manta-dev 18d8de0 [origin/manta-dev] manta ray: add animal name
  main      6c8d731 [origin/main] Merge pull request #44 from sibgit/dahu-dev
* sunfish   18d8de0 manta ray: add animal name
```

We can see that the branches **main** and **manta-dev** have an upstream branch. The **sunfish** branch does not.

Summary...

Interacting with remotes: commands summary

Command	What it does	Where to run and comments
`git push`	**push new commits** on the current branch to the remote.	**Run on the branch that you wish to push.** (only changes on the active branch are pushed)
`git push -u origin <branch-name>`	Same as git push, but additionally sets the upstream branch to `origin/branch-name`. Only needed if no upstream is set.	`-u` option is only needed when pushing a branch to the remote for the very first time. It is not needed if you initially created the local branch from a remote branch.
`git push origin <branch-name>`	Push new commits on the specified branch to the remote.	When the remote (here `origin`) and branch names are specified, the push command **can be run from anywhere**.
`git push --force-with-lease`	**Overwrite the branch on the remote** with the local version.	**Warning:** this **deletes data** on the remote!
`git fetch`	**Download all updates** from the remote to your local repo (even for non-active branches or branches for which there is no local version). Does not update your local branch pointer to `origin/branch-name`.	**Can be run from any branch.**
`git pull`	**Download all updates** and **merge changes** the upstream `origin/branch-name` into the active branch (i.e. update the active branch to its version on the remote).	**Run on the branch that you wish to update.** `git pull` is a shortcut for `git fetch + git merge origin/branch-name`
`git pull --no-rebase`	Fetch + 3-way merge active branch with its upstream `origin/branch-name`.	
`git pull --rebase`	Fetch + rebase active branch on its upstream `origin/branch-name`.	
`git pull --ff-only`	Fetch + fast-forward merge active branch with its upstream `origin/branch-name`. If a fast-forward merge is not possible, an error is generated.	

On recent versions of Git (>= 2.33), the default pull behavior is to abort the pull if a branch and its upstream are diverging.
On older versions, the default behavior is to merge them (same as `git pull --no-rebase`).



Summary...

`git pull`: a shortcut for fetch + merge

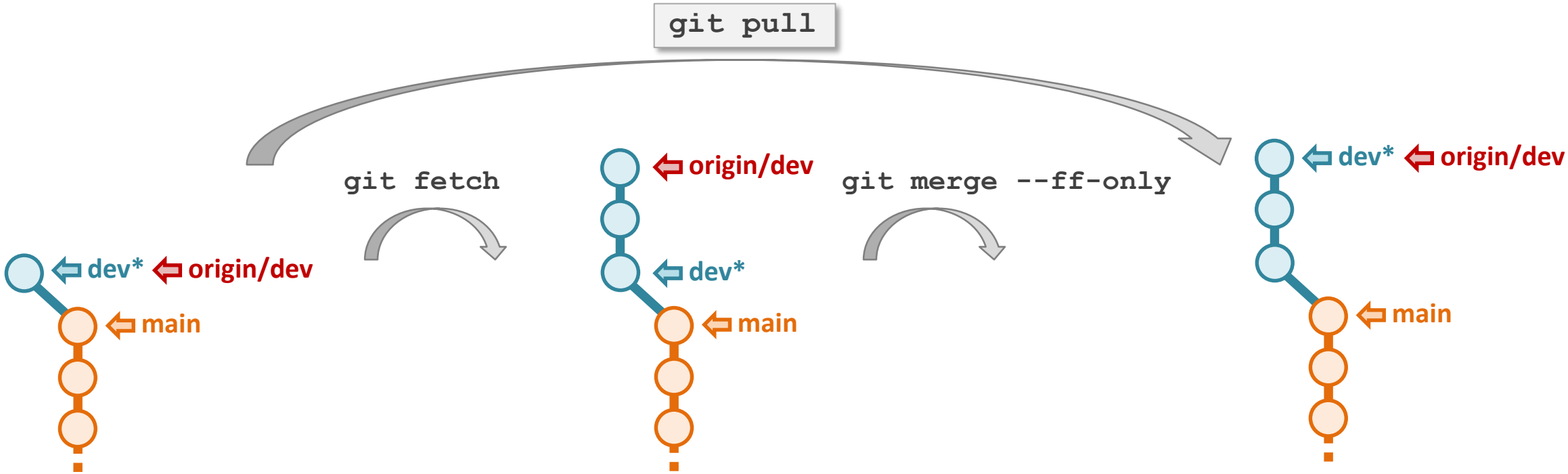
Having the **`git pull`** command use **`--ff-only`** as default merge option is a recent behavior (Git >= 2.33). In older versions, to force **`git pull`** to only allow fast-forward merges, the following option must be set:

```
git config --global pull.ff only
```

The `git pull` command is a shortcut for:

1. `git fetch` : fetches all updates from the remote.
2. `git merge –ff-only` : merge the currently active branch with its upstream branch (`origin/<branch>`).

Fast-forward only -> any divergence in history will cause the command to fail and report an error.



By default, git merges a branch with its upstream branch, so `git merge` is the same as `git merge origin/<branch>` .

Supplementary material...

GitHub / GitLab

collaborate and share your work





GitHub / **GitLab** – an online home for Git repositories

- **GitHub** [github.com] and **GitLab** [gitlab.com] are hosting platforms for Git repositories.
- Very popular to share/distribute software.
- Allows to host public (anybody can access) and private (restricted access) repos.
- Hosting of projects is free, with some paid features.
- Popular alternatives include:
  - **A local instance of GitLab**, the same as GitLab.com but hosted by someone else.
  - **BitBucket** [bitbucket.org].

Project home page on GitHub

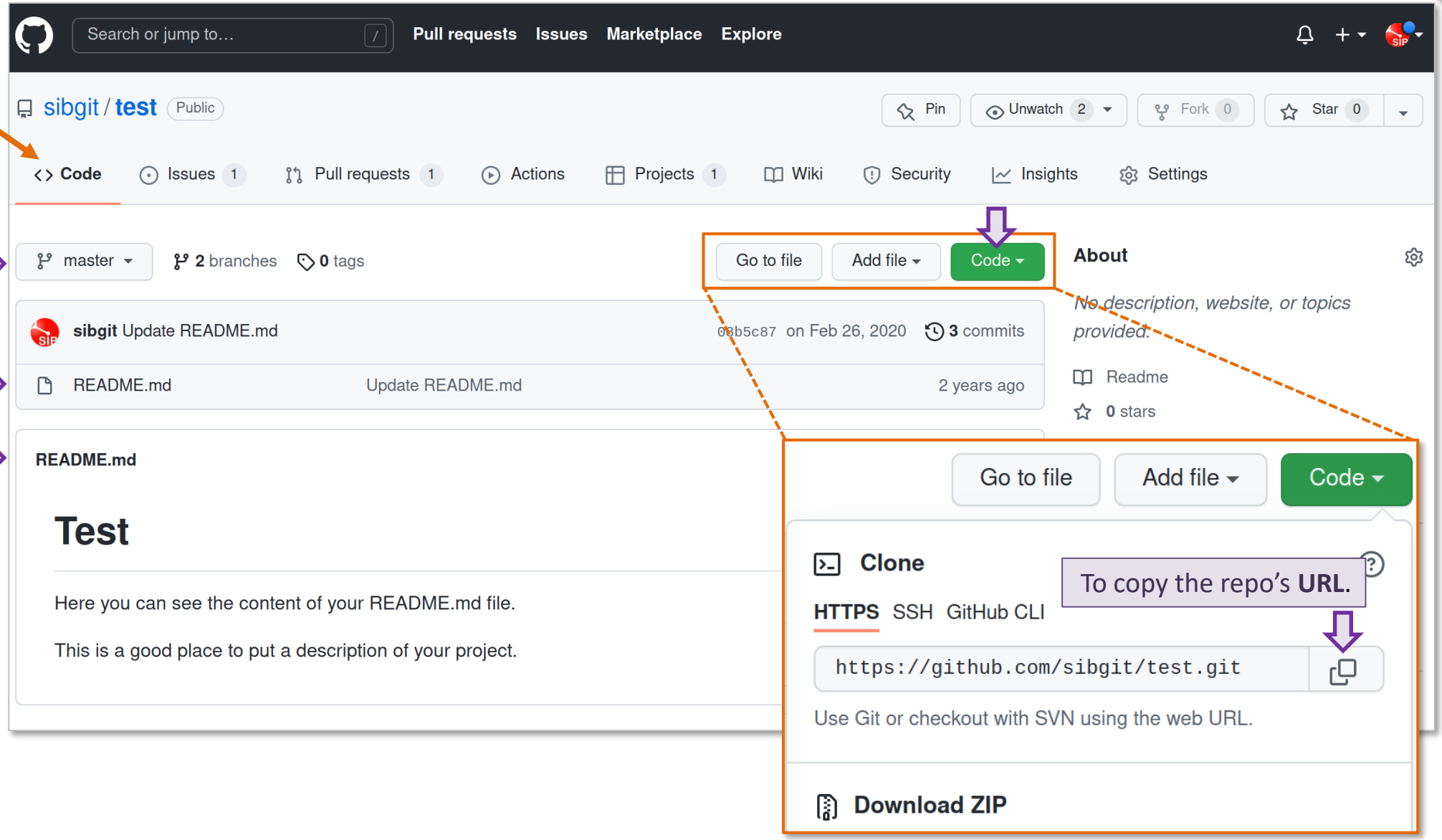
Example of the "**home page**" of a repository on GitHub

Code tab: the "home" page of your repo.

Branch you are currently viewing

List of files present in the repo.

If you have a **README.md** file, it is displayed here (with markdown rendering).



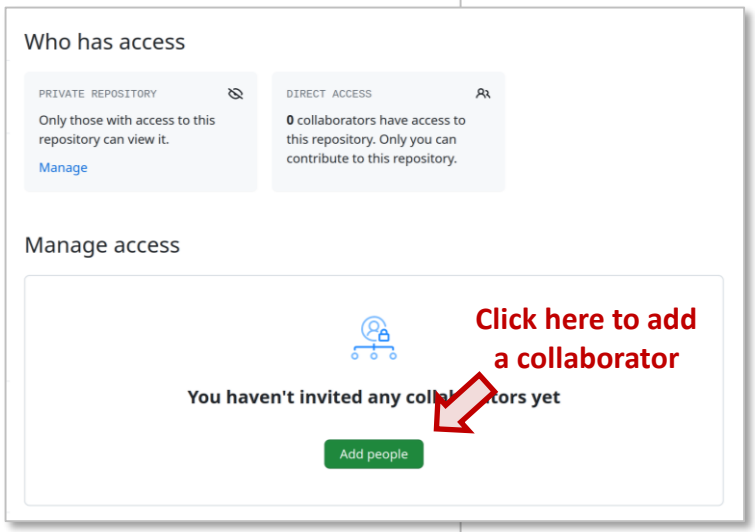
GitHub slide ...

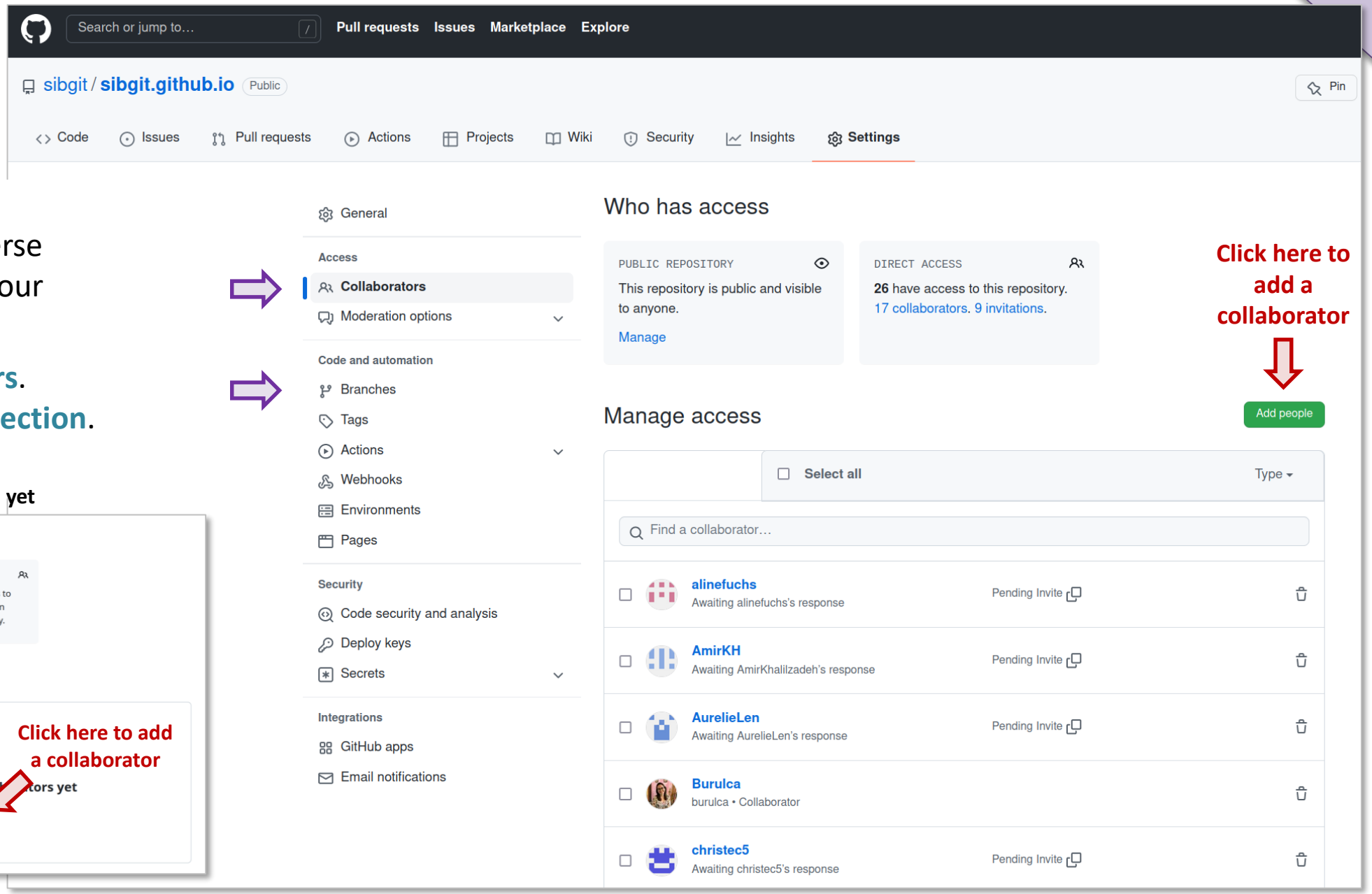
Repository settings (only available to project owner)

Here you can set diverse settings concerning your repository, e.g. :

- Invite **collaborators**.
- Setup **branch protection**.

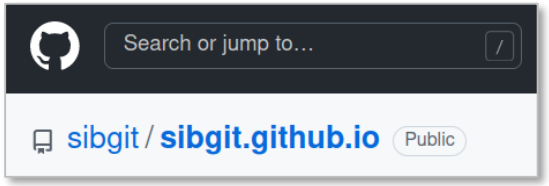
View with no collaborator added yet





GitHub slide …

Other GitHub features (some of them)



"Home" of your repo (repo content)

Issue tracker

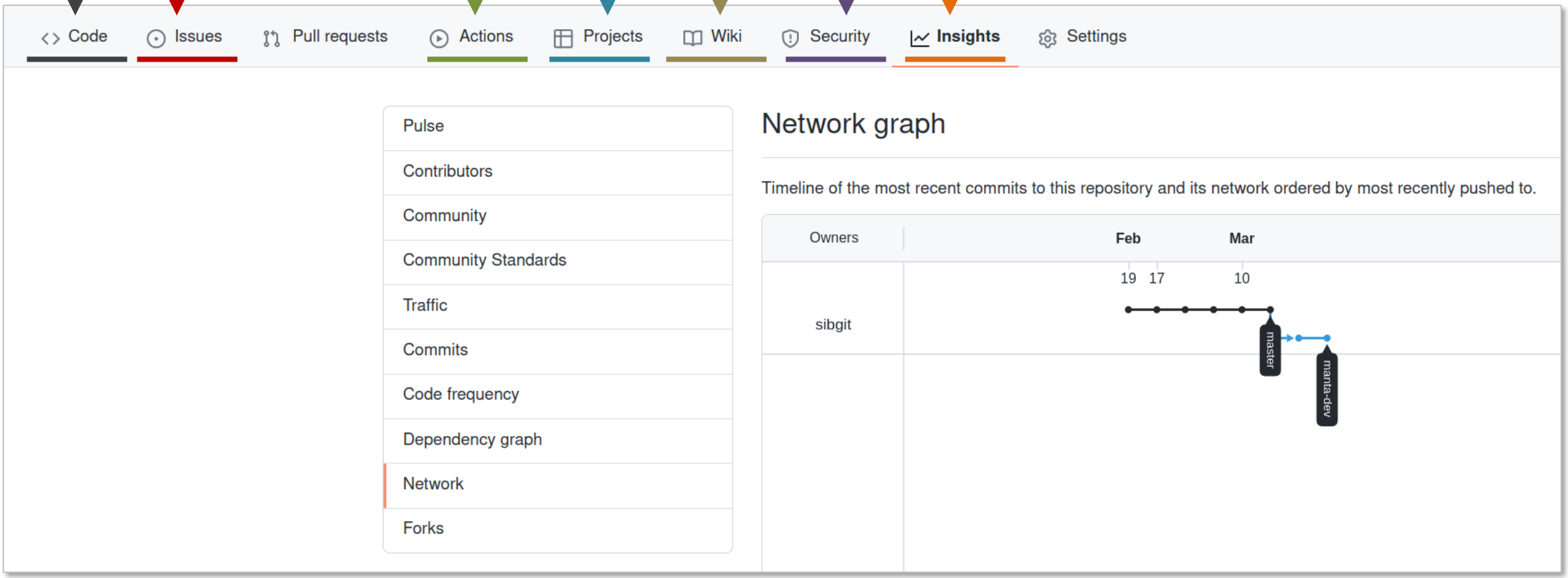
Continuous integration (automated testing)

Group issues and PR by topics.

Add a wiki for your project.

Setup automated security scanning for your code (vulnerability check).

Statistics about your repo's activity.



GitHub slide …

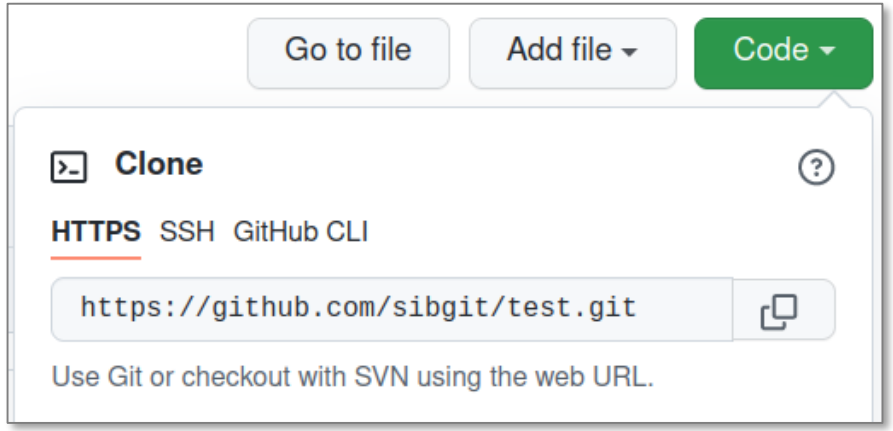
Cloning a repo: HTTPS vs. SSH

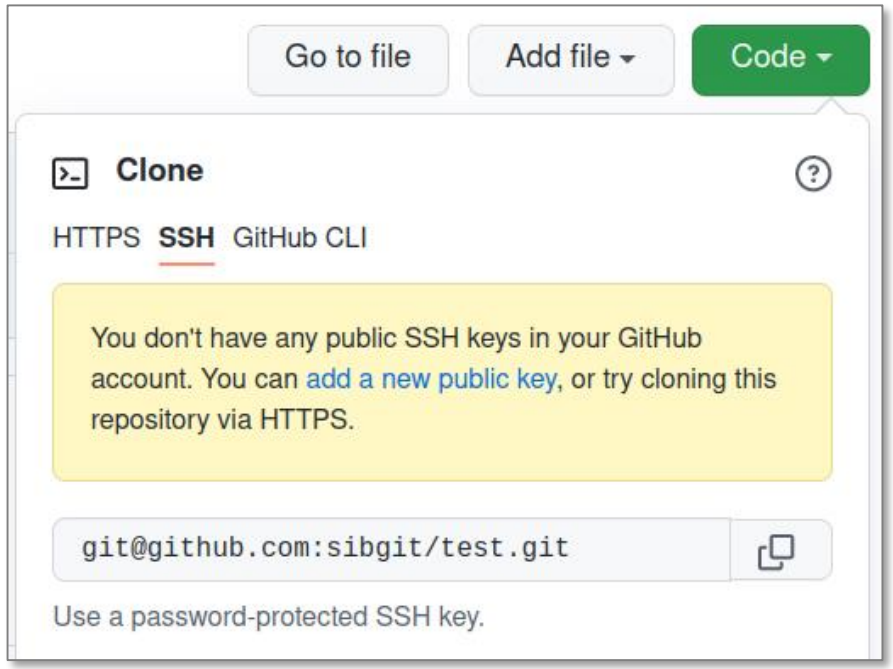
HTTPS and SSH are two different network protocols that machines can use to communicate.

When cloning (or adding a remote) via:

- **HTTPS**, you will need to provide a **personal access token** (PAT) as authentication credential.
  - If the repo is public, credentials are only needed to push data to the remote (not to pull).
  - Your local Git repo will in principle store the login credentials, so you need to provide them only once.

- **SSH**, you will need to add your **public SSH key** to your GitHub account.





Reminder: command to clone a repo (here via https)

```
$ git clone https://github.com/sibgit/test.git
```

Supplementary material...

Personal Access Tokens (PAT)

on GitHub or GitLab

Personal access tokens (PAT) on GitHub/GitLab

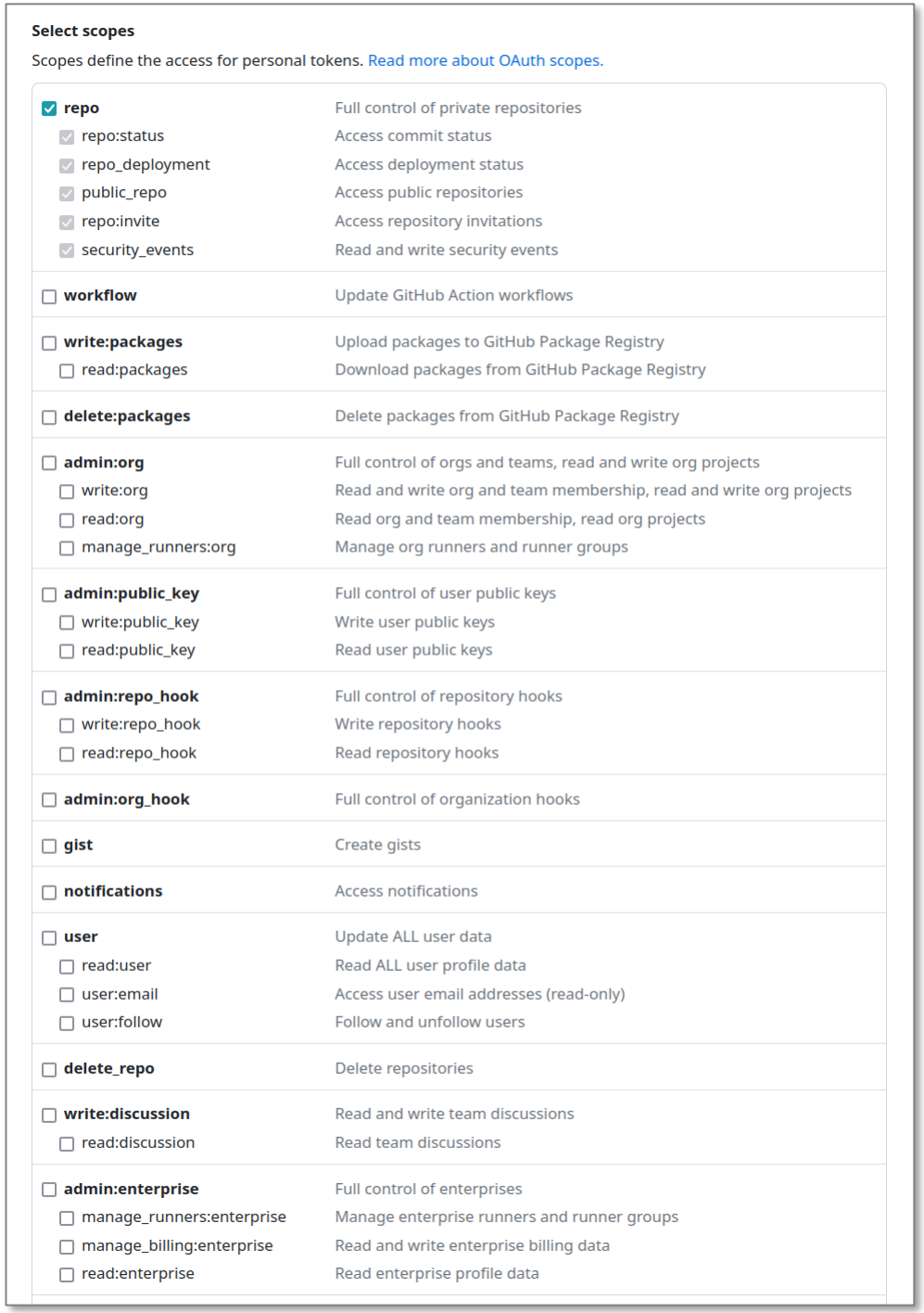
Pushing data to a remote requires **some form of authentication**…
… otherwise anyone could push anything to your remotes!

For security reasons, GitHub does not allow using your user name and password for authentication when running a git push command. Instead you need to use a **personal access token (PAT)**.

In **exercises 4 and 5** you will need a PAT to push commits to GitHub/GitLab *

Let's generate a PAT together now…

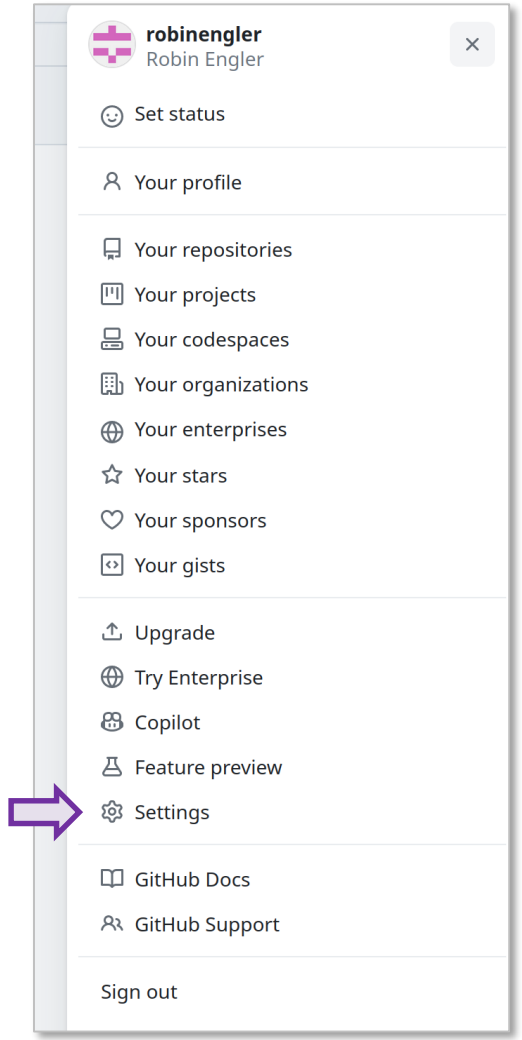
* Alternatively, you can also authenticate to GitHub/GitLab using SSH keys. If your account is already setup to use SSH keys, then you don't need a PAT.



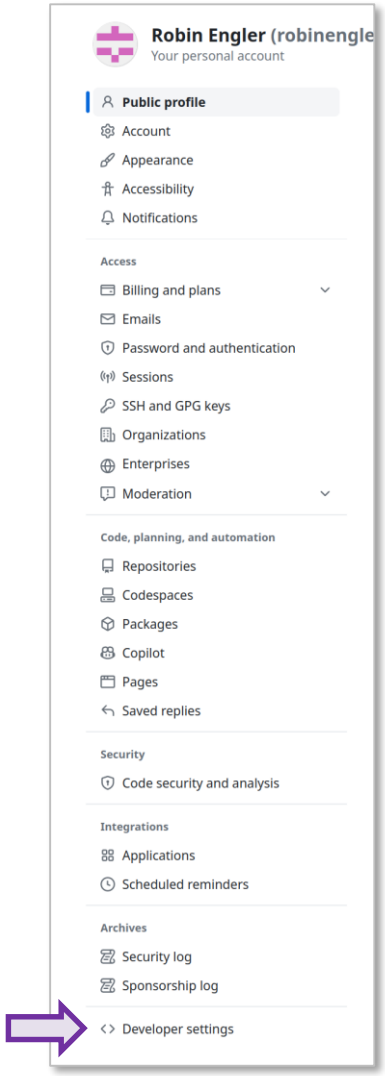
Generating a “personal access token” (PAT) on **GitHub**

In order to push data (commits) to GitHub, you will need a **personal access token (PAT)**.

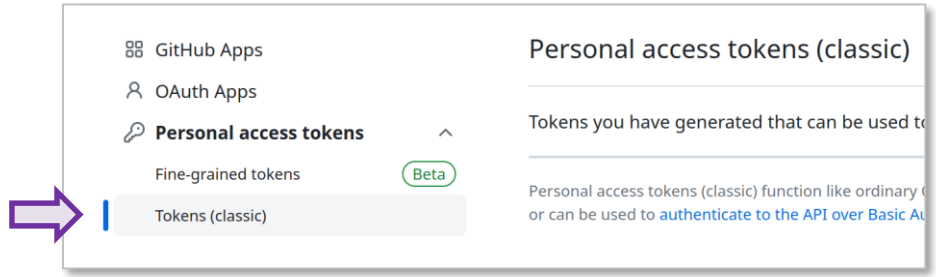
1. In your user profile (top right), click on **Settings**.



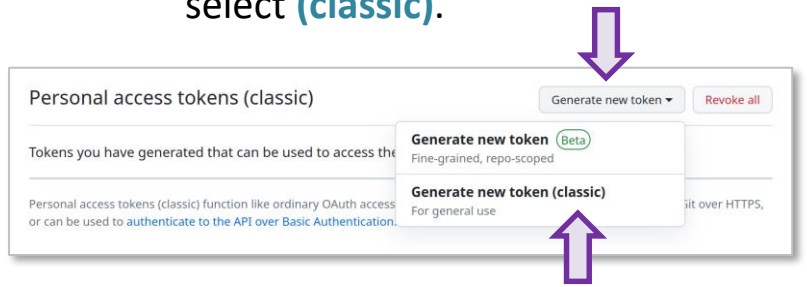
2. In your Account settings, click on **Developer settings** (at the very bottom of the list)



3. In **Developer settings**, click on **Personal access tokens**, and select **Tokens (classic)**.



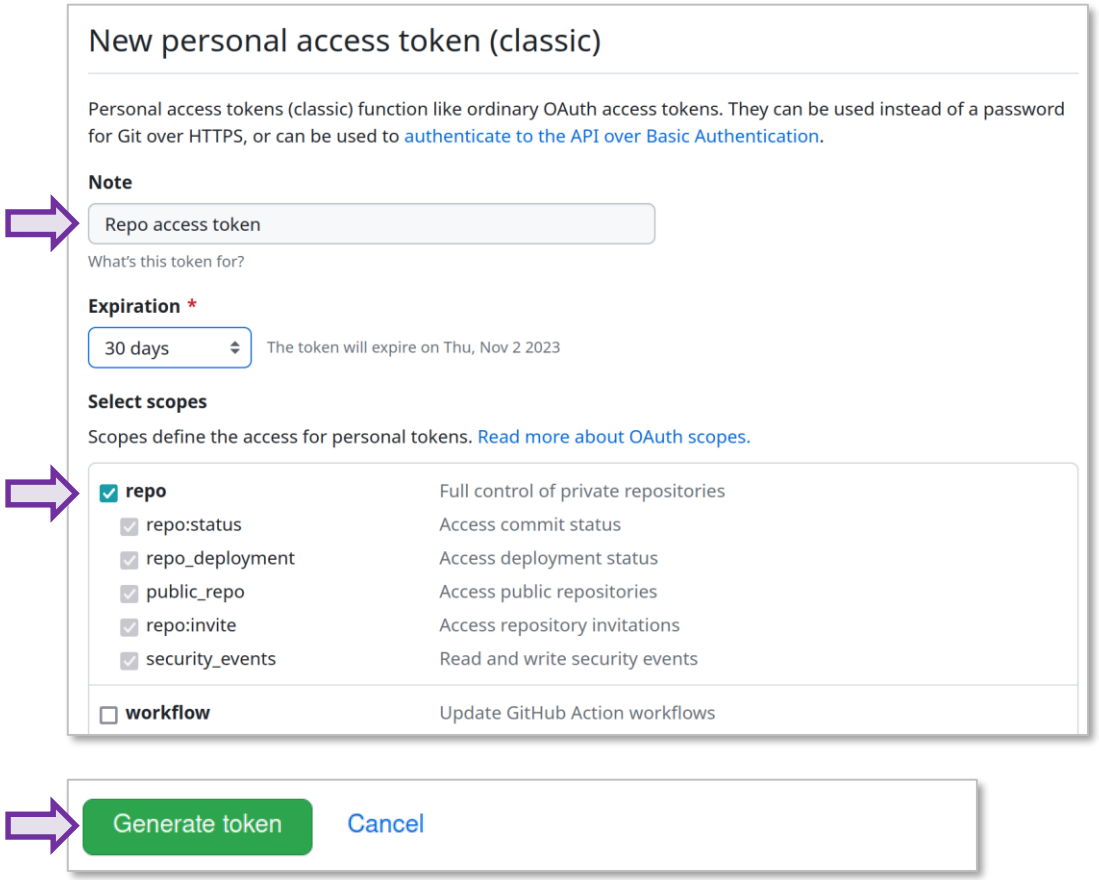
4. Click on **Generate new token**, and select **(classic)**.



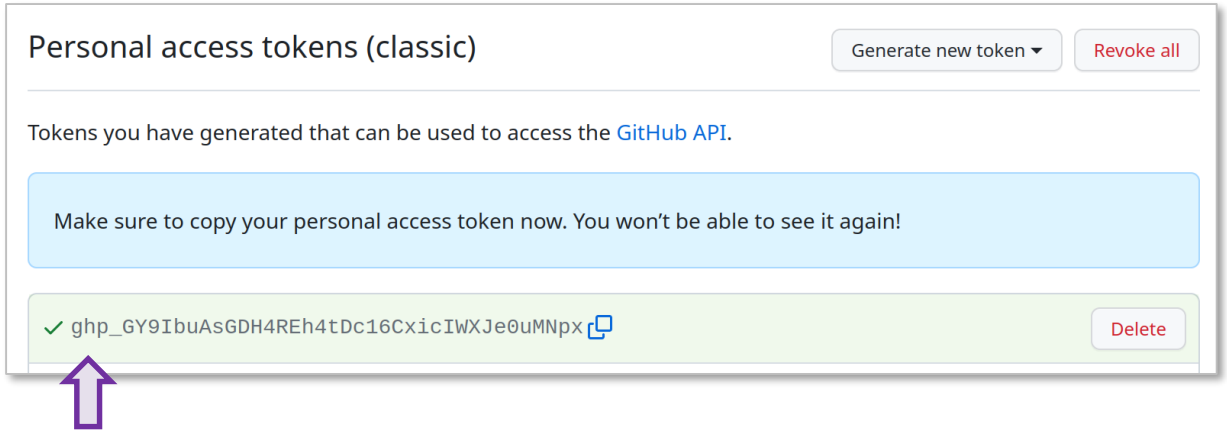
Go to next page

GitHub slide ...

5. Add a **Note** (description) to your token and select the **repo** scope checkbox. The click **Generate token**.



6. **Copy the personal access token** to a safe locations (ideally in a password manager). You will **not be able to access it again** later.



7. When you will push content to GitHub for the first time in the project, you will be asked for your user name and password. **Instead of the password**, enter the **personal access token** you just created.

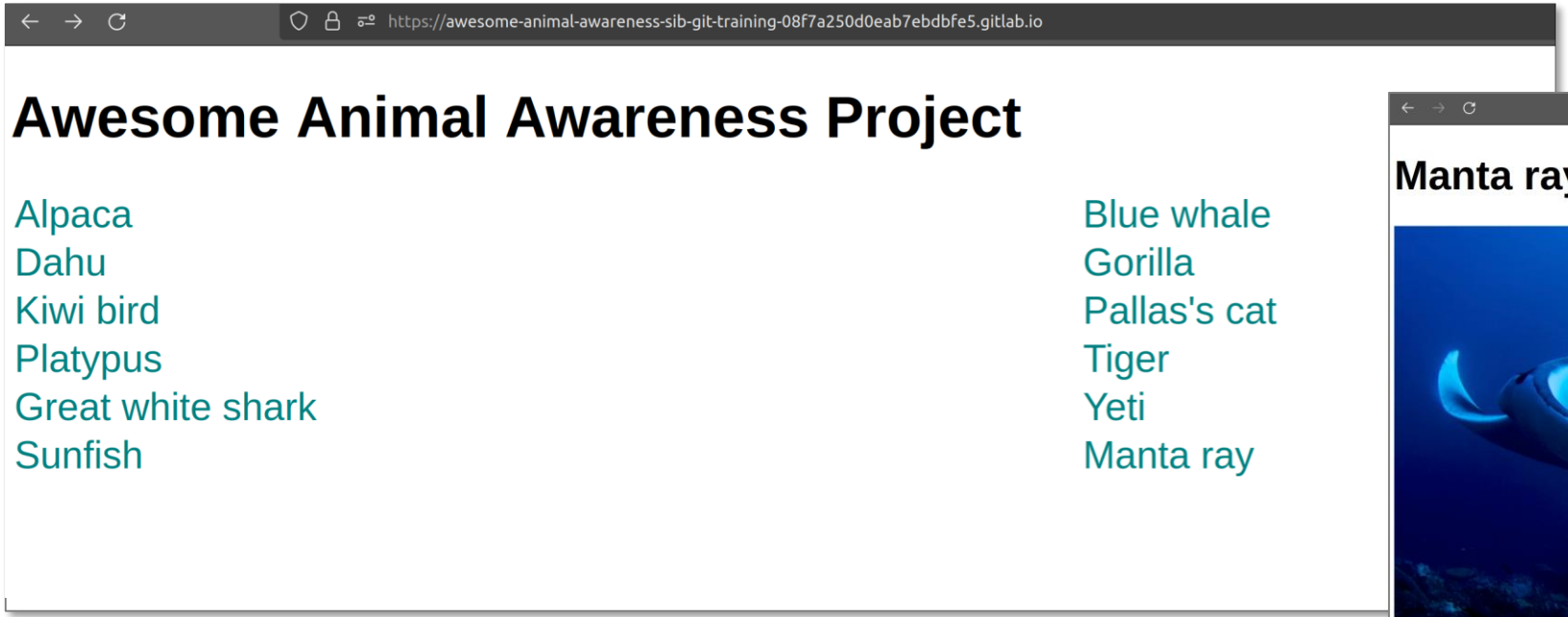
exercise 4

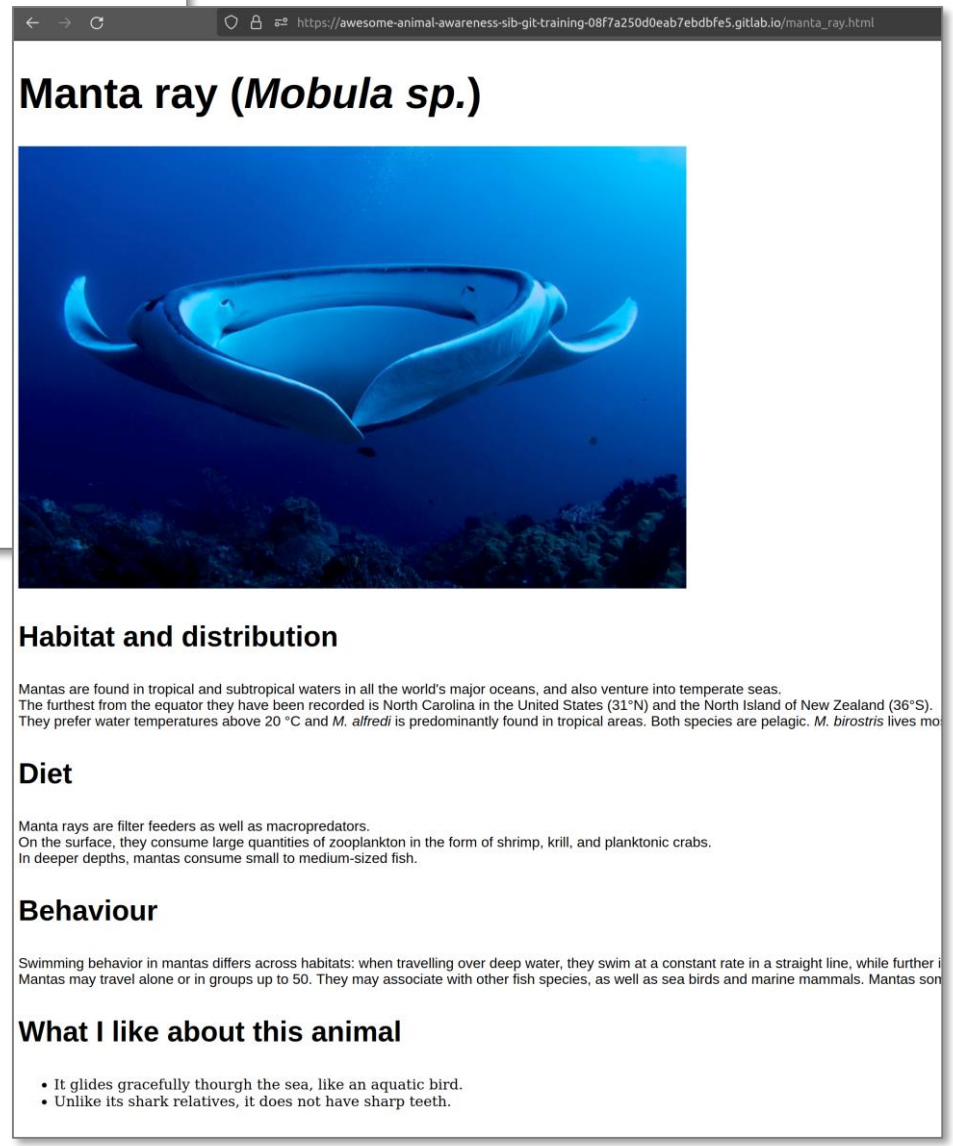
the markdown cheat-sheet: an introduction to working with remotes

Pull Requests (GitHub) and **Merge Requests** (GitLab)

An introduction to the upcoming **exercise 4**…

In exercise 4, we will all work together on building a website for the **Awesome Animal Awareness project**!





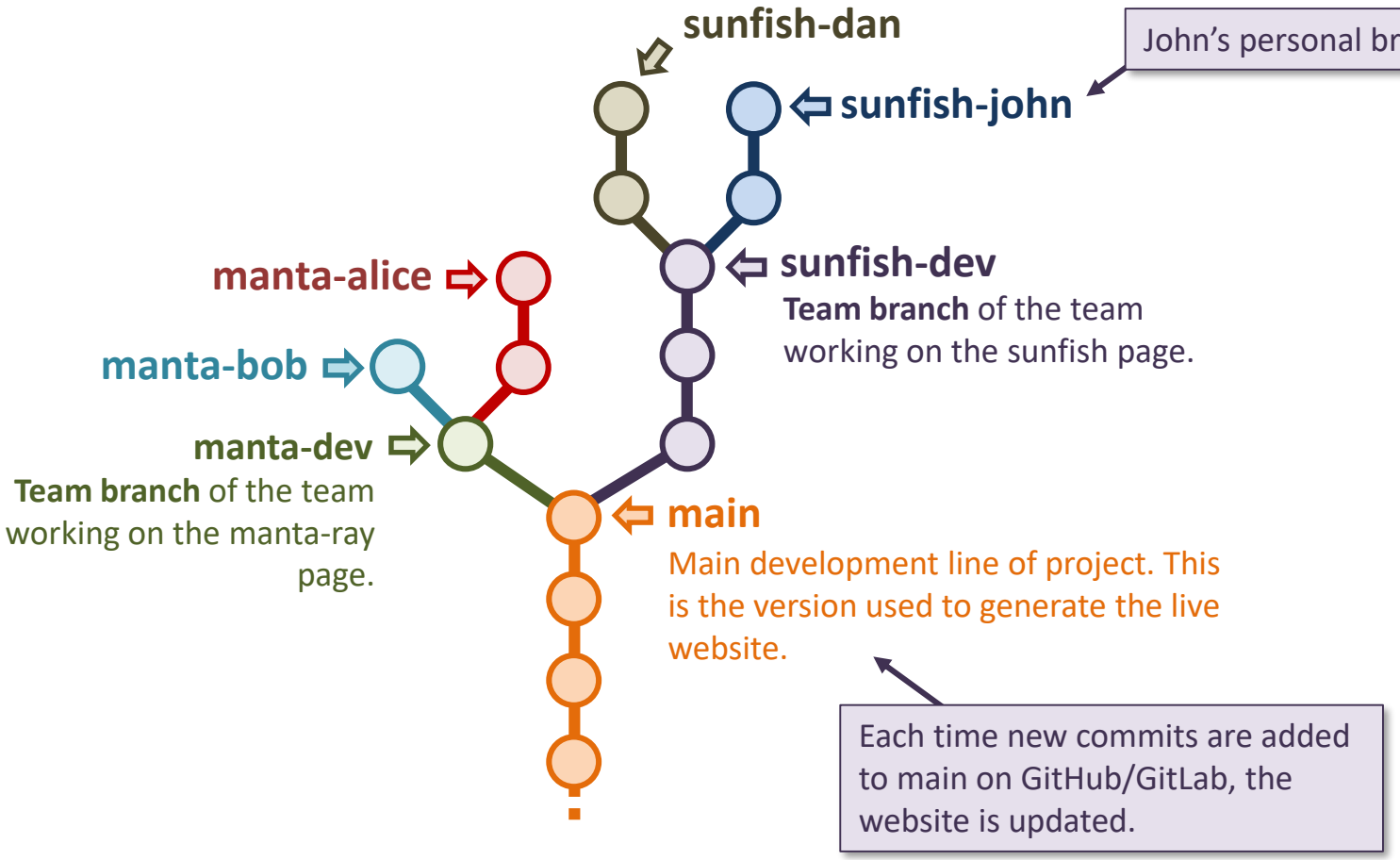
How we will work:

- We will split into teams of 2-3 people.
- Each team will be responsible for creating the page of an (awesome!) animal *.
- Within a team, each person will work on a different part of the animal's page (e.g. one person works on the "Habitat and distribution" section, while another works on the "Diet" or "Behavior").

* Note: every animal in the list is awesome – you can't go wrong!

An introduction to the upcoming **exercise 4**…

- This is how (more or less) our shared repository will look on GitHub/GitLab…
- Changes made to the **main** branch are directly reflected in the production website – so we don't want to mess-up **main** !!
- => You are **not allowed to push directly to main**.



How are we going to contribute changes from our team branches ?

Pull Requests (GitHub) / Merge Requests (GitLab)

Pull Requests (PR) and **Merge Requests (MR)** are a way to perform a merge operation on the remote (on GitHub/GitLab) instead of in your local copy of the repository.

PR/MR are the same thing, they just have different names on GitHub/GitLab.

Why use a PR/MR instead of a local merge (and push) ?

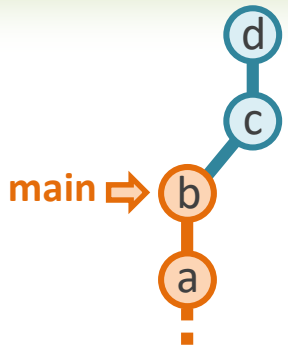
- The branch you want to merge into (e.g. main) is **protected** *.
- Gives the opportunity to the repository owner(s) to **review changes** before merging them.
- Makes it easy to merge changes from a **forked** ** repository.

* **Protected** branches are branches where push operations are limited to users with enough privileges.

** A **fork** is a copy of an entire repository under a new ownership.



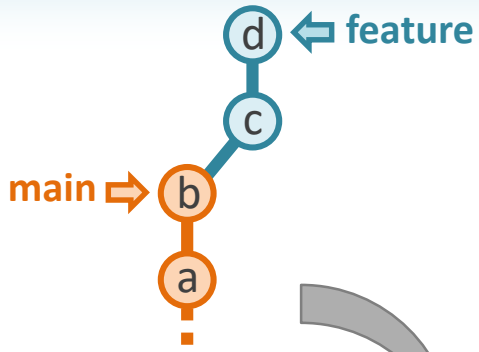
Bob's computer



`git push`

Bob has completed his work on a new feature. He **pushes his changes to the remote**.

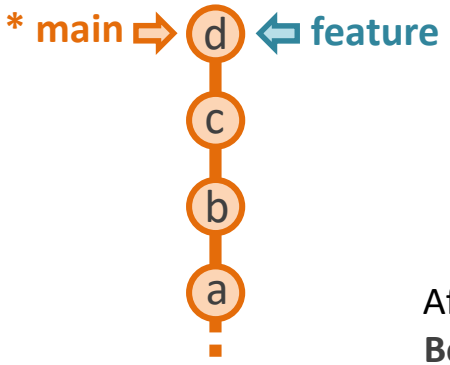
GitHub GitLab **Remote**



The PR/MR workflow:

- Bob opens a PR/MR on GitHub/GitLab.
- Alice reviews the changes made by Bob on branch **feature**.
- Alice approves the PR/MR.
- Bob (or Alice) merges the PR/MR.
- On the remote, the **feature** branch is now merged into **main**. Optionally, **feature** is then deleted.

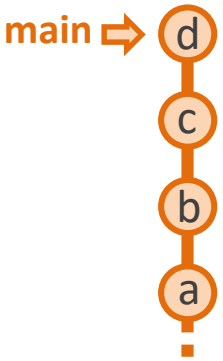




`git fetch`

`git switch main`
`git pull`

After the PR/MR is merged, **Bob updates his local repo**.



After the PR/MR is merged, you can pull the changes from the remote to update your local repo (at this point the merge is only on the remote).

```
❯ git log --all --decorate --oneline --graph
* 9afd6e4 (HEAD -> manta-dev, origin/manta-dev) manta-ray: add behavior information
* ba11531 manta-ray: add distribution and image
* 7b0516b manta-ray: add animal name and diet
* 65cb84f (origin/main, origin/HEAD, main) cicd: add .gitlab-ci.yml file
* 90c5dbe web: rename home page to Awesome Animal Awareness Project
* 4401bd7 web: add animal page templates
* 11a7390 styles: change paragraphs fonts
* 30b07cf styles: add styles.css file
* f1828a1 doc: add README.md
* 998ea08 first commit
```

Using **`git fetch`** is optional, it's useful if you want to preview the position of **`origin/main`** before merging it into your local **`main`** with **`git pull`**.

`git fetch --prune`

`--prune` deletes local references to remote branches (**`origin/manta-dev`** has been deleted).

```
❯ git log --all --decorate --oneline --graph
*   13c625f (origin/main, origin/HEAD) Merge branch 'manta-dev' into 'main'
|\
| * 9afd6e4 (HEAD -> manta-dev) manta-ray: add behavior information
| * ba11531 manta-ray: add distribution and image
| * 7b0516b manta-ray: add animal name and diet
|/
* 65cb84f (main) cicd: add .gitlab-ci.yml file
* 90c5dbe web: rename home page to Awesome Animal Awareness Project
* 4401bd7 web: add animal page templates
* 11a7390 styles: change paragraphs fonts
```

```
git switch main
git pull --prune
git branch -d manta-dev
```

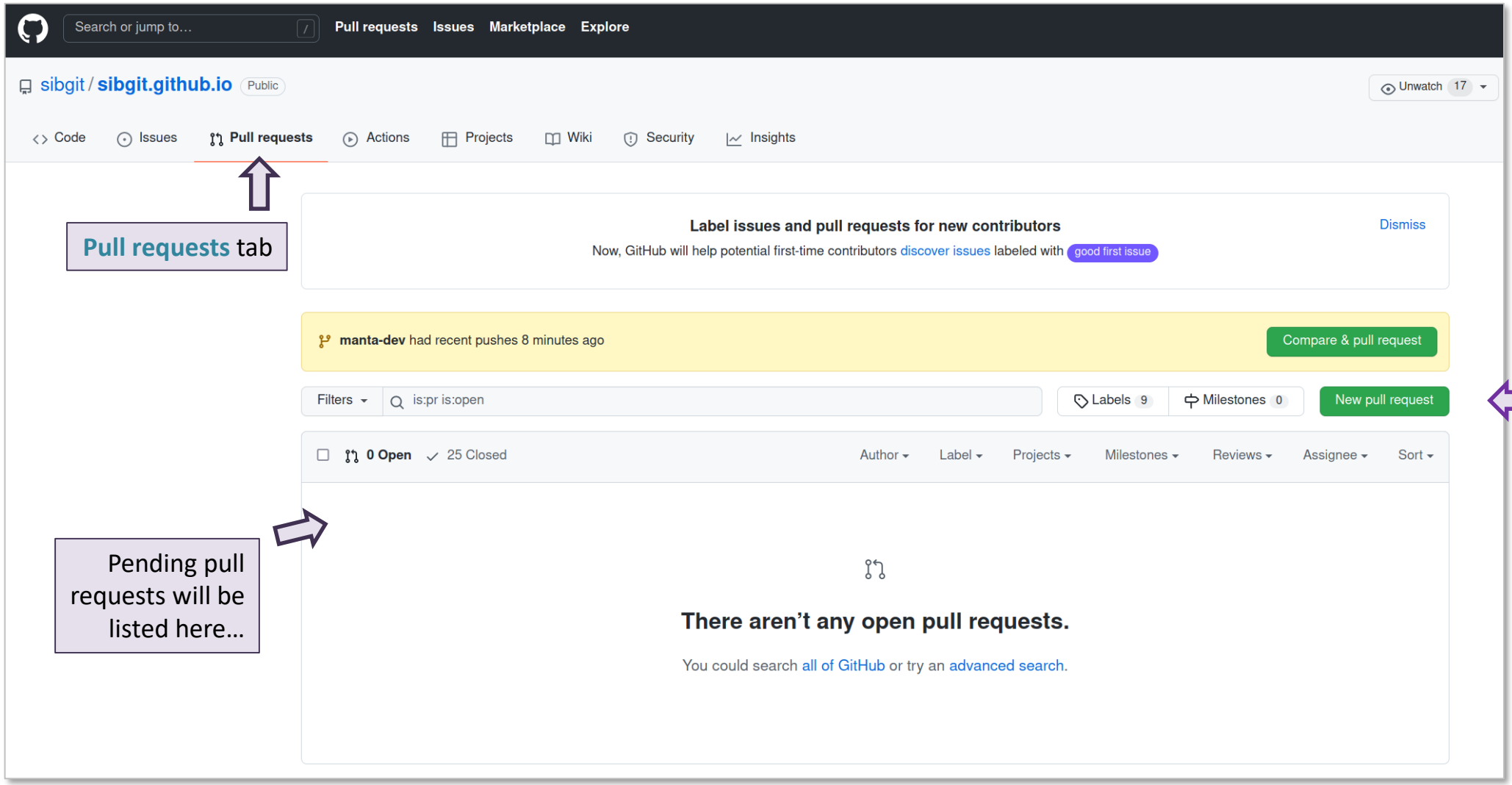
```
❯ git log --all --decorate --oneline --graph
*   13c625f (HEAD -> main, origin/main, origin/HEAD) Merge branch 'manta-dev'
|\
| * 9afd6e4 manta-ray: add behavior information
| * ba11531 manta-ray: add distribution and image
| * 7b0516b manta-ray: add animal name and diet
|/
* 65cb84f cicd: add .gitlab-ci.yml file
* 90c5dbe web: rename home page to Awesome Animal Awareness Project
* 4401bd7 web: add animal page templates
```

```
git switch main
git pull
git branch -d manta-dev
```

Open a Pull Request on GitHub: step-by-step

You will need to do this in exercise 4 !

1. On the project's page on GitHub, go to the **Pull requests** tab.



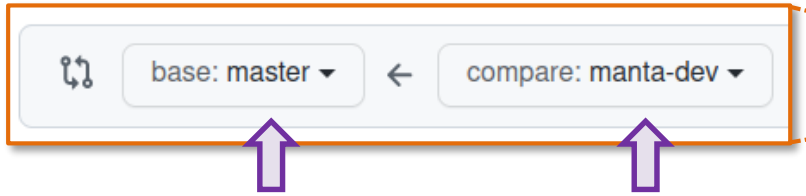
Pull requests tab

Pending pull requests will be listed here…

2. Click on **New pull request**.

GitHub slide …

3. Select the branches to merge:



Branch to merge into

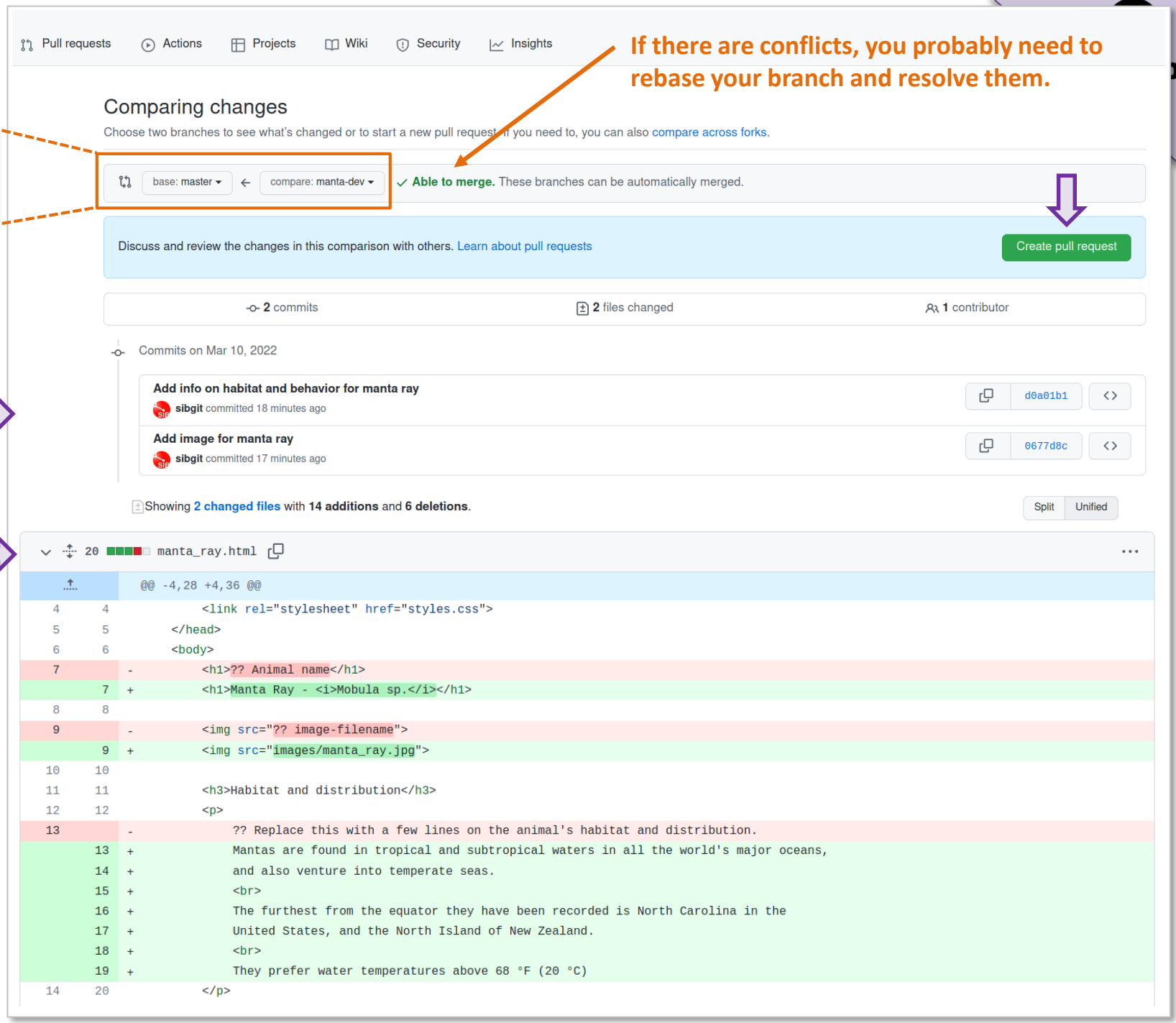
Branch to merge (your contribution)

List of commits that will be merged

In this example, there are 2 commits on branch “manta-dev” that will be merged into “master”.

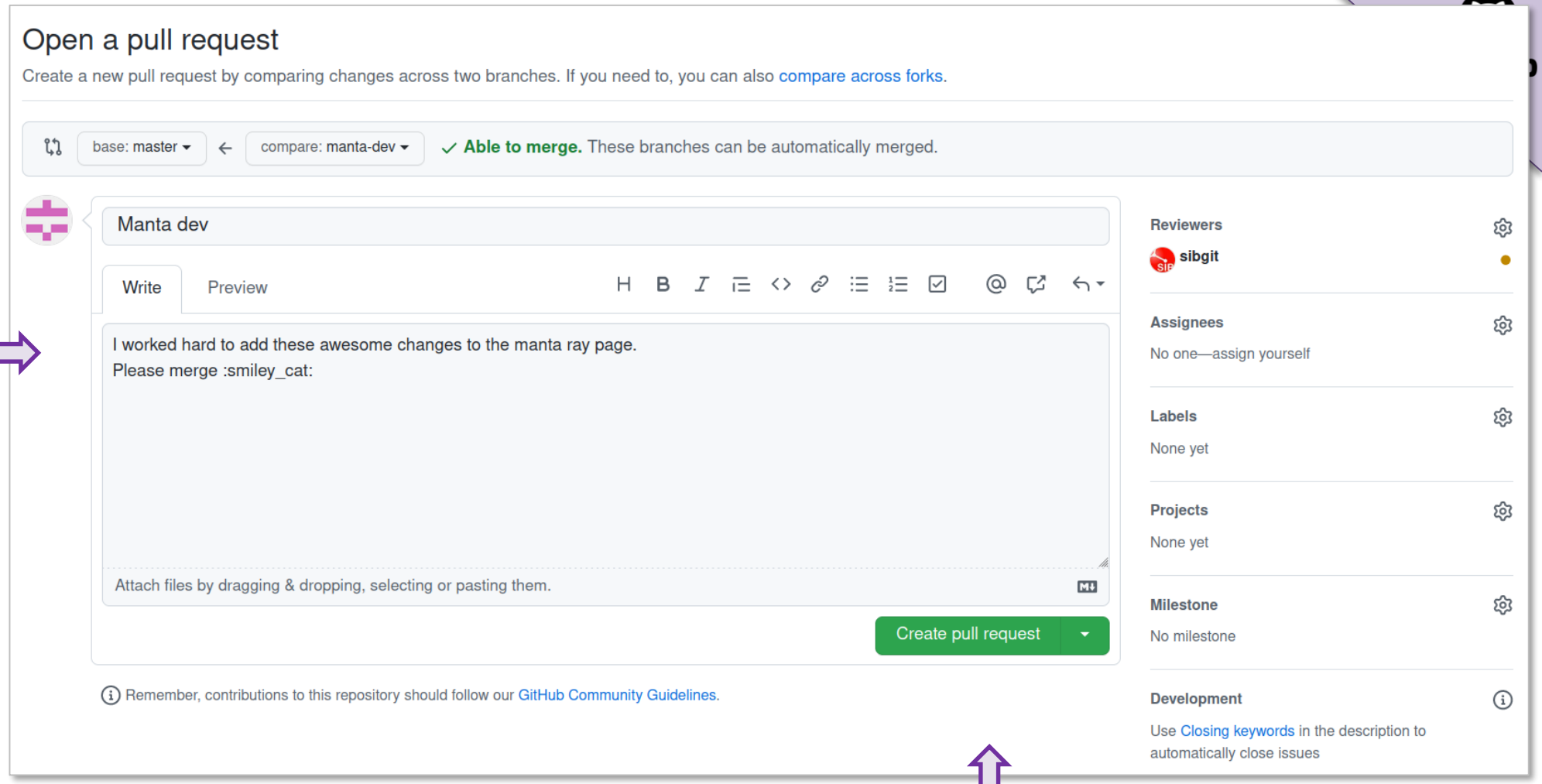
Summary of changes introduced by the pull request.

Green lines = new content.
Red lines = deleted content.



4. Click on Create pull request.

5. Optionally, enter a **message** for the people that will review your pull request.



6. Submit your pull request by clicking **Create pull request**.



The pull request is now **created**, and **awaiting approval** from an authorized person.
(e.g. the repo owner or a colleague)

Merging is **blocked**, because someone has to approve your PR.

Pull requests 1 | Actions | Projects | Wiki | Security | Insights

Manta dev #27

Open **robinengler** wants to merge 2 commits into `master` from `manta-dev`

Conversation 0 | Commits 2 | Checks 0 | Files changed 2

robinengler commented now — Collaborator

I worked hard to add these awesome changes to the manta ray page.
Please merge 😸

sibgit added 2 commits 31 minutes ago

`Add info on habitat and behavior for manta ray` `d0a01b1`

`Add image for manta ray` `0677d8c`

Add more commits by pushing to the **`manta-dev`** branch on **sibgit/sibgit.github.io**.

This branch has not been deployed
No deployments

Review required
At least 1 approving review is required by reviewers with write access. Learn more.

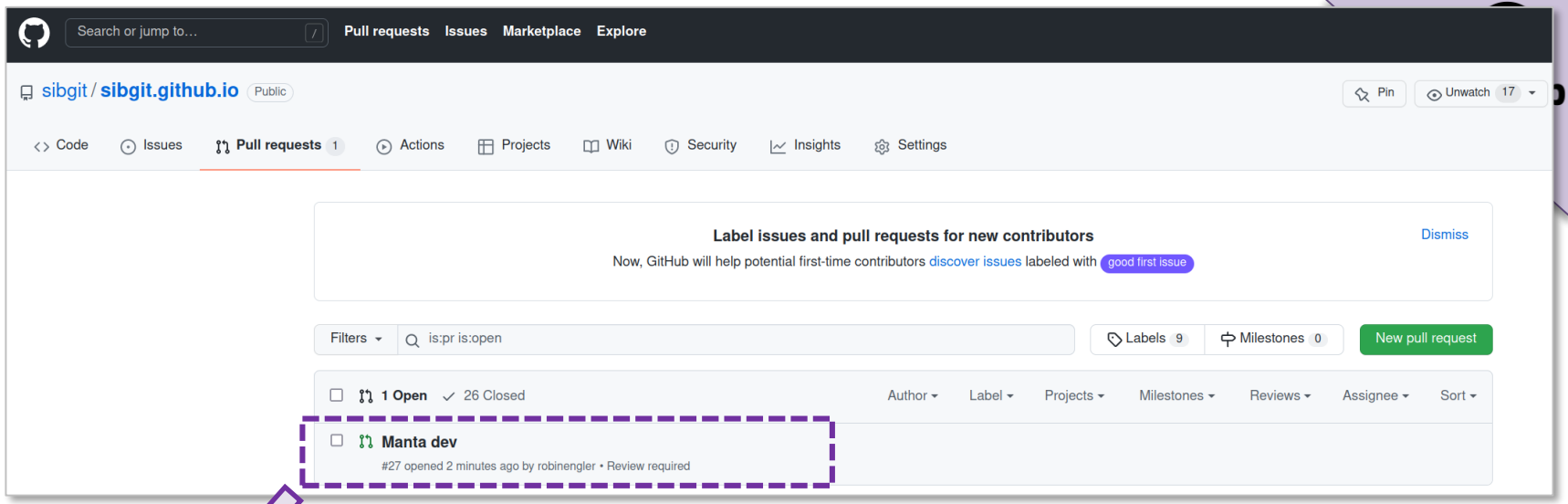
Merging is blocked
Merging can be performed automatically with 1 approving review.

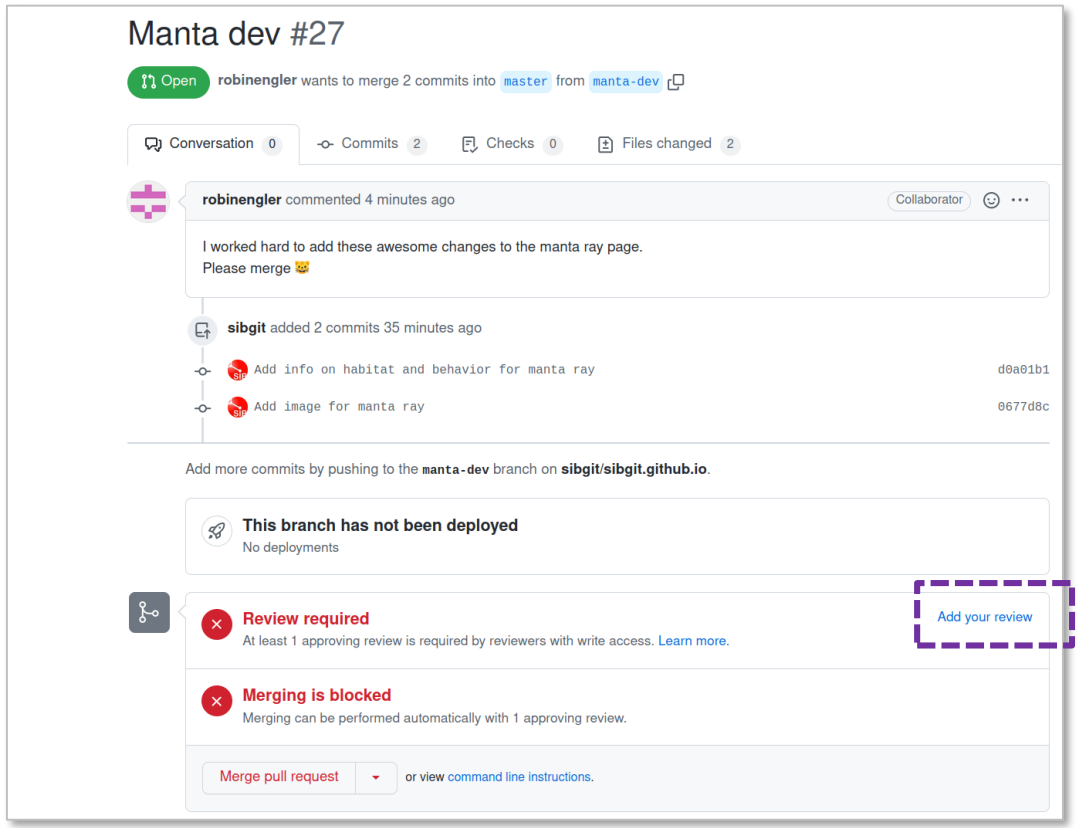
Merge pull request | or view command line instructions.

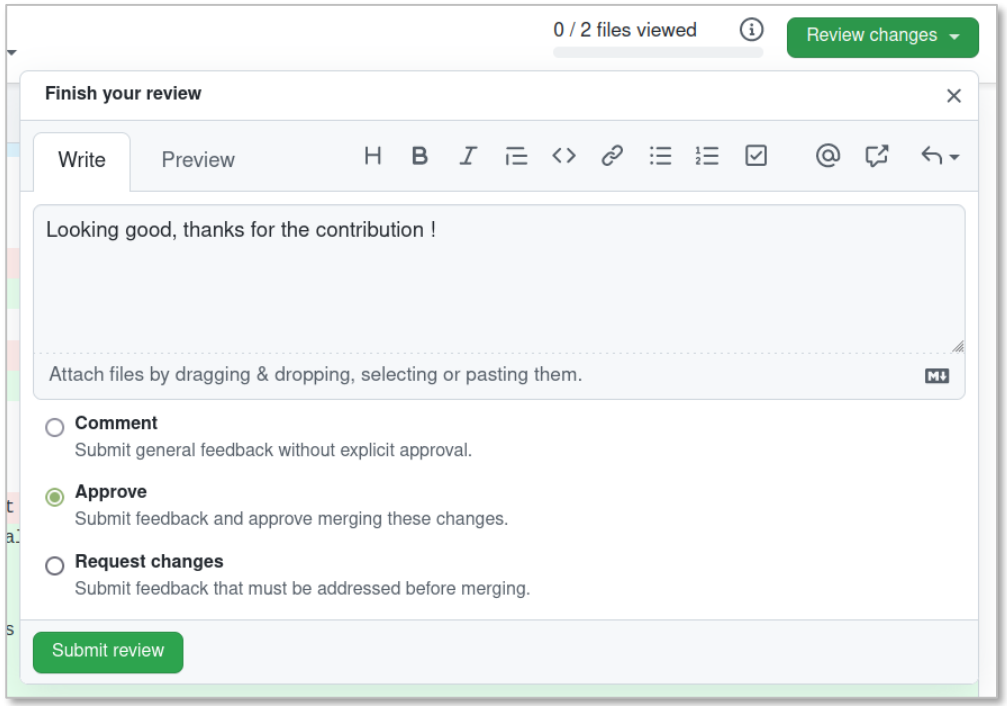
GitHub slide ...



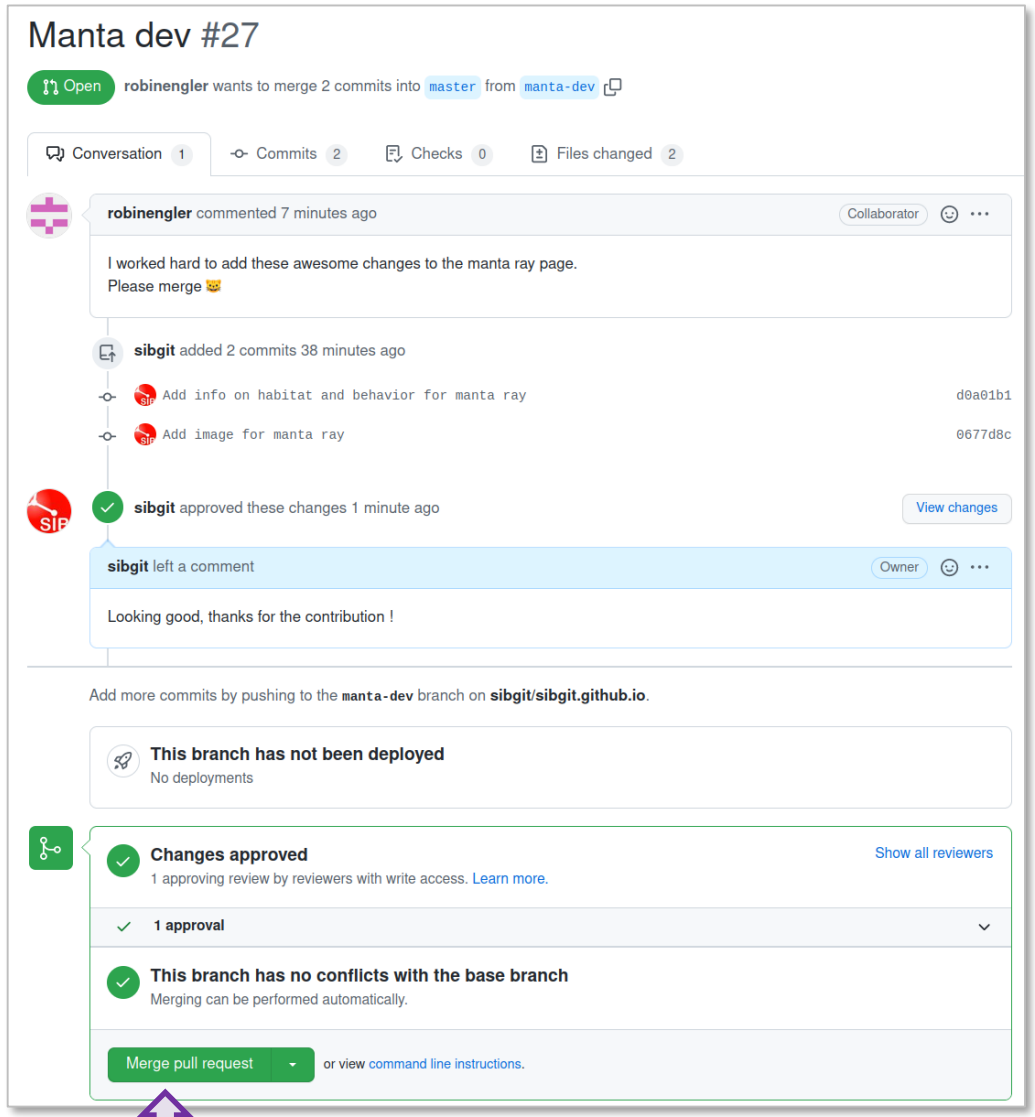
The **reviewer** of your PR will then have a look at your changes (the modifications introduced with your commits) and **approve them or request changes**.



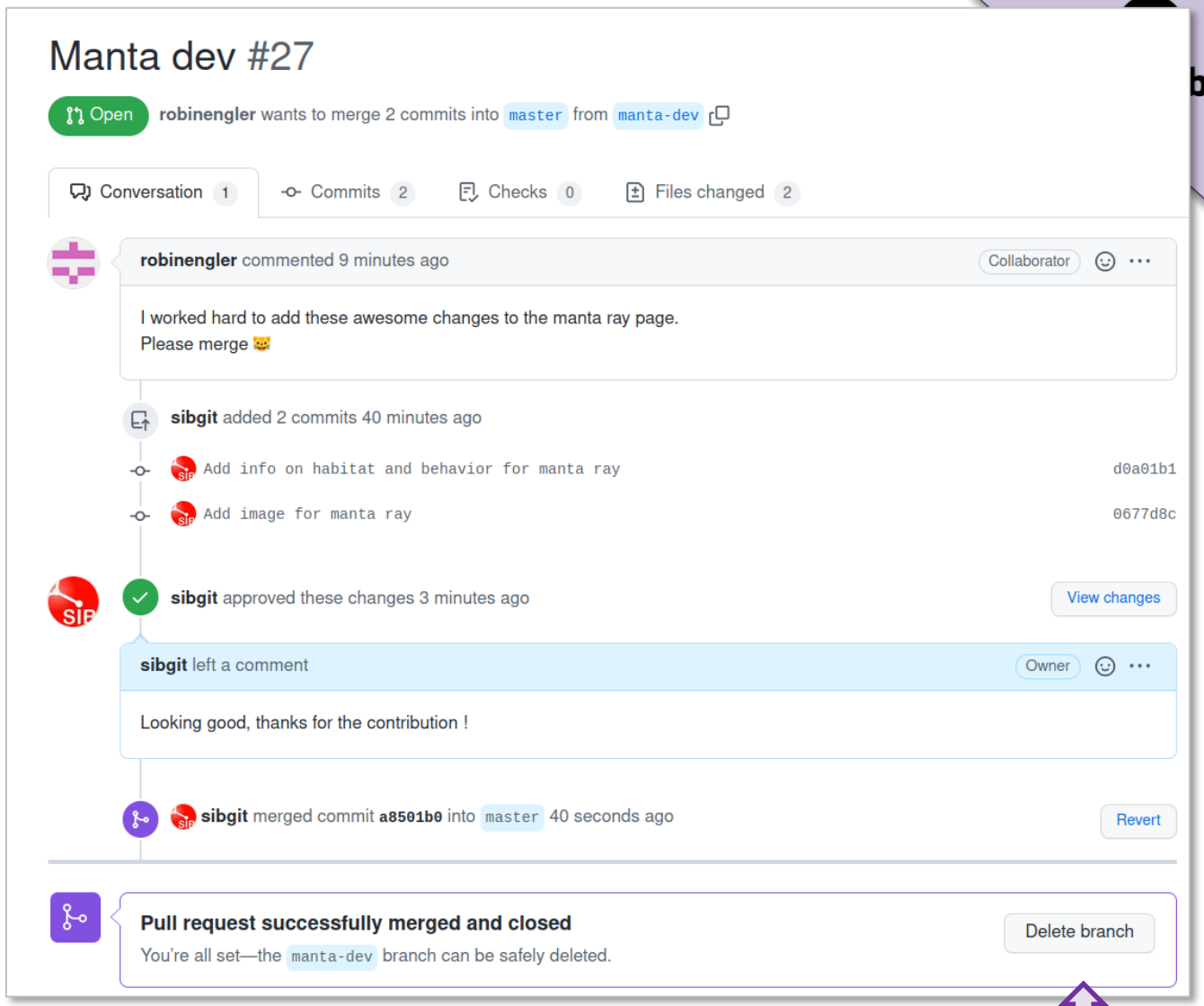




GitHub slide ...



Now that the pull request is approved, it can be merged (either by the reviewer or by you) by clicking **Merge pull request**.



Completed ! Optionally, you can **delete your branch** on the remote (this will not delete it locally).

GitHub slide ...

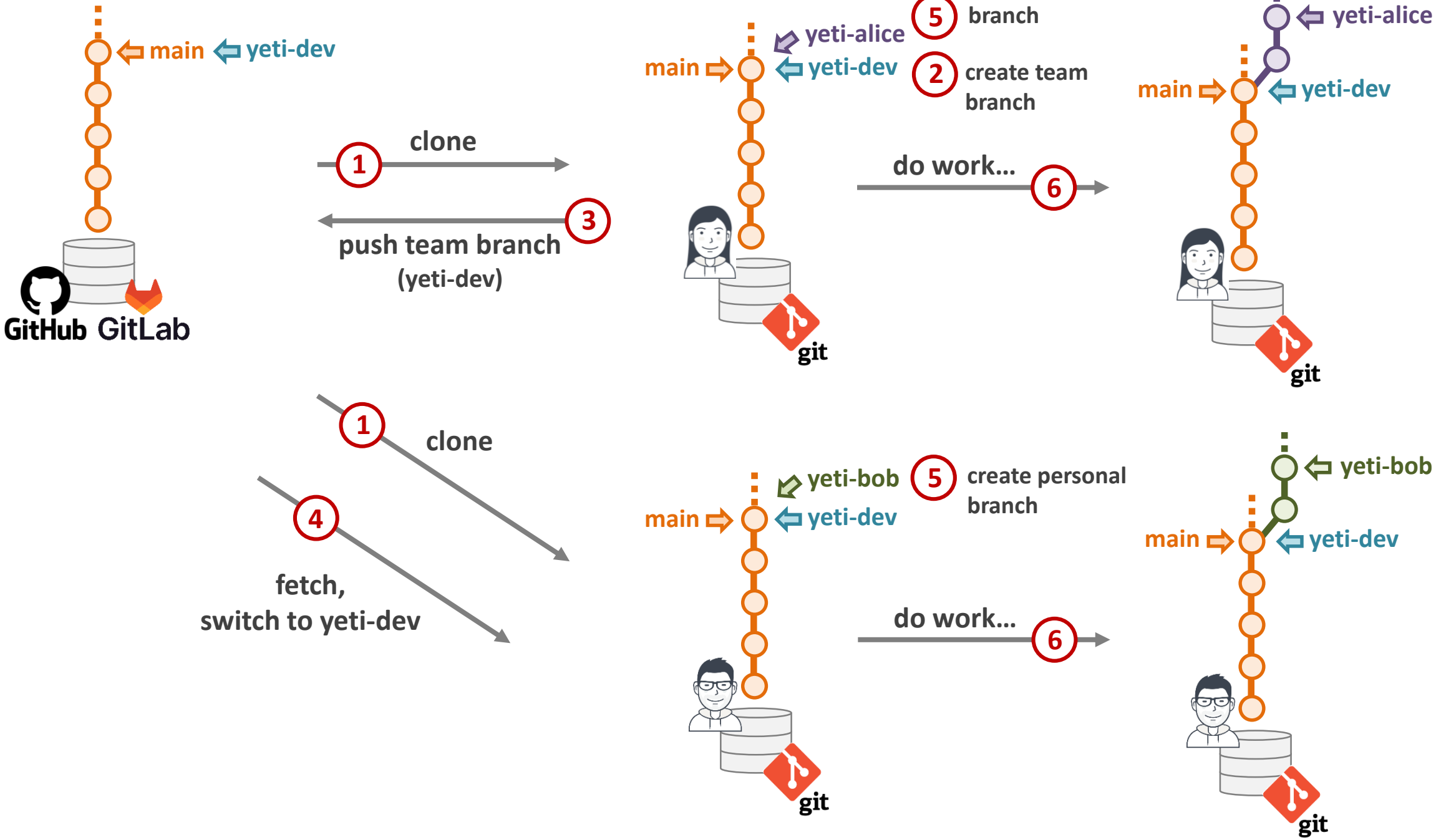
exercise 5

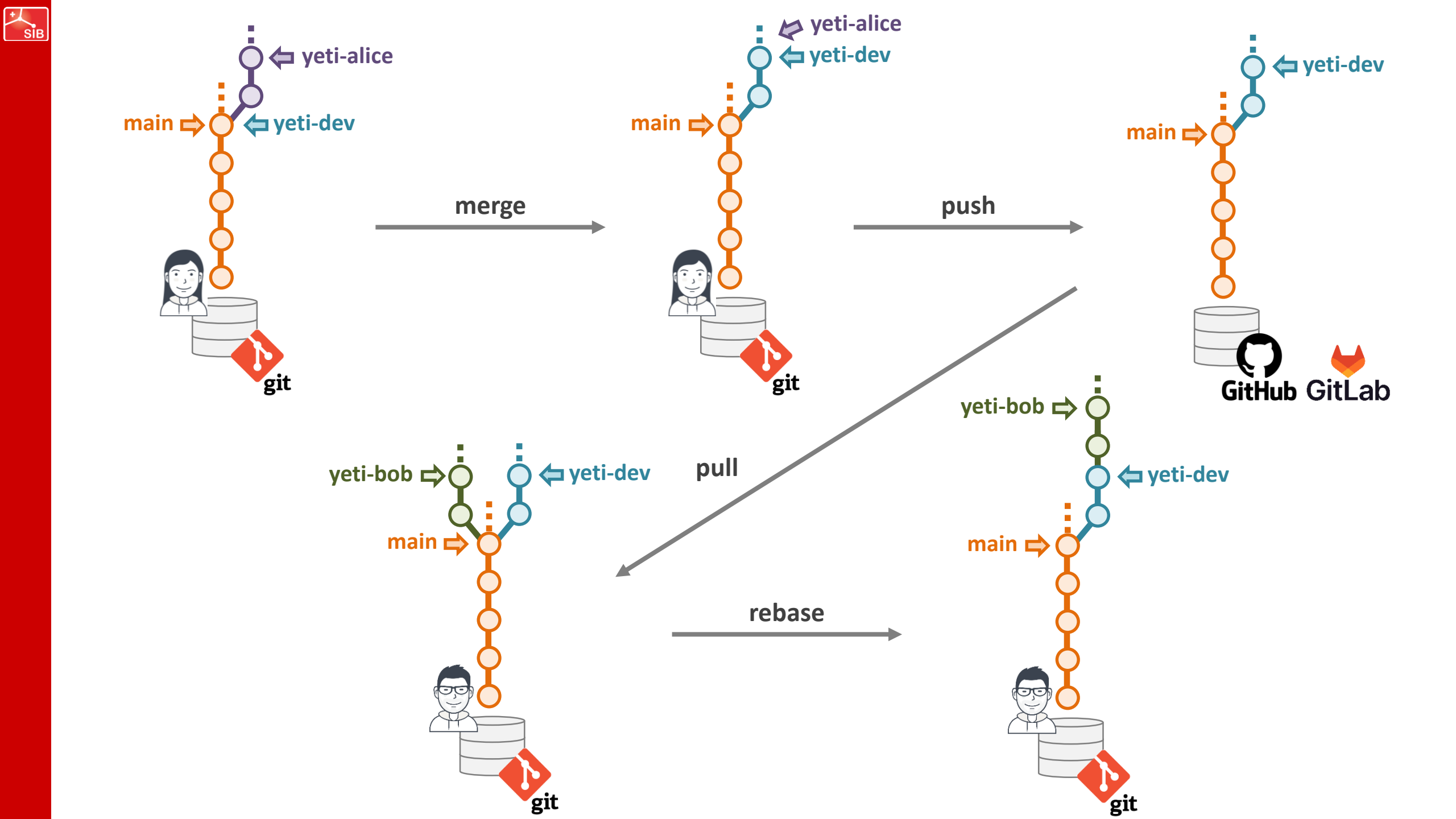
The Awesome Animal Awareness Project

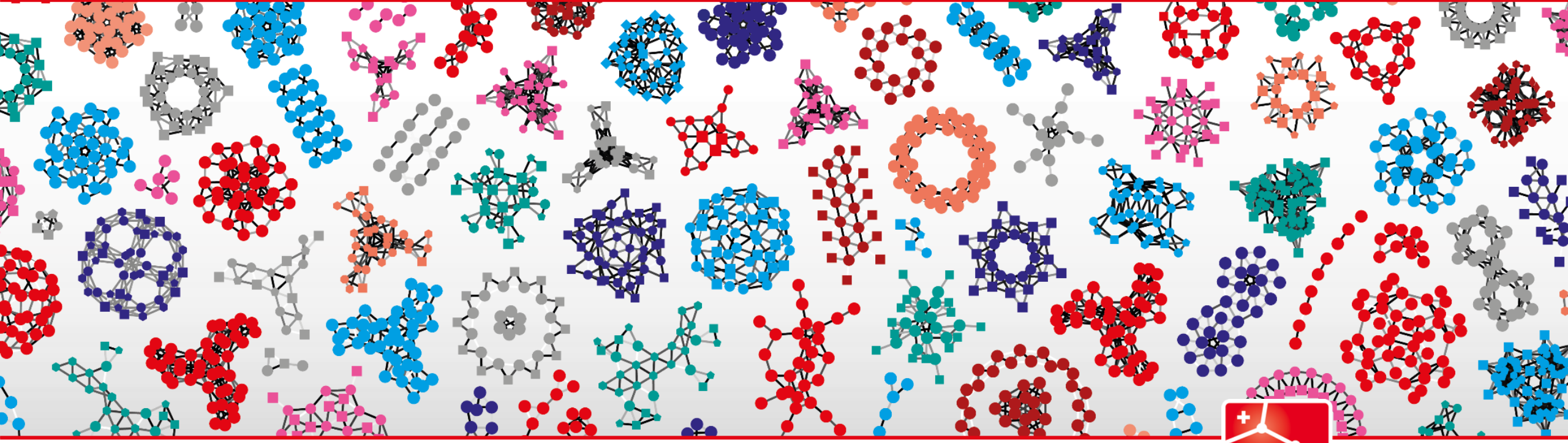


This exercise has helper slides

Exercise 5 workflow: branch – rebase – merge sequence




yeti-alice
main
yeti-dev
git
merge
yeti-alice
yeti-dev
main
git
push
yeti-dev
main
GitHub
GitLab
pull
yeti-bob
yeti-dev
main
git
rebase
yeti-bob
yeti-dev
main
git



Thank you for attending this course

SIB
Swiss Institute of Bioinformatics